Pestaña 1

“UNIVERSIDAD AUTÓNOMA GABRIEL RENÉ MORENO”

“FACULTAD DE INGENIERÍA EN CIENCIAS DE LA COMPUTACIÓN Y TELECOMUNICACIONES”

Grupo #1

“Sistema de Información Web y Móvil para la Gestión de Servicios de Auxilio Vehicular con Suministro de Repuestos y Asistencias Rápidas”

- Chavez Melgar Tatiana — Ing. Informática
- Silvestre Carrasco Noelia Stephanie — Ing. Sistemas

MATERIA:	Sistemas de Información II
SIGLA:	INF342 - SA
DOCENTE:	ING. Garzon Cuellar Angelica
GESTION:	1 - 2026{{

ÍNDICE

Parte I – Fundamentación Teórica....4
1) Perfil....4
1.1 Introducción....4
1.2 Objetivo General....5
1.3 Objetivos Específicos....5
1.4 Descripción del problema....6
1.4 Alcance y Funcionalidades del Sistema....6
1.4.1 Aplicación móvil (Cliente)....6
1.4.2 Aplicación web (Talleres)....7
1.4.3 Backend (FastAPI)....8
1.4.4 Base de datos (PostgreSQL)....8
1.4.5 Módulos de Inteligencia Artificial 1. Procesamiento de audio....9
1.4.6 Sistema de asignación inteligente....9
1.4.7 Notificaciones....9
1.5. CARACTERÍSTICAS DEL SISTEMA....10
1.6. STACK TECNOLOGICO....10
1.7. EJEMPLOS DE SITUACIONES DE USO....10
1.8. CONSIDERACIONES IMPORTANTES....11
Parte II. Marco Teórico....11
2.1. Marco Teórico Referencial....11
2.1.1. Antecedentes Internacionales....12
2.1.2. Antecedentes Nacionales....13
2.1.3. Justificación del Proyecto Basada en Referencias....13
2.1.4. Arquitectura Cliente- Servidor....14
2.1.5. Patrón de Arquitectura....15
2.1.6 Metodología de Desarrollo de Software....16
2.1.7. Tecnologías Utilizadas....17
2.1.8. Base de Datos Relacional....18
2.1.9. Lenguaje Python y Framework FastApi....19
2.2. Datos....20
2.3. Procesos....21
2.3.1 Módulo de Autenticación y Seguridad....21
2.3.2. Módulo de Gestión de Usuarios....22
2.3.3. Módulo de Gestión de Vehículos....22
2.3.4. Módulo de Gestión de Emergencias....23
2.3.5. Módulo de Gestión de Talleres....23
2.3.6. Módulo de Gestión de Pagos....24
2.3.7. Administración del Sistema....25
2.3.8. Bitácora del Sistema....25
Parte III – Proceso de desarrollo....26

3.1) FLUJO DE TRABAJO: CAPTURA DE REQUISITOS..... 26
3.1.1 Identificar Actores y Casos de Uso..... 26
3.1.1.1 Actores..... 26
3.1.2 Tablas de Casos de Uso y sus Prioridades..... 28
CICLO #1..... 30
CICLO #2..... 30
CICLO #3..... 31
3.1.3 Detallar un Caso de Uso..... 32
3.1.4 Estructura de los Casos de Uso..... 51
3.2) FLUJO DE TRABAJO: ANÁLISIS..... 54
3.2.1 Análisis de Arquitectura..... 54
3.2.1.1 Identificar Paquetes..... 54
3.2.1.2 Relacion de Paquetes y Casos de Uso..... 56
3.2.1.3 Vista de Paquete (Encapsular CU)..... 59
3.1.2 Analizar un Caso de Uso..... 64
3.1.2.1 Diagrama de Comunicacion..... 64
Ciclo #1..... 64
Ciclo #2..... 66
Ciclo # 3..... 68
3.1.3 Análisis de Clase..... 69
Ciclo #1..... 69
Ciclo #2..... 72
Ciclo #3..... 74
3.1.4 Análisis de Paquetes..... 76
3.3) FLUJO DE TRABAJO: DISEÑO..... 77
3.3.1 Diseño de Arquitectura..... 77
3.3.1.2 Diseño Lógico (Diagrama Organizado en Capas)..... 78
3.3.2.1. Diseño de Datos Lógico..... 78
Diagrama de Clases..... 78
DIAGRAMA DE CLASE..... 78
Mapeo y Normalización..... 79
MAPEO..... 79
3.3.2.2 Diseño de Datos Físico..... 80
TABLA DE VOLUMEN..... 80
SCRIPT..... 86
DIAGRAMA RELACIONAL..... 91
ACTUALIZACIÓN DE TUPLAS..... 91
CONSULTAS..... 95
PROCEDIMIENTOS ALMACENADOS..... 97
TRIGGERS..... 103
3.3.1.3 Diseñar Casos de Uso (Diagramas de Secuencia)..... 105
3.4) FLUJO DE TRABAJO: IMPLEMENTACIÓN..... 120
3.4.1 Elección de Plataforma de Desarrollo de Software..... 120
Lenguaje de Programación..... 120

Framework Backend: FastApi..... 121
Librería Frontend: Angular.....121
Framework Móvil: Flutter..... 121
Bases de Datos..... 122
Entorno de Desarrollo.....122
3.4.2 Implementación de la Arquitectura del Sistema..... 123
3.4.3 Implementación de la Arquitectura de Subsistemas..... 124
3.5) FLUJO DE TRABAJO: PRUEBAS..... 125
3.5.1 Planificar Pruebas..... 125
3.5.1.1 Objetivo de las Pruebas..... 125
3.5.1.2 Estrategia de Pruebas.....126
Pruebas de Validación (Caja Negra)..... 126
3.5.1.3 Entorno de Pruebas.....126
3.5.2 Casos de Pruebas.....127
CONCLUSIÓN..... 164
RECOMENDACIÓN.....164
BIBLIOGRAFÍA..... 165
URL Y QR..... 166

Parte I – Fundamentación Teórica

1) Perfil

1.1 Introducción

En la actualidad, el crecimiento del parque automotor y la constante movilidad en entornos urbanos y carreteras han incrementado la probabilidad de que los conductores enfrenten situaciones imprevistas relacionadas con fallas mecánicas, accidentes leves o problemas técnicos en sus vehículos. Estas situaciones generan la necesidad de contar con servicios de asistencia rápida, eficiente y confiable que permitan resolver emergencias vehiculares en el menor tiempo posible.

Sin embargo, los mecanismos tradicionales para solicitar ayuda presentan diversas limitaciones, tales como la dependencia de llamadas telefónicas, la falta de información precisa sobre el incidente, tiempos de respuesta variables y la dificultad para identificar proveedores adecuados. Asimismo, los talleres mecánicos no disponen de herramientas

tecnológicas que les permitan gestionar solicitudes de manera organizada, priorizar casos y optimizar la asignación de sus recursos.

En este contexto, surge la necesidad de desarrollar una plataforma tecnológica que integre a los usuarios con los talleres mecánicos mediante un sistema inteligente de atención de emergencias vehiculares. Este sistema propone el uso de tecnologías modernas como aplicaciones web y móviles, geolocalización en tiempo real y módulos de inteligencia artificial capaces de procesar información multimodal (imagen, audio y texto), con el fin de mejorar el diagnóstico preliminar de los incidentes, su clasificación y priorización.

Además, la incorporación de mecanismos automatizados de asignación de servicios y notificaciones en tiempo real permitirá optimizar la atención de emergencias, garantizando mayor eficiencia, trazabilidad y calidad en el servicio. De esta manera, el sistema no solo facilitará la experiencia del usuario, sino que también contribuirá a mejorar la gestión operativa de los talleres, consolidando una solución integral e innovadora en el ámbito de la asistencia vehicular.

1.2 Objetivo General

Desarrollar una plataforma inteligente de atención de emergencias vehiculares que permita conectar usuarios con talleres mecánicos mediante el análisis automatizado de incidentes utilizando datos multimodales (imagen, audio, texto y geolocalización), optimizando el proceso de diagnóstico preliminar, priorización y asignación del servicio.

1.3 Objetivos Específicos

- Diseñar una arquitectura basada en servicios que soporte procesamiento en tiempo real.
- Implementar una aplicación móvil para usuarios que permita reportar emergencias vehiculares.
- Diseñar una aplicación web para talleres que gestione solicitudes y operaciones.
- Integrar mecanismos de geolocalización para ubicar incidentes y proveedores.
  Incorporar módulos de inteligencia artificial para:
  - transcripción de audio
  - clasificación de incidentes
  - análisis básico de imágenes

- Diseñar un sistema de priorización de emergencias.
- Implementar un mecanismo de asignación inteligente de talleres.
- Gestionar notificaciones en tiempo real (push).
- Mantener trazabilidad completa de cada incidente.

1.4 Descripción del problema

En entornos urbanos y carreteras, los conductores frecuentemente enfrentan situaciones imprevistas como fallas mecánicas, pinchazos de llantas, problemas de batería, sobrecalentamiento del motor o accidentes leves, perder la llave del vehículo, dejar llave dentro el vehículo y otros.

En muchos casos, el proceso de conseguir ayuda es ineficiente, lento y poco confiable.

Actualmente, las alternativas existentes presentan limitaciones como:

- dependencia de llamadas telefónicas
- falta de información clara sobre el problema
- tiempos de respuesta impredecibles
- dificultad para identificar el proveedor adecuado
- ausencia de trazabilidad del servicio

Por otro lado, los talleres mecánicos no cuentan con una plataforma estructurada que les permita:

- recibir solicitudes de manera organizada
- evaluar rápidamente la naturaleza del problema
- priorizar casos
- optimizar la asignación de recursos

La plataforma debe integrar múltiples fuentes de información (imágenes, audio, ubicación) para asistir en la clasificación automática del incidente y facilitar la toma de decisiones.

1.4 Alcance y Funcionalidades del Sistema

1.4.1 Aplicación móvil (Cliente)

El usuario deberá poder:

Los clientes se deben registrar en la aplicación, como también registrar sus vehículos

Registro de emergencia

- enviar ubicación en tiempo real
- adjuntar fotos del vehículo

- enviar audio describiendo el problema
- ingresar texto adicional opcional

Gestión de solicitudes

- visualizar estado de su solicitud:

  - pendiente

  - en proceso

  - atendido

- ver taller asignado
- ver tiempo estimado de llegada

Interacción

- recibir notificaciones push
- comunicarse con el taller (opcional)

El cliente debe realizar los pagos correspondientes desde la aplicación

1.4.2 Aplicación web (Talleres)

Los Talleres se deben registrar para que sean los que provean el servicio de asistencia a los clientes que lo soliciten, para esto un taller puede tener uno o más técnicos quienes serán los asignados para asistir a un cliente

Cuando un taller recibe una alerta de asistencia revisa el lugar donde se encuentran sus técnicos y disponibilidad para asignarle la orden correspondiente según el tipo de percance y ubicación del cliente que solicita asistencia

El taller debe pagar un porcentaje (10%) del precio cobrado a la plataforma como comisión

Los talleres deberán poder:

Gestión de solicitudes

- visualizar solicitudes disponibles
- ver información estructurada del incidente
- aceptar o rechazar solicitudes

Operación

- actualizar estado del servicio

- gestionar disponibilidad
- visualizar historial de atenciones

Información enriquecida (IA)

- ver resumen automático del incidente
- ver clasificación del problema
- ver nivel de prioridad

1.4.3 Backend (FastAPI)

El sistema backend deberá gestionar:

- autenticación y autorización
- gestión de usuarios y talleres
- gestión de incidentes
- procesamiento de datos
- integración con módulos de IA
- motor de asignación
- sistema de notificaciones
- APIs REST

1.4.4 Base de datos (PostgreSQL)

El modelo de datos deberá contemplar:

- usuarios
- talleres
- vehículos
- incidentes
- evidencias (imagen, audio, texto)
- estados del servicio
- historial
- métricas

Debe considerar:

- integridad de datos
- relaciones complejas
- trazabilidad

1.4.5 Módulos de Inteligencia Artificial

1. Procesamiento de audio

- conversión de audio a texto
- extracción de información relevante

2. Clasificación de incidentes (Imágenes a través de fotos lo clasifica, visión artificial)
Categorización automática del problema:

- batería
- llanta
- choque
- motor
- otros

3. Análisis de imágenes (básico)

- identificación de daños visibles
- apoyo en la clasificación del incidente

4. Generación de resumen

creación automática de una ficha estructurada del incidente

1.4.6 Sistema de asignación inteligente

Debe considerar:

- ubicación del incidente
- tipo de problema
- disponibilidad del taller
- capacidad del taller
- distancia

prioridad del caso

El sistema debe generar:

- lista de talleres candidatos
- selección del más adecuado

1.4.7 Notificaciones

- notificaciones push al cliente
- notificaciones a talleres
- actualizaciones en tiempo real

1.5. CARACTERÍSTICAS DEL SISTEMA

- integración de múltiples fuentes de datos
- uso de inteligencia artificial aplicada al flujo del sistema
- diseño de interfaces centrado en experiencia de usuario

1.6. STACK TECNOLOGICO

- frontend web: Angular
- backend: FastAPI (framework) - phyton
- base de datos: PostgreSQL
- aplicación móvil: Flutter

1.7. EJEMPLOS DE SITUACIONES DE USO

Caso 1: Problema de batería

Un usuario se encuentra en un estacionamiento y su vehículo no enciende. Envía:

- ubicación
- audio indicando que el auto no responde
- foto del tablero

El sistema:

- transcribe el audio
- clasifica como “problema de batería”
- asigna prioridad media
- sugiere talleres con servicio de auxilio eléctrico

Caso 2: Pinchazo de llanta

El usuario reporta:

- foto de llanta dañada
- ubicación en carretera

El sistema:

- detecta posible pinchazo
- clasifica como incidente leve
- asigna prioridad media
- sugiere talleres cercanos con servicio móvil

Caso 3: Accidente leve

El usuario envía:

- múltiples fotos

- audio indicando choque

El sistema:

- detecta daño visible
- clasifica como choque
- asigna prioridad alta
- sugiere talleres con capacidad de remolque

Caso 4: Situación ambigua

El usuario envía información poco clara.

El sistema:

- clasifica como “incierto”
- solicita más información
- permite intervención manual

1.8. CONSIDERACIONES IMPORTANTES

- La IA no debe considerarse un módulo aislado, sino parte del flujo principal.
- El sistema debe manejar incertidumbre (no todos los casos serán claros

Parte II. Marco Teórico

2.1. Marco Teórico Referencial

El presente marco teórico tiene como finalidad sustentar el desarrollo del sistema AutoGo, una plataforma web y móvil orientada a la gestión de servicios de auxilio vehicular, suministro de repuestos y asistencias rápidas. El sistema busca conectar a los clientes que presentan emergencias con talleres mecánicos y técnicos disponibles, utilizando herramientas tecnológicas como geolocalización, procesamiento de imágenes, audio, texto, notificaciones en tiempo real, pagos digitales y registro histórico de las atenciones realizadas.

AutoGo responde a una problemática frecuente en entornos urbanos y carreteras, donde los conductores pueden enfrentar fallas mecánicas, pinchazos de llantas, problemas de batería, sobrecalentamiento del motor, accidentes leves u otras situaciones imprevistas. En muchos casos, el proceso para conseguir ayuda suele ser lento, desorganizado y poco confiable, ya que depende de llamadas telefónicas, contactos informales o búsquedas manuales de talleres cercanos. Esta situación genera incertidumbre en el cliente, dificulta la localización exacta del incidente y limita la capacidad de los talleres para atender solicitudes de manera ordenada.

Desde el punto de vista de los sistemas de información, AutoGo se plantea como una solución integral que permite capturar, procesar, almacenar y distribuir información relevante para apoyar la toma de decisiones. La plataforma registra datos de usuarios, vehículos,

talleres, técnicos, emergencias, evidencias multimedia, pagos, estados del servicio e historial de atención. Esta información permite mejorar la trazabilidad del proceso, desde el momento en que el cliente reporta una emergencia hasta que el servicio es atendido, finalizado y pagado.

Asimismo, el sistema incorpora conceptos actuales de transformación digital, tales como arquitectura cliente-servidor, aplicaciones móviles, aplicaciones web, APIs REST, base de datos relacional, servicios en la nube e inteligencia artificial. Estos elementos permiten construir una plataforma escalable, organizada y orientada a mejorar la eficiencia en la atención de emergencias vehiculares.

2.1.1. Antecedentes Internacionales

A nivel internacional, los servicios de asistencia vehicular han evolucionado hacia modelos digitales bajo demanda, en los cuales los usuarios pueden solicitar ayuda desde una aplicación móvil, compartir su ubicación en tiempo real y recibir información sobre el proveedor asignado. Este tipo de plataformas permite reducir los tiempos de espera, mejorar la comunicación entre el cliente y el proveedor, y registrar el historial de cada atención realizada.

En países con mayor desarrollo tecnológico, existen aplicaciones orientadas a la asistencia en carretera, solicitud de grúas, cambio de llantas, carga de batería, desbloqueo de vehículos y auxilio mecánico. Estas soluciones funcionan mediante plataformas digitales que conectan al usuario con prestadores de servicios cercanos, utilizando geolocalización, sistemas de asignación automática y seguimiento del estado de la solicitud.

También se han incorporado tecnologías de inteligencia artificial en procesos relacionados con la atención de emergencias y servicios técnicos. Por ejemplo, el procesamiento automático de audio permite transformar mensajes de voz en texto para analizar con mayor claridad la descripción del problema. De igual manera, la visión artificial puede apoyar en la identificación preliminar de daños visibles en imágenes enviadas por el usuario, tales como llantas dañadas, golpes, fallas visibles o partes afectadas del vehículo.

Estos antecedentes son importantes para AutoGo porque demuestran que la atención vehicular puede mejorar mediante plataformas digitales que integran geolocalización, comunicación en tiempo real, clasificación automática de incidentes y trazabilidad del servicio. En este sentido, AutoGo adapta estos principios al contexto local, ofreciendo una solución que permite a los clientes registrar emergencias de forma más precisa y a los talleres organizar mejor sus recursos disponibles.

2.1.2. Antecedentes Nacionales

En el contexto nacional, los servicios de auxilio vehicular suelen gestionarse de manera tradicional, principalmente mediante llamadas telefónicas, contactos de talleres, recomendaciones personales, grupos de mensajería o búsquedas informales en redes sociales. Este modelo presenta limitaciones importantes, ya que no siempre permite conocer con precisión la ubicación del cliente, el tipo de problema, la disponibilidad del taller o el tiempo estimado de atención.

En muchas situaciones, el conductor que sufre una emergencia vehicular debe explicar manualmente el problema, buscar un taller cercano y esperar una respuesta sin contar con información clara sobre el estado del servicio. Esto puede generar pérdida de tiempo, inseguridad e incertidumbre, especialmente cuando el incidente ocurre en una zona poco conocida, en carretera o en horarios de baja disponibilidad.

Por otro lado, los talleres mecánicos no siempre cuentan con una plataforma estructurada para recibir solicitudes, gestionar técnicos disponibles, priorizar casos o registrar el historial de servicios atendidos. La falta de herramientas digitales dificulta la organización interna del taller y limita la posibilidad de brindar una atención más rápida y controlada.

Frente a esta situación, AutoGo propone una alternativa tecnológica orientada a digitalizar la atención de emergencias vehiculares en el entorno nacional. El sistema permite centralizar la información, registrar solicitudes de auxilio, compartir ubicación, adjuntar evidencias, asignar talleres, actualizar estados, gestionar pagos y mantener trazabilidad completa del servicio. Esto representa una mejora frente a los mecanismos tradicionales de atención vehicular.

2.1.3. Justificación del Proyecto Basada en Referencias

El desarrollo de AutoGo se justifica por la necesidad de contar con una plataforma tecnológica que mejore la gestión de emergencias vehiculares, tanto para los clientes como para los talleres mecánicos. Actualmente, el proceso tradicional de solicitar auxilio puede ser ineficiente debido a la dependencia de llamadas telefónicas, la falta de información clara, los tiempos de respuesta impredecibles, la dificultad para identificar proveedores adecuados y la ausencia de trazabilidad del servicio.

AutoGo busca solucionar estas limitaciones mediante una plataforma web y móvil que permita registrar emergencias con información estructurada. El cliente podrá enviar su ubicación en tiempo real, adjuntar imágenes del vehículo, grabar audio describiendo el problema e ingresar texto adicional. Esta información permite que el sistema y los talleres cuenten con una visión más clara del incidente antes de realizar la atención.

La incorporación de inteligencia artificial también justifica el valor del proyecto, ya que permite apoyar el diagnóstico preliminar del problema mediante transcripción de audio, clasificación de incidentes, análisis básico de imágenes y generación de una ficha estructurada. Según los requisitos del sistema, la plataforma debe incorporar módulos de IA para procesar audio, clasificar incidentes, analizar imágenes y generar resúmenes automáticos del caso.

Asimismo, el sistema permite mejorar la gestión operativa de los talleres, ya que estos podrán visualizar solicitudes disponibles, revisar información estructurada del incidente, aceptar o rechazar servicios, gestionar disponibilidad de técnicos, actualizar estados y consultar historial de atenciones. De esta manera, AutoGo no solo beneficia al cliente final, sino también a los proveedores del servicio.

En conclusión, el proyecto se justifica porque cubre una necesidad real de digitalización en el sector de asistencia vehicular, mejora la comunicación entre clientes y talleres, reduce la incertidumbre del proceso, facilita la asignación del proveedor adecuado y mantiene un registro completo de cada incidente atendido.

2.1.4. Arquitectura Cliente- Servidor

La arquitectura cliente-servidor es un modelo de organización de software en el cual los clientes realizan solicitudes a un servidor central, y este procesa la información para devolver una respuesta. En este modelo, el cliente se encarga de la interacción con el usuario, mientras que el servidor concentra la lógica de negocio, la autenticación, la conexión con la base de datos y el procesamiento de las operaciones principales.

En AutoGo, esta arquitectura se aplica mediante tres componentes principales: la aplicación móvil, la aplicación web y el backend. La aplicación móvil funciona como cliente para los usuarios que solicitan auxilio vehicular. A través de ella, el cliente puede registrarse, iniciar sesión, registrar sus vehículos, reportar emergencias, enviar ubicación, adjuntar imágenes o audios, consultar el estado de su solicitud y realizar pagos.

La aplicación web funciona como cliente para los talleres y administradores del sistema. Desde esta interfaz, los talleres pueden gestionar solicitudes, revisar información de incidentes, administrar técnicos, actualizar disponibilidad, aceptar o rechazar servicios y consultar historial de atenciones. Por su parte, el administrador puede supervisar usuarios, talleres, incidentes y registros del sistema.

El servidor está representado por el backend desarrollado con FastAPI. Este componente recibe las solicitudes enviadas desde la aplicación móvil y la aplicación web, valida los datos, aplica las reglas de negocio, se comunica con la base de datos PostgreSQL e integra los módulos de inteligencia artificial. Además, el backend expone APIs REST que permiten la comunicación entre los distintos clientes y el sistema central.

Esta arquitectura resulta adecuada para AutoGo porque permite separar responsabilidades, mejorar la seguridad, facilitar el mantenimiento y permitir que diferentes plataformas consuman los mismos servicios del backend. Además, favorece la escalabilidad del sistema, ya que la aplicación web, la aplicación móvil y el servidor pueden evolucionar de manera independiente.

2.1.5. Patrón de Arquitectura

AutoGo utiliza un patrón de arquitectura organizado por capas y servicios, debido a que el sistema está compuesto por una aplicación móvil, una aplicación web, un backend, una base de datos y módulos de inteligencia artificial. Este enfoque permite separar las responsabilidades del sistema y organizar el código de forma modular, mantenible y escalable.

La primera capa corresponde a la **capa de presentación**, donde se encuentran las interfaces utilizadas por los usuarios. En esta capa se ubica la aplicación móvil desarrollada para los clientes y la aplicación web destinada a talleres y administradores. Su función principal es mostrar información, recibir datos del usuario y enviar solicitudes al backend.

La segunda capa corresponde a la **capa de servicios o API**, implementada mediante FastAPI. Esta capa se encarga de recibir las solicitudes provenientes de la aplicación móvil y web, validar la información, controlar el acceso, ejecutar procesos y devolver respuestas estructuradas. También actúa como intermediaria entre la interfaz del usuario, la base de datos y los módulos de inteligencia artificial.

La tercera capa corresponde a la **capa de lógica de negocio**, donde se encuentran las reglas principales del sistema. En esta capa se gestionan procesos como el registro de emergencias, la clasificación del incidente, la priorización, la generación de talleres candidatos, la asignación del servicio, la actualización de estados, el cálculo de pagos y el registro de historial.

La cuarta capa corresponde a la **capa de datos**, donde se encuentra PostgreSQL como sistema gestor de base de datos relacional. Esta capa almacena usuarios, vehículos, talleres, técnicos, incidentes, historial y pagos.

Finalmente, se considera una **capa de inteligencia artificial**, encargada de apoyar el procesamiento de audio, imágenes y texto. Esta capa permite generar información enriquecida para mejorar la toma de decisiones durante el registro y atención de emergencias.

Este patrón de arquitectura es adecuado para AutoGo porque permite que cada módulo tenga una función clara, facilita el mantenimiento del sistema y permite incorporar nuevas funcionalidades en el futuro, como nuevas formas de pago, mejores modelos de clasificación o reportes administrativos avanzados.

2.1.6 Metodología de Desarrollo de Software

Para el desarrollo de AutoGo se considera el **Proceso Unificado de Desarrollo de Software (PUDS)**, debido a que el documento de requisitos indica que el proceso de desarrollo debe seguir los flujos de trabajo de captura de requisitos, análisis, diseño, implementación y pruebas, utilizando modelos UML para representar la estructura y comportamiento del sistema.

El PUDS es una metodología iterativa e incremental que permite construir el sistema de forma progresiva. Esto significa que el proyecto no se desarrolla completamente en una sola etapa, sino mediante ciclos de trabajo en los cuales se priorizan los casos de uso más importantes y se avanza gradualmente en el análisis, diseño, implementación y validación de las funcionalidades.

En AutoGo, la metodología se aplica mediante los siguientes flujos de trabajo:

Captura de requisitos: en esta etapa se identifican los actores principales del sistema, los casos de uso, sus prioridades, riesgos y ciclos de desarrollo. Los actores principales son Cliente, Taller, Técnico Mecánico y Administrador del Sistema. Cada actor participa en distintos procesos, como registro, gestión de vehículos, reporte de emergencias, gestión de talleres, pagos, administración y bitácora.

Análisis: en esta etapa se estudia la estructura lógica del sistema, se identifican paquetes funcionales, se relacionan los casos de uso con los módulos correspondientes y se elaboran diagramas de comunicación y análisis de clases. Esto permite comprender cómo interactúan los componentes del sistema antes de pasar al diseño.

Diseño: en esta etapa se define la arquitectura física y lógica del sistema, el diseño de datos, el modelo relacional, los diagramas de secuencia y la estructura de la base de datos. También se establecen las relaciones entre entidades como usuarios, vehículos, talleres, técnicos, incidentes, pagos e historial.

Implementación: en esta etapa se construyen los módulos del sistema utilizando las tecnologías seleccionadas: Angular para la aplicación web, Flutter para la aplicación móvil, FastAPI con Python para el backend y PostgreSQL como base de datos.

Pruebas: en esta etapa se verifica que cada caso de uso funcione correctamente, validando entradas, condiciones, procedimientos y resultados esperados. Las pruebas permiten comprobar que el sistema cumple con los requisitos funcionales establecidos.

El uso del PUDS permite que AutoGo sea desarrollado de manera ordenada, documentada y verificable, facilitando la comprensión del proyecto y su defensa académica.

2.1.7. Tecnologías Utilizadas

El sistema AutoGo utiliza un conjunto de tecnologías modernas orientadas al desarrollo de aplicaciones web, móviles, backend, base de datos e inteligencia artificial. Estas herramientas fueron seleccionadas considerando la necesidad de construir una plataforma escalable, segura, mantenible y capaz de procesar información en tiempo real.

Angular: se utiliza para el desarrollo de la aplicación web destinada a talleres y administradores. Esta tecnología permite construir interfaces dinámicas, organizadas por componentes y adecuadas para la gestión de solicitudes, talleres, técnicos, usuarios, incidentes e historial. En AutoGo, Angular facilita la creación de paneles administrativos y pantallas de gestión para el seguimiento de servicios.

Flutter: se utiliza para el desarrollo de la aplicación móvil del cliente. Esta tecnología permite construir aplicaciones móviles multiplataforma desde una sola base de código. En AutoGo, Flutter permite que el cliente pueda registrarse, iniciar sesión, registrar vehículos, reportar emergencias, enviar ubicación, adjuntar imágenes o audio, consultar el estado de su solicitud y realizar pagos.

Python: es el lenguaje utilizado en el backend del sistema. Su elección se justifica por su facilidad de uso, claridad, amplio ecosistema de librerías y compatibilidad con herramientas de inteligencia artificial. En AutoGo, Python permite desarrollar la lógica del servidor y conectar el sistema con módulos de procesamiento de audio, imágenes y texto.

FastAPI: es el framework utilizado para construir las APIs REST del sistema. En AutoGo, FastAPI se encarga de recibir solicitudes desde la aplicación móvil y web, gestionar autenticación, procesar emergencias, administrar usuarios, vehículos, talleres, técnicos, pagos e historial. También permite integrar los módulos de inteligencia artificial y generar documentación automática de los servicios.

PostgreSQL: se utiliza como sistema gestor de base de datos relacional. Su función es almacenar de forma estructurada la información del sistema, garantizando integridad, relaciones entre tablas y consistencia de los datos. En AutoGo, PostgreSQL almacena usuarios, vehículos, talleres, técnicos, incidentes, historial y pagos.

Inteligencia artificial: AutoGo contempla módulos de IA para apoyar el análisis de incidentes. Estos módulos permiten transcribir audios, clasificar incidentes, analizar imágenes y generar resúmenes automáticos. Esto ayuda a que el sistema transforme datos enviados por el cliente en información estructurada para los talleres.

Git y GitHub: se utilizan para el control de versiones del código fuente. Estas herramientas permiten registrar cambios, organizar el trabajo del equipo y mantener un repositorio centralizado del proyecto.

Despliegue en la nube: el sistema puede ser desplegado en servicios cloud para permitir el acceso remoto desde distintos dispositivos. Esto facilita la disponibilidad de la aplicación web, la aplicación móvil y el backend.

2.1.8. Base de Datos Relacional

Una base de datos relacional es un modelo de almacenamiento que organiza la información en tablas relacionadas entre sí mediante claves primarias y claves foráneas. Este modelo permite representar entidades del sistema, mantener la integridad de la información y realizar consultas eficientes.

En AutoGo, la base de datos relacional es fundamental porque el sistema maneja múltiples entidades conectadas entre sí. Por ejemplo, un usuario cliente puede registrar uno o varios vehículos; un taller puede tener uno o varios técnicos; una emergencia puede estar asociada a un cliente, un vehículo, un taller y un técnico; y cada incidente puede tener historial de estados y un pago relacionado.

La base de datos del sistema se implementa en PostgreSQL, debido a su robustez, seguridad, soporte para relaciones complejas y capacidad para manejar información estructurada. Las principales tablas consideradas en AutoGo son:

users: almacena la información de los usuarios del sistema, incluyendo clientes, talleres, técnicos y administradores.

vehicles: registra los vehículos pertenecientes a los clientes, incluyendo marca, modelo, año, placa y color.

workshops: almacena la información de los talleres registrados, como nombre, dirección, ubicación geográfica, porcentaje de comisión y estado de actividad.

technicians: registra los técnicos asociados a cada taller, incluyendo nombre, teléfono, disponibilidad y ubicación actual.

incidents: almacena las emergencias vehiculares reportadas por los clientes, incluyendo descripción, estado, prioridad, ubicación, evidencias multimedia, clasificación y resumen generado por IA.

incident_history: registra los cambios de estado de cada incidente, permitiendo mantener trazabilidad del proceso.

payments: almacena la información de los pagos asociados a los servicios atendidos, incluyendo monto, método de pago, comisión y ganancia del taller.

Estas tablas permiten que AutoGo gestione la información de manera organizada y confiable. Además, el uso de relaciones entre tablas evita duplicidad de datos y permite consultar información completa sobre cada emergencia, desde el usuario que la reportó hasta el taller y técnico asignado.

2.1.9. Lenguaje Python y Framework FastApi

Python es un lenguaje de programación ampliamente utilizado en el desarrollo de aplicaciones web, automatización, procesamiento de datos e inteligencia artificial. Su sintaxis clara y su amplio ecosistema de librerías lo convierten en una herramienta adecuada para construir sistemas modernos y escalables.

En AutoGo, Python se utiliza principalmente en el backend del sistema. Su función es implementar la lógica de negocio, procesar solicitudes, validar datos, conectar con la base de datos y permitir la integración con módulos de inteligencia artificial. Esto es importante porque AutoGo no solo gestiona formularios y registros, sino también datos multimodales como audio, imágenes, texto y ubicación.

FastAPI es el framework utilizado para desarrollar el backend de AutoGo. Este framework permite construir APIs REST de manera rápida, clara y eficiente. A través de estas APIs, la aplicación móvil y la aplicación web pueden comunicarse con el servidor central.

En AutoGo, FastAPI cumple las siguientes funciones:

- Gestionar la autenticación y autorización de usuarios.
- Registrar clientes, talleres, técnicos y administradores.
- Permitir el registro y gestión de vehículos.
- Procesar solicitudes de emergencia vehicular.
- Recibir ubicación, imágenes, audios y descripciones.
- Integrar módulos de inteligencia artificial para clasificación de incidentes.
- Generar listas de talleres candidatos.
- Gestionar estados del servicio.
- Registrar pagos.
- Mantener historial de incidentes.
- Exponer servicios REST para la aplicación móvil y web.

FastAPI también facilita la documentación automática de los endpoints, lo cual permite probar las funcionalidades del backend antes de integrarlas completamente con la interfaz web o móvil. Esto resulta útil durante el desarrollo, ya que permite verificar si los servicios responden correctamente y si los datos se procesan de acuerdo con los requisitos del sistema.

2.2. Datos

Los datos son elementos fundamentales dentro de AutoGo, ya que permiten registrar la información necesaria para gestionar usuarios, vehículos, talleres, técnicos, emergencias, pagos e historial de atención. Estos datos son almacenados en la base de datos PostgreSQL y se relacionan entre sí para garantizar la trazabilidad completa del servicio.

Se registran los siguientes datos en el sistema:

Usuarios:
Nombre completo, correo electrónico, contraseña cifrada, teléfono, rol del usuario, fecha de creación y fecha de actualización. Los roles principales son cliente, dueño de taller, técnico y administrador.

Vehículos:
Marca, modelo, año, placa, color, usuario propietario, fecha de creación y fecha de actualización. Estos datos permiten identificar el vehículo involucrado en una emergencia.

Talleres:
Nombre del taller, dirección, latitud, longitud, porcentaje de comisión, estado activo o inactivo, usuario propietario, fecha de creación y fecha de actualización.

Técnicos:
Nombre del técnico, teléfono, disponibilidad, latitud actual, longitud actual, taller al que pertenece, usuario asociado, fecha de creación y fecha de actualización.

Incidentes:
Descripción del problema, estado del incidente, prioridad, ubicación geográfica, texto de ubicación, imagen, audio, clasificación del problema, resumen generado por IA, tiempo estimado de llegada, fecha de aceptación, fecha de inicio, fecha de finalización, usuario, vehículo, taller y técnico asignado.

Historial de incidentes:
Incidente asociado, estado registrado, usuario que realizó el cambio, notas del cambio, fecha y hora del registro. Este historial permite conocer la evolución del servicio desde su creación hasta su finalización.

Pagos:
Incidente asociado, monto del servicio, método de pago, porcentaje de comisión, monto de comisión, ganancia del taller, fecha de pago, estado de pago, número de referencia, notas, fecha de creación y fecha de actualización.

Bitácora del sistema:
Usuario que realizó la acción, tipo de acción, módulo afectado, fecha, hora, descripción del evento e información relevante para auditoría.

Estos datos permiten que AutoGo funcione como una plataforma integral, organizada y trazable. Cada entidad registrada cumple una función dentro del proceso de atención vehicular, permitiendo que el sistema conecte clientes con talleres de manera más eficiente.

2.3. Procesos

Los procesos representan las actividades que realiza AutoGo para transformar los datos ingresados por los usuarios en información útil para la atención de emergencias vehiculares. Estos procesos se organizan en módulos funcionales, los cuales permiten estructurar el sistema de acuerdo con sus principales responsabilidades.

En AutoGo, los procesos abarcan desde el registro e inicio de sesión de usuarios hasta la gestión de emergencias, asignación de talleres, actualización de estados, pagos, administración del sistema y registro en bitácora. Cada proceso se relaciona con uno o varios actores, como Cliente, Taller, Técnico Mecánico y Administrador.

2.3.1 Módulo de Autenticación y Seguridad

El módulo de autenticación y seguridad permite controlar el acceso al sistema y proteger la información de los usuarios. Su objetivo es garantizar que solo los usuarios registrados puedan ingresar a la plataforma y utilizar las funciones correspondientes a su rol.

Este módulo contempla los siguientes procesos:

- Registro de usuarios en el sistema.
- Inicio de sesión mediante correo electrónico y contraseña.
- Validación de credenciales.
- Cierre de sesión seguro.
- Asignación de roles de usuario.
- Control de permisos según el tipo de actor.
- Protección de información sensible.
- Registro de accesos en la bitácora.

En AutoGo, los roles principales son Cliente, Taller, Técnico Mecánico y Administrador. Cada uno tiene acceso a diferentes funcionalidades. Por ejemplo, el cliente puede registrar vehículos y emergencias; el taller puede gestionar solicitudes y técnicos; el técnico puede atender servicios asignados; y el administrador puede supervisar el funcionamiento general del sistema.

Este módulo es importante porque AutoGo maneja datos sensibles como información personal, ubicación en tiempo real, historial de emergencias y pagos. Por ello, es necesario

aplicar mecanismos de seguridad que protejan la integridad y confidencialidad de la información.

2.3.2. Módulo de Gestión de Usuarios

El módulo de gestión de usuarios permite registrar, consultar, modificar y administrar las cuentas existentes dentro de AutoGo. Este módulo es utilizado principalmente por el administrador del sistema, aunque cada usuario también puede gestionar ciertos datos de su propio perfil.

Los procesos principales de este módulo son:

- Registrar nuevos usuarios.
- Consultar usuarios registrados.
- Actualizar información personal.
- Gestionar perfiles de usuario.
- Asignar o modificar roles.
- Activar o desactivar cuentas.
- Eliminar o suspender usuarios según corresponda.
- Registrar acciones administrativas en la bitácora.

Este módulo permite mantener actualizada la información de los usuarios y asegurar que cada actor tenga acceso únicamente a las funcionalidades permitidas. Además, facilita el control administrativo sobre clientes, talleres, técnicos y administradores registrados en la plataforma.

2.3.3. Módulo de Gestión de Vehículos

El módulo de gestión de vehículos permite que los clientes registren y administren la información de sus vehículos dentro de AutoGo. Esta información es necesaria para que, al momento de reportar una emergencia, el sistema pueda identificar el vehículo afectado.

Los procesos principales de este módulo son:

- Registrar vehículo.
- Consultar vehículos registrados.
- Visualizar detalle de un vehículo.
- Editar información del vehículo.
- Eliminar vehículo.
- Asociar un vehículo a una emergencia.
- Validar que el vehículo pertenezca al cliente autenticado.

Los datos registrados pueden incluir marca, modelo, año, placa y color. Estos datos ayudan al taller o técnico a conocer mejor el vehículo que será atendido, permitiendo una preparación más adecuada para el servicio.

Este módulo es importante porque un cliente puede tener más de un vehículo registrado, por lo que el sistema debe permitir seleccionar correctamente cuál está involucrado en la emergencia.

2.3.4. Módulo de Gestión de Emergencias

El módulo de gestión de emergencias es uno de los componentes principales de AutoGo. Su función es permitir que el cliente registre una solicitud de auxilio vehicular cuando presenta un problema con su vehículo.

Este módulo contempla los siguientes procesos:

- Registrar una emergencia vehicular.
- Seleccionar el vehículo involucrado.
- Ingresar una descripción del problema.
- Enviar ubicación en tiempo real.
- Adjuntar imágenes del vehículo o del daño.
- Enviar audio describiendo el incidente.
- Agregar texto adicional.
- Procesar la información recibida.
- Clasificar preliminarmente el tipo de incidente.
- Asignar un nivel de prioridad.
- Guardar la emergencia en la base de datos.
- Consultar el estado de la solicitud.

La información enviada por el cliente puede ser procesada por módulos de inteligencia artificial. El audio puede ser transcrito a texto, las imágenes pueden ser analizadas para identificar daños visibles y la descripción puede apoyar la clasificación del incidente. Según los requisitos del sistema, AutoGo debe poder trabajar con datos multimodales como imagen, audio, texto y geolocalización para optimizar el diagnóstico preliminar, la priorización y la asignación del servicio.

Este módulo permite convertir una emergencia reportada por el cliente en una solicitud estructurada, facilitando que el taller conozca el problema antes de aceptar el servicio.

2.3.5. Módulo de Gestión de Talleres

El módulo de gestión de talleres permite administrar la información de los talleres mecánicos que brindan servicios de asistencia dentro de AutoGo. Los talleres son los proveedores encargados de atender las emergencias solicitadas por los clientes.

Este módulo contempla los siguientes procesos:

- Registrar taller.
- Actualizar información del taller.
- Consultar datos del taller.
- Gestionar técnicos del taller.
- Gestionar disponibilidad de técnicos.
- Visualizar solicitudes disponibles.
- Revisar información estructurada del incidente.
- Aceptar o rechazar solicitudes.
- Actualizar el estado del servicio.
- Consultar historial de atenciones.

Cuando un taller recibe una alerta de asistencia, puede revisar el tipo de problema, la ubicación del cliente, la disponibilidad de sus técnicos y la distancia aproximada. Con esta información, el taller puede aceptar o rechazar la solicitud de manera más informada.

Este módulo es importante porque permite organizar la operación del taller, mejorar la asignación de técnicos y mantener un registro de los servicios realizados.

2.3.6. Módulo de Gestión de Pagos

El módulo de gestión de pagos permite registrar y controlar el pago de los servicios de auxilio vehicular dentro de AutoGo. Según los requisitos del sistema, el cliente debe realizar los pagos correspondientes desde la aplicación y el taller debe pagar a la plataforma un porcentaje del precio cobrado como comisión.

Los procesos principales de este módulo son:

- Generar monto del servicio.
- Seleccionar método de pago.
- Registrar datos o referencia del pago.
- Confirmar pago realizado.
- Asociar el pago al incidente correspondiente.
- Calcular la comisión de la plataforma.
- Calcular la ganancia del taller.
- Guardar el comprobante o referencia del pago.
- Consultar historial de pagos.

Este módulo permite mantener control financiero sobre cada atención realizada. Además, evita duplicidad de pagos, registra el estado de la transacción y permite conocer cuánto corresponde a la plataforma y cuánto corresponde al taller.

La gestión de pagos contribuye a la transparencia del servicio, ya que tanto el cliente como el taller pueden contar con un registro del pago realizado.

2.3.7. Administración del Sistema

El módulo de administración del sistema está orientado al usuario administrador. Su función principal es supervisar el funcionamiento general de AutoGo y mantener el control de la plataforma.

Este módulo contempla los siguientes procesos:

- Gestionar usuarios.
- Gestionar talleres.
- Supervisar incidentes.
- Revisar emergencias registradas.
- Monitorear estados de atención.
- Consultar pagos.
- Consultar métricas del sistema.
- Intervenir en casos necesarios.
- Verificar el cumplimiento de los procesos.
- Mantener el control general de la plataforma.

El administrador puede revisar la información registrada en el sistema y tomar acciones cuando sea necesario. Por ejemplo, puede supervisar incidentes pendientes, verificar talleres registrados, controlar usuarios activos y revisar información de auditoría.

Este módulo es importante porque permite mantener la estabilidad operativa de AutoGo, controlar el correcto uso de la plataforma y garantizar que los procesos funcionen de acuerdo con los objetivos del sistema.

2.3.8. Bitácora del Sistema

La bitácora del sistema es el módulo encargado de registrar las acciones relevantes realizadas dentro de AutoGo. Su objetivo es mantener trazabilidad, control y auditoría sobre las operaciones del sistema.

Este módulo puede registrar eventos como:

- Inicio de sesión.
- Cierre de sesión.
- Registro de usuarios.
- Modificación de perfiles.
- Registro de vehículos.
- Registro de emergencias.
- Cambio de estado de incidentes.
- Asignación de talleres o técnicos.
- Registro de pagos.
- Cambios realizados por el administrador.

- Errores o eventos importantes del sistema.

La bitácora permite conocer quién realizó una acción, cuándo la realizó y qué módulo fue afectado. Esto es importante para mantener transparencia y control dentro de la plataforma. Además, facilita la revisión de problemas, reclamos o inconsistencias que puedan presentarse durante la operación del sistema.

En AutoGo, la bitácora complementa el historial de incidentes, ya que mientras el historial registra los cambios específicos de una emergencia, la bitácora registra acciones generales del sistema. Esto fortalece la seguridad y la trazabilidad completa del servicio.

Parte III – Proceso de desarrollo

3.1) FLUJO DE TRABAJO: CAPTURA DE REQUISITOS

3.1.1 Identificar Actores y Casos de Uso

3.1.1.1 Actores

A1. Cliente:
Persona que tiene un problema con su vehículo y solicita ayuda. Puede registrarse e iniciar sesión, registrar su vehículo, reportar emergencias (foto, audio, ubicación), ver estado de su solicitud, recibir notificaciones, comunicarse con el taller y realizar pagos

A2. Taller: Empresa o negocio que brinda asistencia mecánica. Puede registrarse en la plataforma, recibir solicitudes de emergencia, ver detalles del incidente, aceptar o rechazar servicios, gestionar técnicos, actualizar estado del servicio, y consultar historial

A3. Técnico Mecánico: Persona del taller que realiza la asistencia en campo. Puede recibir asignacione, ver ubicación del cliente, atender la emergencia y actualizar estado del servicio

A4. Administrador del Sistema: Encargado de gestionar la plataforma. Puede Gestionar usuarios, Gestionar talleres, Supervisar incidentes, Ver métricas, Mantener seguridad y control

3.1.1.2 Casos de Uso

CU1. Registrarse en el sistema (cliente): Permite al usuario crear una cuenta dentro de la plataforma ingresando datos personales como nombre, correo y contraseña. Este proceso es necesario para poder acceder a las funcionalidades del sistema y realizar solicitudes de auxilio.

CU2. Iniciar sesión: Permite al usuario acceder al sistema mediante la validación de sus credenciales (correo y contraseña), garantizando la seguridad y el acceso personalizado a su información.

CU3. Cerrar sesión: Permite al usuario finalizar su sesión activa, asegurando que su información quede protegida, especialmente en dispositivos compartidos.

CU4. Gestionar perfil de usuario: Permite al usuario consultar y actualizar sus datos personales, como nombre, contacto o contraseña, manteniendo su información actualizada en el sistema.

CU5. Registrar vehículo: Permite al usuario ingresar los datos de uno o varios vehículos (marca, modelo, placa, etc.), los cuales serán utilizados al momento de solicitar asistencia.

CU6. Gestionar información del vehículo: Permite al usuario modificar, consultar o eliminar la información de sus vehículos registrados, facilitando la administración de sus datos vehiculares.

CU7. Gestionar emergencias: Permite al usuario registrar una emergencia vehicular proporcionando información relevante como ubicación en tiempo real, imágenes, audio o texto descriptivo del problema. Esta información es utilizada por el sistema para analizar y procesar la solicitud
mediante el uso de IA para detección de imágenes, audio. (Whisper, Yolo, Sklear)

CU8. Consultar estado de la solicitud: Permite al usuario realizar seguimiento de su solicitud, visualizando su estado (pendiente, en proceso o atendido), el taller asignado y el tiempo estimado de llegada del servicio.

CU9. Realizar pago del servicio: Permite al usuario efectuar el pago del servicio recibido a través de la plataforma, garantizando una transacción segura y registrando el historial de pagos.

CU10. Registrar taller: Permite a los talleres mecánicos registrarse en la plataforma para ofrecer sus servicios, proporcionando información relevante del negocio.

CU11. Gestionar técnicos del taller: Permite al taller registrar, actualizar o eliminar información de los técnicos que forman parte de su equipo, quienes serán asignados a las solicitudes.

CU12. Gestionar disponibilidad de técnicos: Permite al taller indicar qué técnicos están disponibles para atender servicios en un determinado momento, facilitando la asignación eficiente.

CU13. Gestionar solicitudes (Taller): Permite al taller visualizar las solicitudes de emergencia disponibles, revisar la información del incidente y decidir si acepta o rechaza la atención del servicio**.**

CU14. Actualizar estado del servicio: Permite al taller cambiar el estado del servicio a medida que avanza la atención (por ejemplo: en proceso o finalizado), manteniendo informado al cliente.

CU15. Consultar historial de atenciones: Permite al taller acceder a un registro de todos los servicios realizados, útil para control, análisis y seguimiento.

CU16. Generar lista de talleres candidatos: Permite al sistema analizar variables como ubicación, tipo de problema, disponibilidad y capacidad de los talleres para generar una lista de posibles opciones que pueden atender la emergencia **con ayuda de la IA Recomendación + Optimización (geopy, sklearn)**

CU17. Gestionar usuarios: Permite al administrador supervisar, crear, modificar o eliminar cuentas de usuarios dentro del sistema.

CU18. Gestionar talleres: Permite al administrador controlar el registro y funcionamiento de los talleres, incluyendo su aprobación o suspensión y al taller gestionar su información.
CU19. Gestionar incidentes: Permite al administrador monitorear todas las emergencias registradas, asegurando el correcto funcionamiento del sistema.

CU20. Bitácora del sistema: Permite registrar y consultar todas las actividades relevantes del sistema, como accesos, cambios y eventos importantes, con fines de control y auditoría.

3.1.2 Tablas de Casos de Uso y sus Prioridades

NÚMERO	CASO DE USO	ESTADO	PRIORIDAD	RIESGO	ACTORES	CICLO
CU1	Registrarse en el sistema (cliente)	Aprobado	Crítico	Crítico	Cliente	C1
CU2	Iniciar sesión	Aprobado	Crítico	Crítico	Cliente, Taller, Administrador	C1
CU3	Cerrar sesión	Aprobado	Crítico	Crítico	Cliente, Taller, Administrador	C1
CU4	Gestionar perfil de usuario	Aprobado	Crítico	Crítico	Cliente, Taller	C1
CU5	Registrar vehículo	Aprobado	Crítico	Crítico	Cliente	C1

CU6	Gestionar información del vehículo	Aprobado	Crítico	Crítico	Cliente	C1
CU7	Gestionar emergencias	Aprobado	Crítico	Crítico	Cliente	C2
CU8	Consultar estado de la solicitud	Aprobado	Crítico	Crítico	Cliente	C2
CU9	Realizar pago del servicio	Aprobado	Crítico	Crítico	Cliente	C2
CU10	Registrar taller	Aprobado	Crítico	Crítico	Taller, Administrador	C2
CU11	Gestionar técnicos del taller	Aprobado	Crítico	Crítico	Taller	C2
CU12	Gestionar disponibilidad de técnicos	Aprobado	Crítico	Crítico	Taller	C2
CU13	Gestionar solicitudes (Taller)	Aprobado	Crítico	Crítico	Taller	C2
CU14	Actualizar estado del servicio	Aprobado	Significativo	Significativo	Taller, Técnico Mecánico	C2
CU15	Consultar historial de atenciones	Aprobado	Significativo	Significativo	Taller	C2
CU16	Generar lista de talleres candidatos	Aprobado	Crítico	Crítico	Sistema	C2

CU17	Gestionar usuarios	Aprobado	Crítico	Crítico	Administrador	C3
CU18	Gestionar talleres	Aprobado	Crítico	Crítico	Administrador, Taller	C1
CU19	Gestionar incidentes	Aprobado	Crítico	Crítico	Administrador	C3
CU20	Bitácora del sistema	Aprobado	Significativo	Significativo	Administrador	C3

CICLO #1

NÚMERO	CASO DE USO	ESTADO	PRIORIDAD	RIESGO	ACTORES	CICLO
CU1	Registrarse en el sistema (cliente)(móvil)	Aprobado	Crítico	Crítico	Cliente	C1
CU2	Iniciar sesión	Aprobado	Crítico	Crítico	Cliente, Taller, Administrador	C1
CU3	Cerrar sesión	Aprobado	Crítico	Crítico	Cliente, Taller, Administrador	C1
CU4	Gestionar perfil de usuario	Aprobado	Crítico	Crítico	Cliente, Taller	C1
CU5	Registrar vehículo (móvil)	Aprobado	Crítico	Crítico	Cliente	C1
CU6	Gestionar información del vehículo (móvil)	Aprobado	Crítico	Crítico	Cliente	C1
CU18	Gestionar taller (web)	Aprobado	Crítico	Crítico	Taller	C1

CICLO #2

NÚMERO	CASO DE USO	ESTADO	PRIORIDAD	RIESGO	ACTORES	CICLO
CU7	Gestionar emergencias	Aprobado	Crítico	Crítico	Cliente	C2

CU8	Consultar estado de la solicitud	Aprobado	Crítico	Crítico	Cliente	C2
CU9	Realizar pago del servicio	Aprobado	Crítico	Crítico	Cliente	C2
CU10	Registrar taller	Aprobado	Crítico	Crítico	Taller, Administrador	C2
CU11	Gestionar técnicos del taller	Aprobado	Crítico	Crítico	Taller	C2
CU12	Gestionar disponibilidad de técnicos	Aprobado	Crítico	Crítico	Taller	C2
CU13	Gestionar solicitudes (Taller)	Aprobado	Crítico	Crítico	Taller	C2
CU14	Actualizar estado del servicio	Aprobado	Significativo	Significativo	Taller, Técnico Mecánico	C2
CU15	Consultar historial de atenciones	Aprobado	Significativo	Significativo	Taller	C2
CU16	Generar lista de talleres candidatos	Aprobado	Crítico	Crítico	cliente	C2

CICLO #3

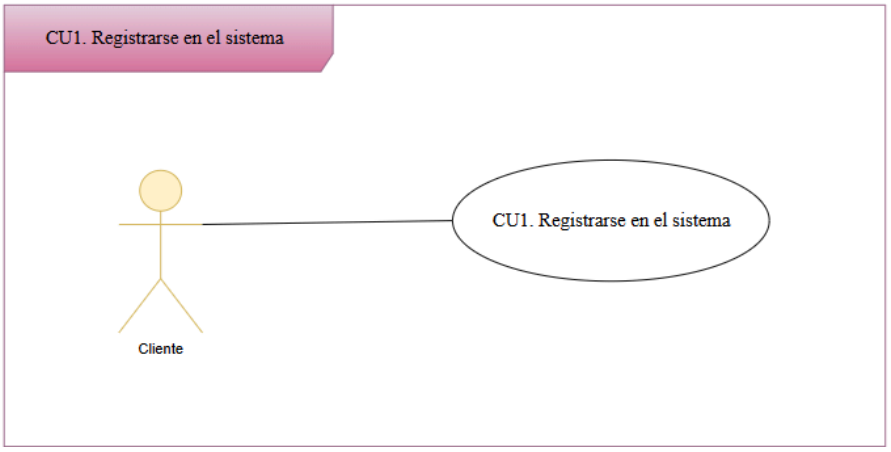
NÚMERO	CASO DE USO	ESTADO	PRIORIDAD	RIESGO	ACTORES	CICLO

CU17	Gestionar usuarios	Aprobado	Crítico	Crítico	Administrador	C3
CU19	Gestionar incidentes	Aprobado	Crítico	Crítico	Administrador	C3
CU20	Bitácora del sistema	Aprobado	Significativo	Significativo	Administrador	C3

3.1.3 Detallar un Caso de Uso

Ciclo#1

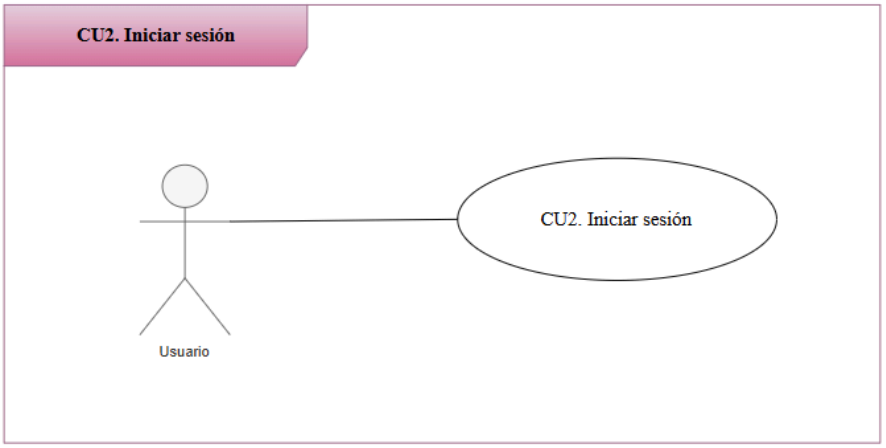
CU1. Registrarse en el sistema (cliente)



Caso de uso	**CU1. Registrarse en el sistema (cliente)**
Propósito	Permitir al usuario crear una cuenta en la plataforma.
Actores	Cliente
Actor Iniciador	Cliente
Precondición	El usuario no debe estar registrado.
Flujo de Suceso Principal	1. El usuario accede al formulario de registro. 2. Ingresa sus datos personales (nombre, correo, contraseña). 3. El sistema valida la información.

	4. El sistema guarda los datos. 5. El sistema confirma el registro exitoso.
Poscondición	Usuario registrado en el sistema.
Excepción	• Datos incompletos o inválidos. • Usuario ya registrado.

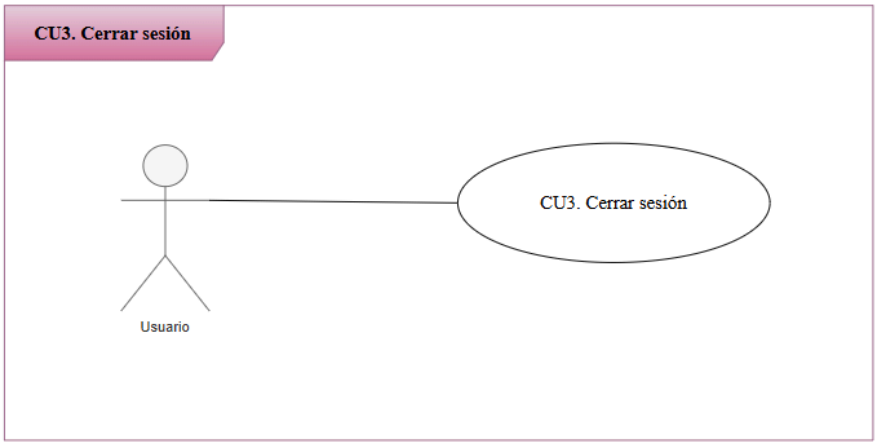
CU2. Iniciar sesión



Caso de uso	**CU2. Iniciar sesión**
Propósito	Permitir al usuario acceder al sistema.
Actores	Cliente, Taller, Administrador
Actor Iniciador	Usuario
Precondición	Usuario registrado previamente.
Flujo de Suceso Principal	1. El usuario ingresa correo y contraseña. 2. El sistema valida credenciales. 3. El sistema permite el acceso.
Poscondición	Usuario autenticado en el sistema.

Excepción	Credenciales incorrectas.

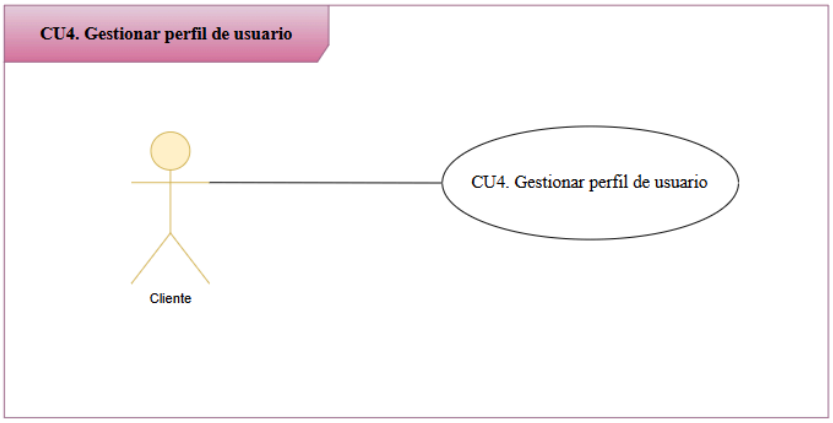
CU3. Cerrar sesión



Caso de uso	**CU3. Cerrar sesión**
Propósito	Finalizar la sesión del usuario.
Actores	Cliente, Taller, Administrador
Actor Iniciador	Usuario
Precondición	Usuario autenticado.
Flujo de Suceso Principal	1. El usuario selecciona “Cerrar sesión”. 2. El sistema finaliza la sesión.
Poscondición	Usuario desconectado.
Excepción	Error en cierre de sesión.

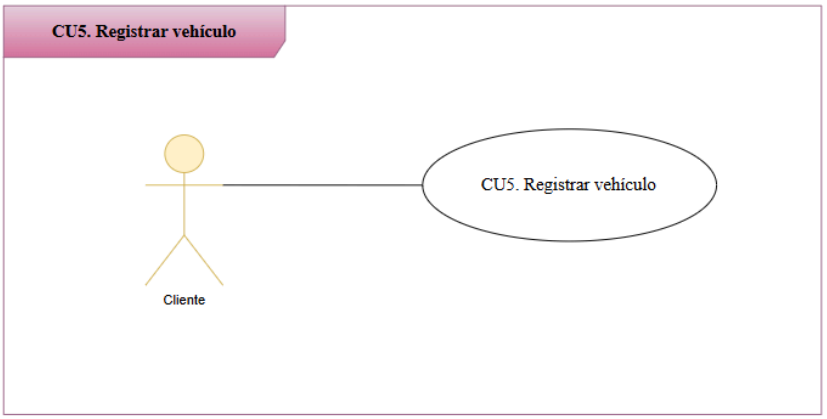
CU4. Gestionar perfil de usuario

Diagrama de Casos de Uso



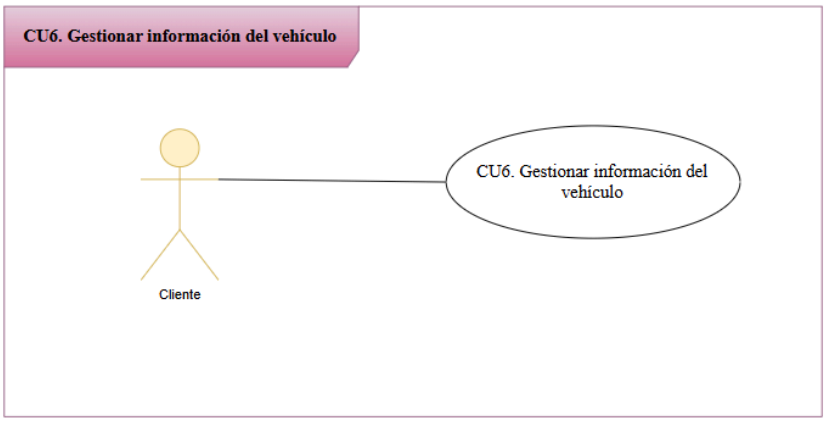
Caso de uso	**CU4. Gestionar perfil de usuario**
Propósito	Permitir actualizar datos personales.
Actores	Cliente, Taller
Actor Iniciador	Usuario
Precondición	Usuario autenticado.
Flujo de Suceso Principal	1. El usuario accede a su perfil. 2. Modifica sus datos. 3. El sistema guarda los cambios.
Poscondición	Datos actualizados.
Excepción	Datos inválidos.

CU5. Registrar vehículo



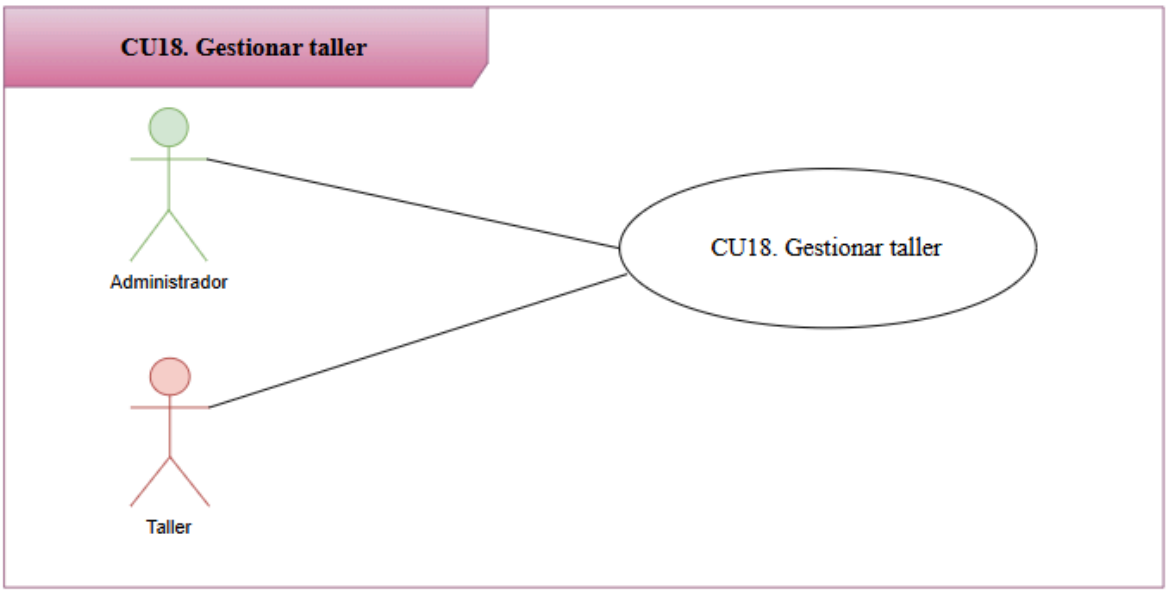
Caso de uso	**CU5. Registrar vehículo**
Propósito	Registrar un vehículo en el sistema.
Actores	Cliente
Actor Iniciador	Cliente
Precondición	Usuario autenticado.
Flujo de Suceso Principal	1. El usuario ingresa datos del vehículo. 2. El sistema valida la información. 3. El sistema guarda el vehículo.
Poscondición	Vehículo registrado.
Excepción	Datos incompletos.

CU6. Gestionar información del vehículo



Caso de uso	**CU6. Gestionar información del vehículo**
Propósito	Modificar o eliminar datos del vehículo.
Actores	Cliente
Actor Iniciador	Cliente
Precondición	Vehículo registrado.
Flujo de Suceso Principal	1. El usuario selecciona un vehículo. 2. Edita o elimina la información. 3. El sistema actualiza los datos.
Poscondición	Información actualizada.
Excepción	Vehículo no encontrado.

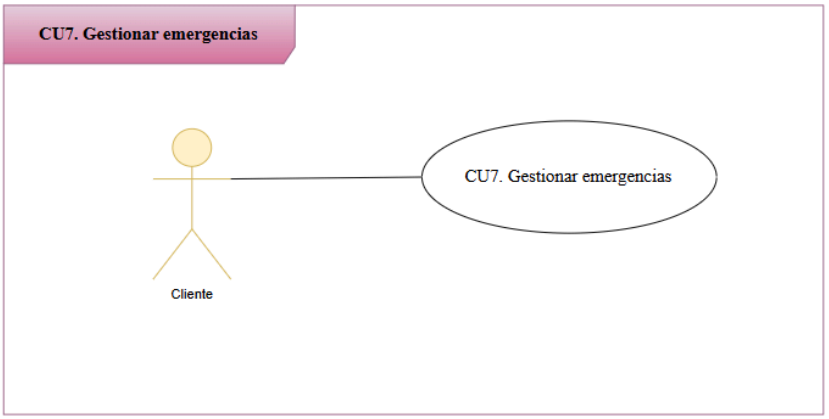
CU18. Gestionar talleres



Caso de uso	**CU18. Gestionar taller**
Propósito	Administrar el taller registrado.
Actores	Taller, Administrador
Actor Iniciador	Administrador, Taller
Precondición	Talleres registrados.
Flujo de Suceso Principal	1. El administrador o taller visualiza la información del taller 2. Aprueba, modifica o elimina registros. 3. El sistema actualiza la información.
Poscondición	Talleres actualizados.
Excepción	Taller sin información

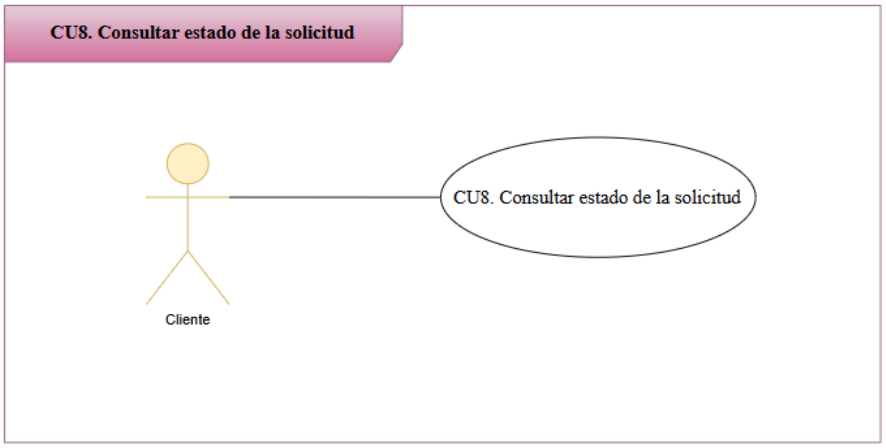
CICLO#2

CU7. Gestionar emergencias



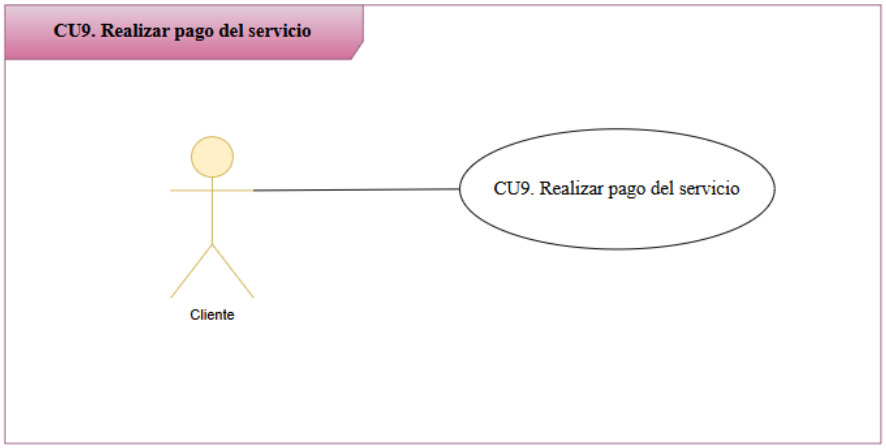
Caso de uso	**CU7. Gestionar emergencias**
Propósito	Registrar una emergencia vehicular.
Actores	Cliente
Actor Iniciador	Cliente
Precondición	Usuario autenticado y con vehículo registrado.
Flujo de Suceso Principal	1. El usuario reporta la emergencia. 2. Envía ubicación en tiempo real. 3. Adjunta imágenes/audio/texto. 4. El sistema procesa la información (IA). 5. El sistema registra la emergencia.
Poscondición	Emergencia registrada en el sistema.
Excepción	• Fallo en envío de datos. • Información insuficiente.

CU8. Consultar estado de la solicitud



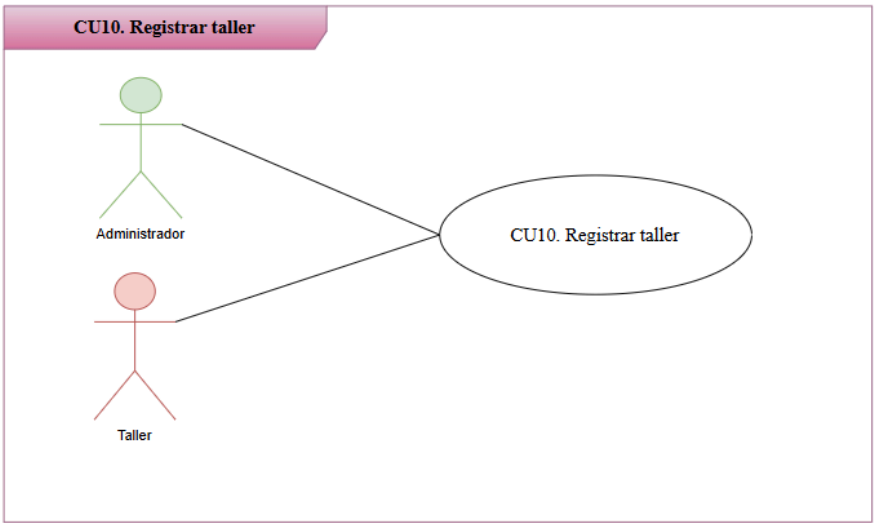
Caso de uso	**CU8. Consultar estado de la solicitud**
Propósito	Permitir seguimiento del servicio.
Actores	Cliente
Actor Iniciador	Cliente
Precondición	Solicitud registrada.
Flujo de Suceso Principal	1. El usuario accede a su solicitud. 2. El sistema muestra estado, taller y tiempo estimado.
Poscondición	Información visualizada.
Excepción	Solicitud no encontrada.

CU9. Realizar pago del servicio



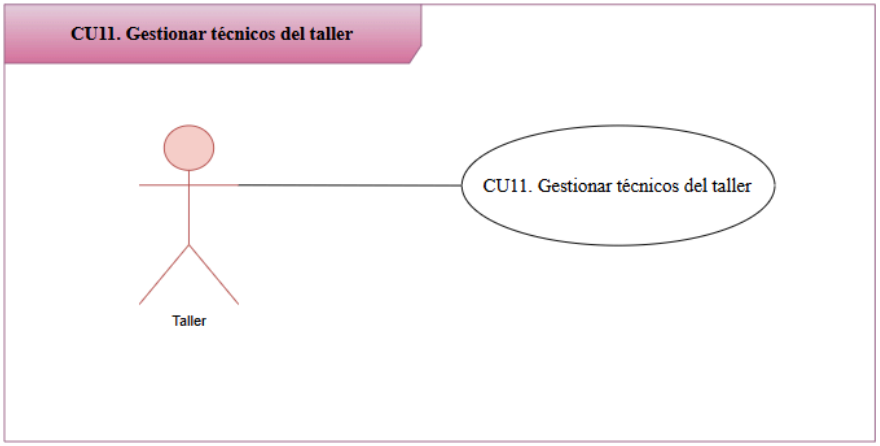
Caso de uso	**CU9. Realizar pago del servicio**
Propósito	Procesar el pago del servicio.
Actores	Cliente
Actor Iniciador	Cliente
Precondición	Servicio finalizado.
Flujo de Suceso Principal	1. El usuario selecciona el método de pago. 2. Ingresa datos de pago. 3. El sistema procesa la transacción. 4. El sistema confirma el pago. 5.
Poscondición	Pago registrado.
Excepción	• Pago rechazado. • Error en la transacción.

CU10. Registrar taller



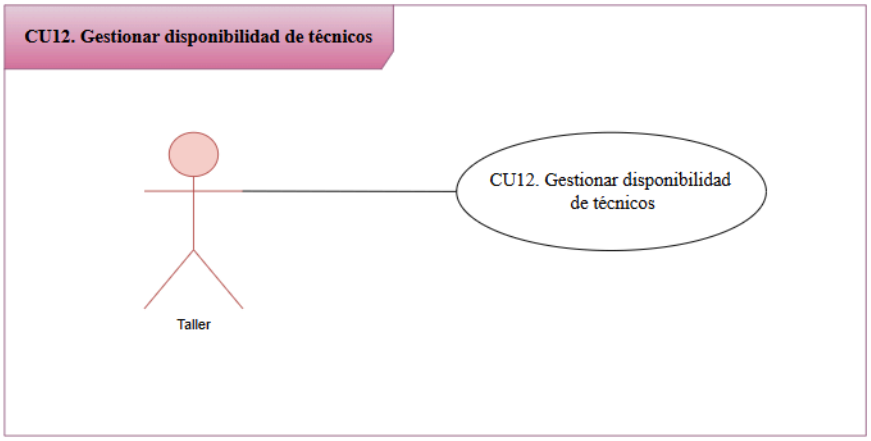
Caso de uso	**CU10. Registrar taller**
Propósito	Registrar un taller como proveedor.
Actores	Taller, Administrador
Actor Iniciador	Taller
Precondición	Taller no registrado.
Flujo de Suceso Principal	1. El taller ingresa sus datos. 2. El sistema valida la información. 3. El administrador aprueba el registro (opcional). 4. El sistema registra el taller.
Poscondición	Taller registrado en el sistema.
Excepción	• Datos inválidos. • Registro rechazado.

CU11. Gestionar técnicos del taller



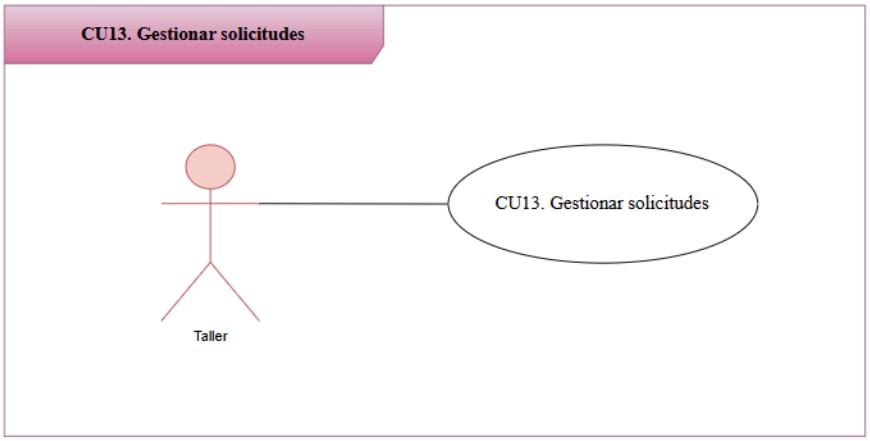
Caso de uso	**CU11. Gestionar técnicos del taller**
Propósito	Permitir al taller administrar la información de sus técnicos.
Actores	Taller
Actor Iniciador	Taller
Precondición	Taller registrado y autenticado.
Flujo de Suceso Principal	1. El taller accede al módulo de técnicos. 2. Registra, edita o elimina información de técnicos. 3. El sistema valida los datos. 4. El sistema guarda los cambios.
Poscondición	Técnicos registrados y actualizados.
Excepción	Datos inválidos o incompletos.

CU12. Gestionar disponibilidad de técnicos



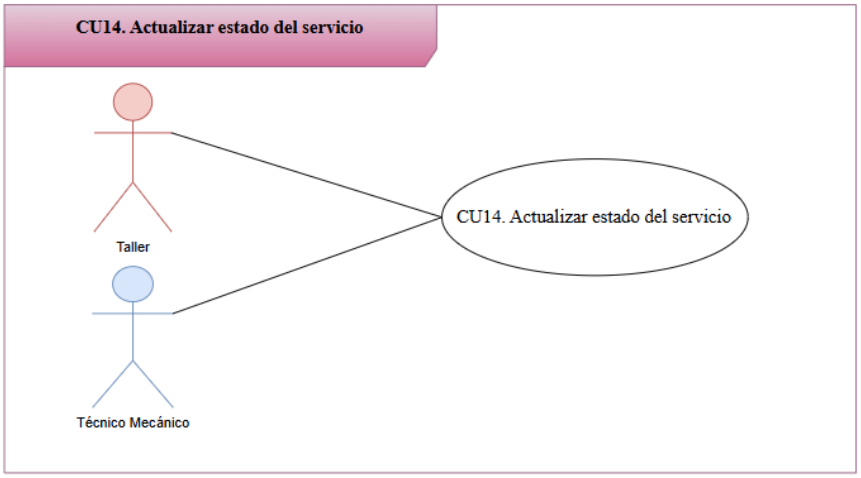
Caso de uso	**CU12. Gestionar disponibilidad de técnicos**
Propósito	Permitir al taller indicar la disponibilidad de sus técnicos.
Actores	Taller
Actor Iniciador	Taller
Precondición	Técnicos registrados.
Flujo de Suceso Principal	1. El taller selecciona un técnico. 2. Define su disponibilidad (disponible/no disponible). 3. El sistema actualiza el estado.
Poscondición	Disponibilidad actualizada.
Excepción	Técnico no encontrado.

CU13. Gestionar solicitudes (Taller)



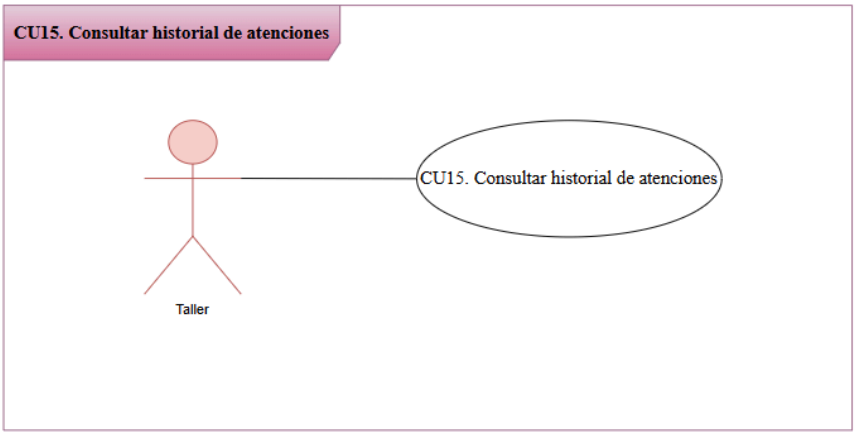
Caso de uso	**CU13. Gestionar solicitudes (Taller)**
Propósito	Permitir al taller gestionar solicitudes de emergencia.
Actores	Taller, Sistema
Actor Iniciador	Taller
Precondición	Solicitudes registradas en el sistema.
Flujo de Suceso Principal	1. El taller visualiza solicitudes disponibles. 2. Revisa la información del incidente. 3. Decide aceptar o rechazar la solicitud. 4. El sistema registra la decisión.
Poscondición	Solicitud aceptada o rechazada.
Excepción	Solicitud ya asignada a otro taller.

CU14. Actualizar estado del servicio



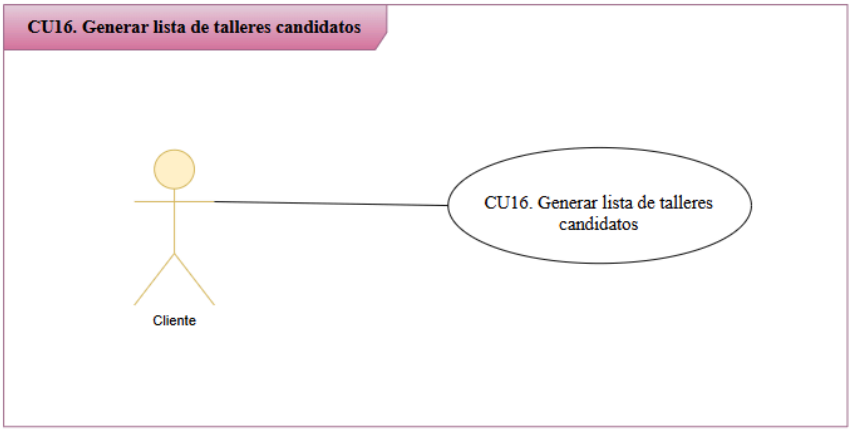
Caso de uso	**CU14. Actualizar estado del servicio**
Propósito	Permitir actualizar el estado del servicio.
Actores	Taller, Técnico
Actor Iniciador	Taller, Técnico
Precondición	Solicitud aceptada.
Flujo de Suceso Principal	1. El taller o técnico accede al servicio. 2. Actualiza el estado (en proceso, finalizado). 3. El sistema guarda el cambio. 4. Se notifica al cliente.
Poscondición	Estado actualizado.
Excepción	Error en actualización.

CU15. Consultar historial de atenciones



Caso de uso	**CU15. Consultar historial de atenciones**
Propósito	Permitir al taller visualizar servicios anteriores.
Actores	Taller
Actor Iniciador	Taller
Precondición	Existencia de registros históricos.
Flujo de Suceso Principal	1. El taller accede al historial. 2. El sistema muestra lista de servicios realizados.
Poscondición	Información visualizada.
Excepción	No existen registros.

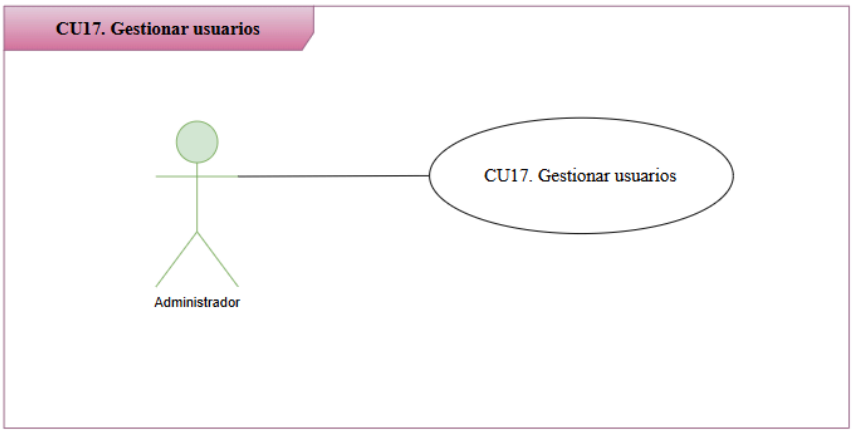
CU16. Generar lista de talleres candidatos



Caso de uso	**CU16. Generar lista de talleres candidatos**
Propósito	Identificar talleres adecuados para atender una emergencia.
Actores	Cliente
Actor Iniciador	Cliente
Precondición	Emergencia registrada.
Flujo de Suceso Principal	1. El sistema recibe datos del incidente. 2. Analiza ubicación, tipo de problema y disponibilidad. 3. Calcula distancia y prioridad. 4. Genera lista de talleres candidatos.
Poscondición	Lista de talleres disponibles generada.
Excepción	No hay talleres disponibles.

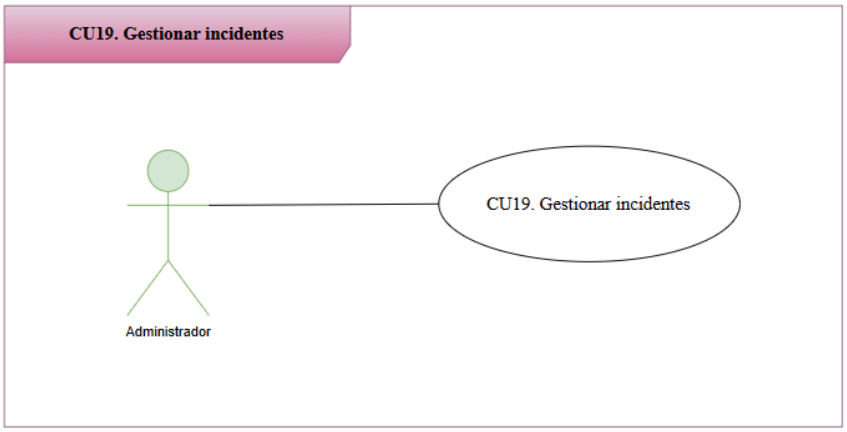
CICLO#3

CU17. Gestionar usuarios-



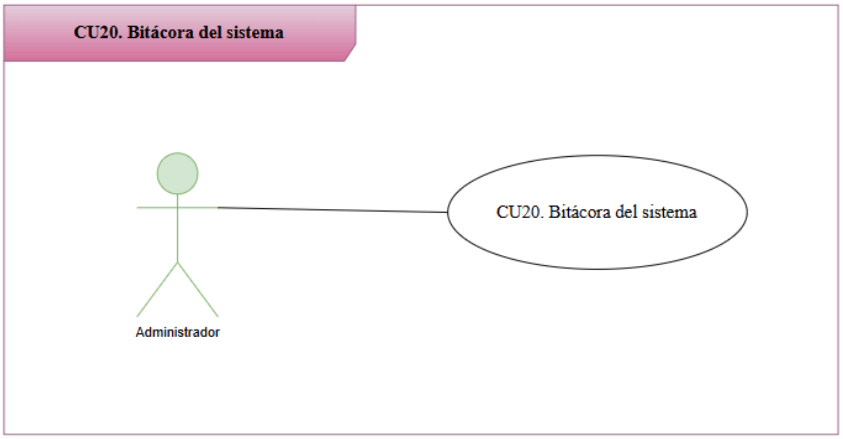
Caso de uso	**CU17. Gestionar usuarios**
Propósito	Administrar cuentas de usuarios.
Actores	Administrador
Actor Iniciador	Administrador
Precondición	Administrador autenticado.
Flujo de Suceso Principal	1. El administrador accede al módulo de usuarios. 2. Visualiza, edita o elimina usuarios. 3. El sistema guarda cambios.
Poscondición	Usuarios gestionados.
Excepción	Usuario no encontrado.

CU19. Gestionar incidentes



Caso de uso	**CU19. Gestionar incidentes**
Propósito	Supervisar las emergencias registradas.
Actores	Administrador
Actor Iniciador	Administrador
Precondición	Incidentes registrados.
Flujo de Suceso Principal	1. El administrador accede a incidentes. 2. Visualiza detalles. 3. Realiza seguimiento o interviene si es necesario.
Poscondición	Incidentes monitoreados.
Excepción	Incidente no encontrado.

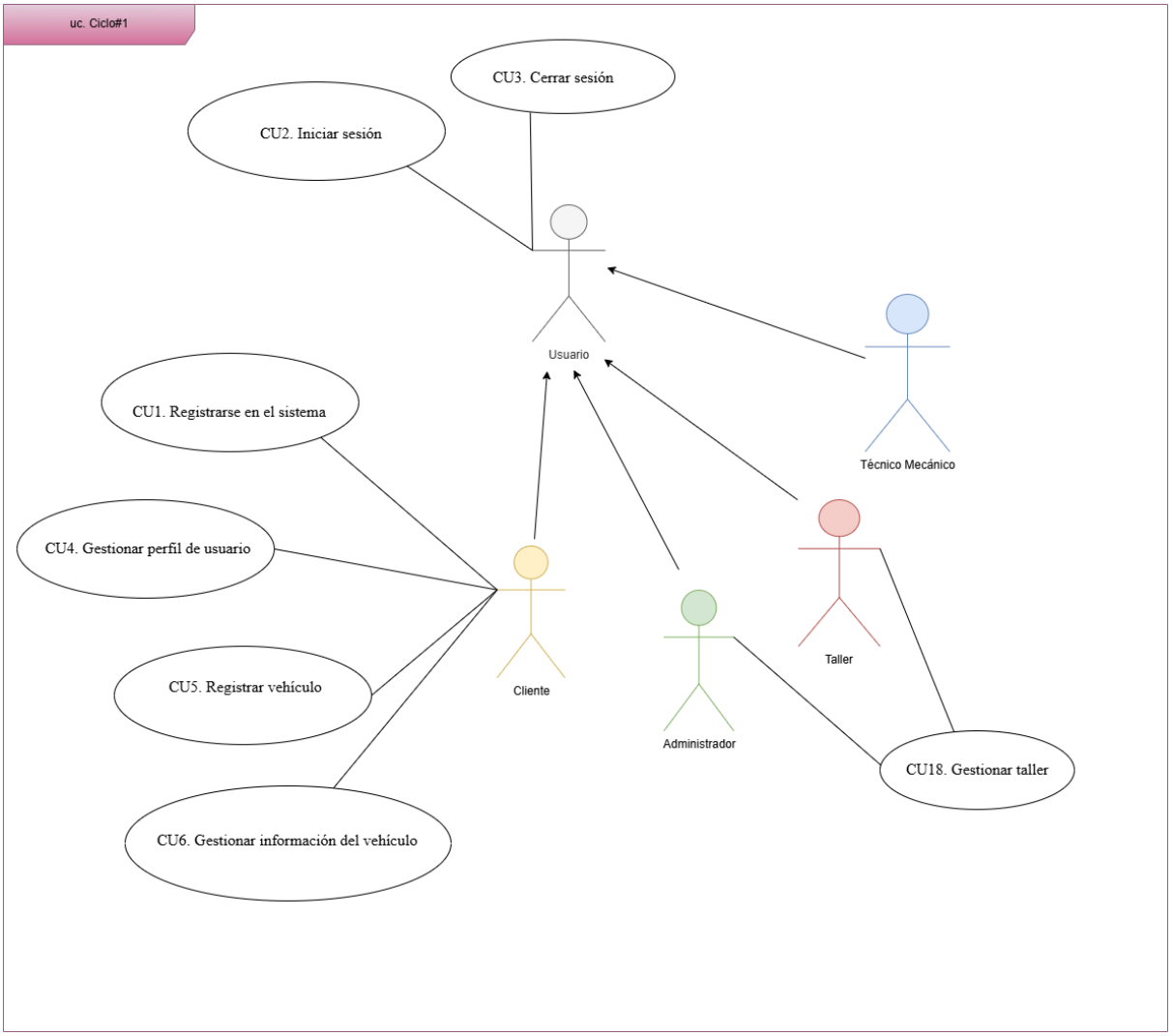
CU20. Bitácora del sistema



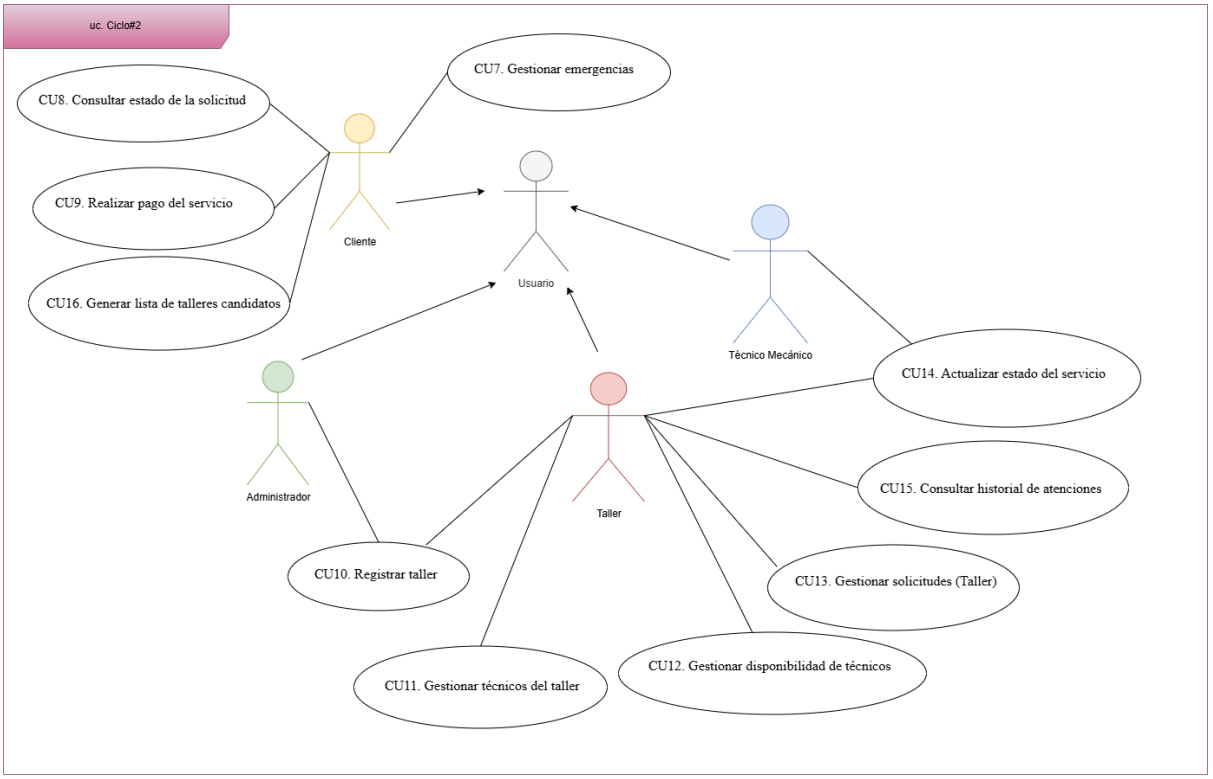
Caso de uso	**CU20. Bitácora del sistema**
Propósito	Registrar eventos del sistema para auditoría.
Actores	Administrador
Actor Iniciador	Administrador
Precondición	Sistema en funcionamiento.
Flujo de Suceso Principal	1. El sistema registra eventos automáticamente. 2. El administrador accede a la bitácora. 3. Consulta las actividades del sistema.
Poscondición	Eventos registrados y disponibles para consulta.
Excepción	Error en registro de eventos.

3.1.4 Estructura de los Casos de Uso

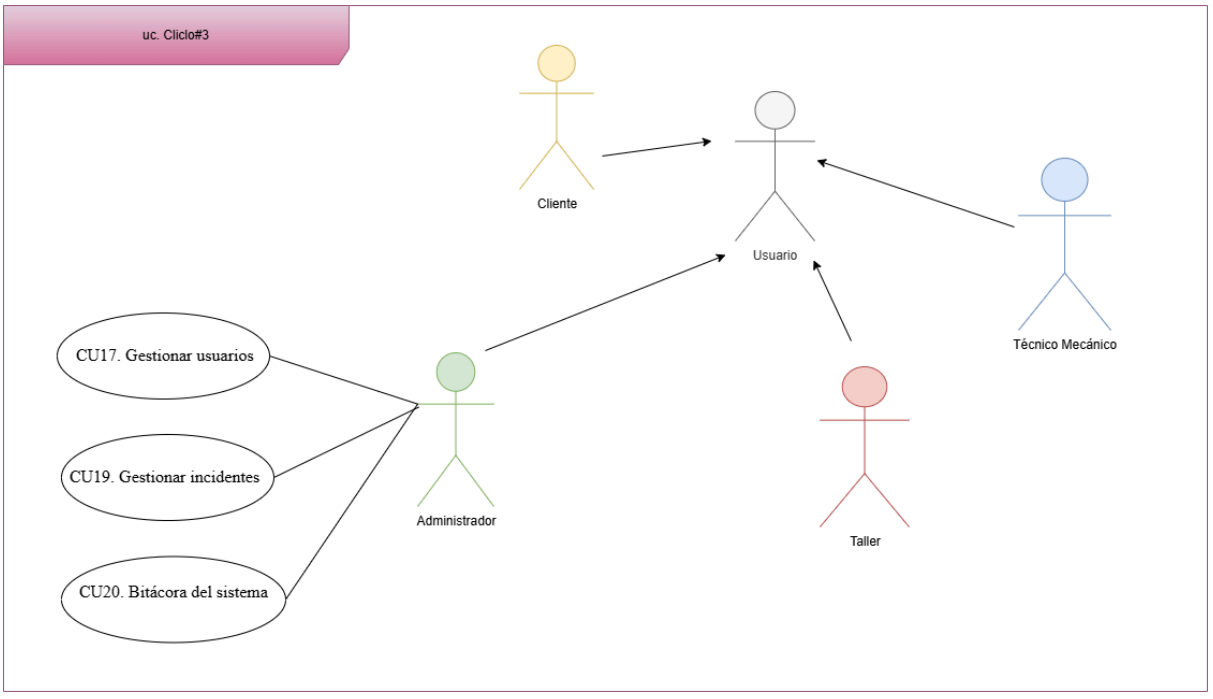
Ciclo I#1


uc. Ciclo#1
CU3. Cerrar sesión
CU2. Iniciar sesión
Usuario
Técnico Mecánico
CU1. Registrarse en el sistema
CU4. Gestionar perfil de usuario
Cliente
Taller
CU5. Registrar vehículo
Administrador
CU18. Gestionar taller
CU6. Gestionar información del vehículo

Ciclo #2


uc. Ciclo#2
CU7. Gestionar emergencias
CU8. Consultar estado de la solicitud
CU9. Realizar pago del servicio
CU16. Generar lista de talleres candidatos
Cliente
Usuario
Técnico Mecánico
Administrador
Taller
CU14. Actualizar estado del servicio
CU15. Consultar historial de atenciones
CU10. Registrar taller
CU13. Gestionar solicitudes (Taller)
CU11. Gestionar técnicos del taller
CU12. Gestionar disponibilidad de técnicos

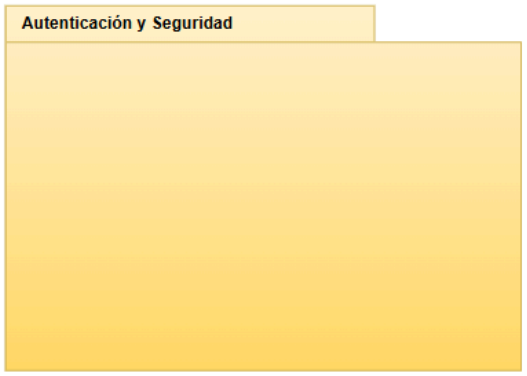
Ciclo #3


uc. Ciclo#3
Cliente
Usuario
Técnico Mecánico
CU17. Gestionar usuarios
CU19. Gestionar incidentes
CU20. Bitácora del sistema
Administrador
Taller

3.2) FLUJO DE TRABAJO: ANÁLISIS

3.2.1 Análisis de Arquitectura

3.2.1.1 Identificar Paquetes



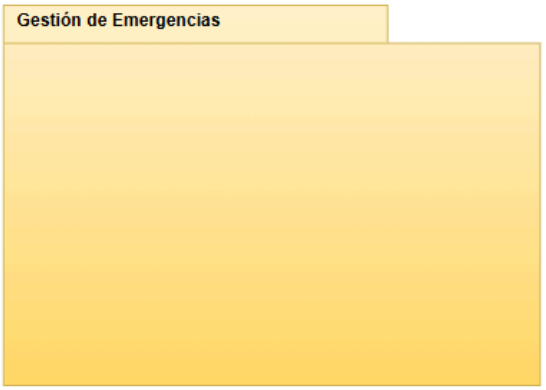
Se implementarán mecanismos de autenticación y control de acceso basados en roles. Los usuarios deberán iniciar sesión mediante credenciales válidas para acceder a las funcionalidades del sistema según su perfil. El sistema permitirá el registro de nuevos usuarios, el cierre de sesión seguro y la protección de rutas mediante validación de tokens. Además, se gestionarán permisos y se garantizará la seguridad de las sesiones a través de interceptores y controles de acceso.



Se permitirá la administración de la información de los usuarios del sistema. Incluye la gestión del perfil personal, actualización de datos y control de usuarios por parte del administrador. Se garantizará la integridad de la información y la correcta asignación de roles, permitiendo diferenciar entre clientes, talleres y administradores.



Se permitirá a los usuarios registrar y administrar sus vehículos, almacenando información relevante como marca, modelo, placa y características. Este módulo facilitará la asociación de vehículos con emergencias, permitiendo una atención más eficiente y contextualizada de los incidentes reportados.



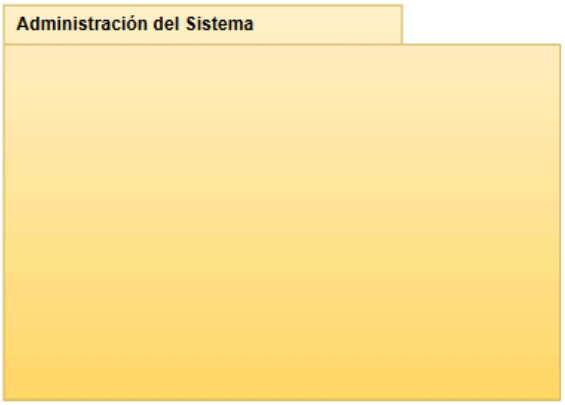
Se encargará de la gestión de emergencias vehiculares. Permitirá registrar incidentes, enviar la ubicación en tiempo real y adjuntar evidencias como imágenes, audios o texto. Además, los usuarios podrán consultar el estado de su solicitud, visualizar el taller asignado y obtener información sobre el tiempo estimado de atención.



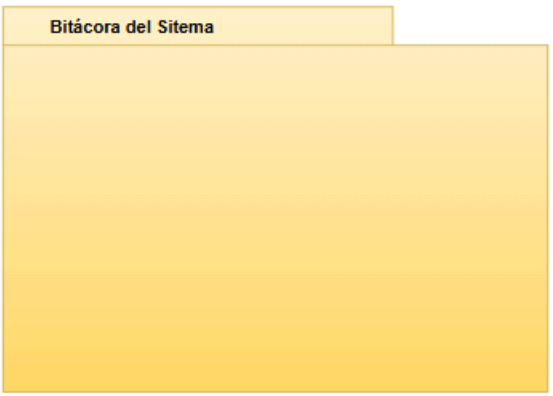
Se permitirá registrar talleres y administrar su información, incluyendo la gestión de técnicos y su disponibilidad. Este módulo facilita la organización interna de los talleres, permitiendo asignar recursos de manera eficiente para atender las solicitudes de servicio.



Se encargará de procesar los pagos correspondientes a los servicios prestados. Permitirá a los usuarios realizar pagos de forma segura, gestionando transacciones y asegurando la integridad de la información financiera dentro del sistema.



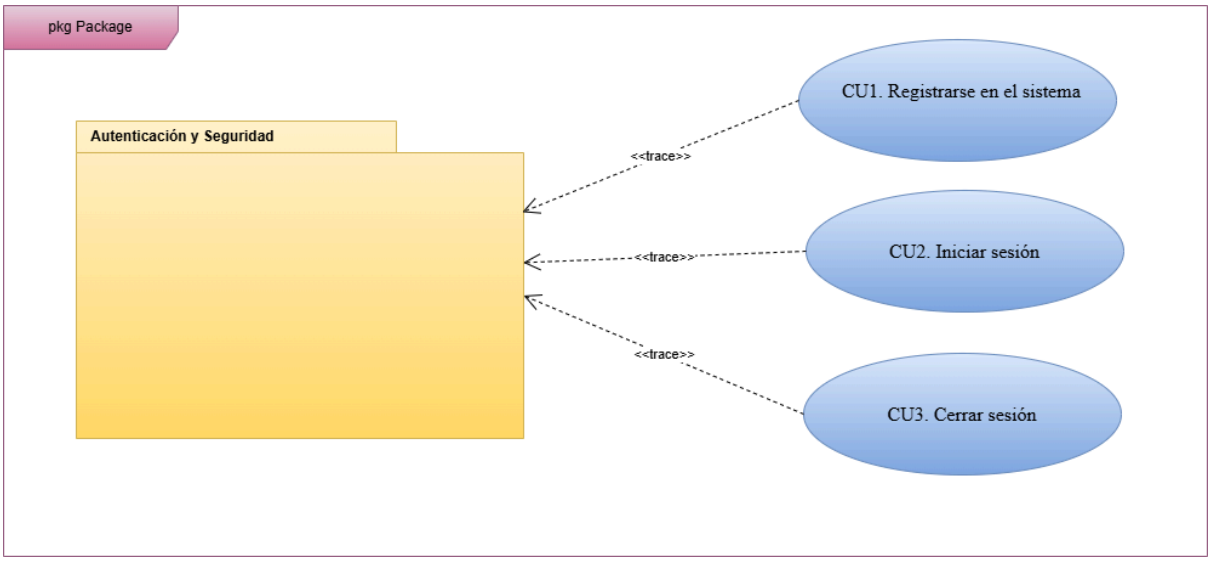
Se permitirá la gestión global del sistema, incluyendo la administración de usuarios, talleres e incidentes. Además, incorpora una bitácora que registra las actividades relevantes, facilitando la auditoría, supervisión y control del funcionamiento general del sistema.



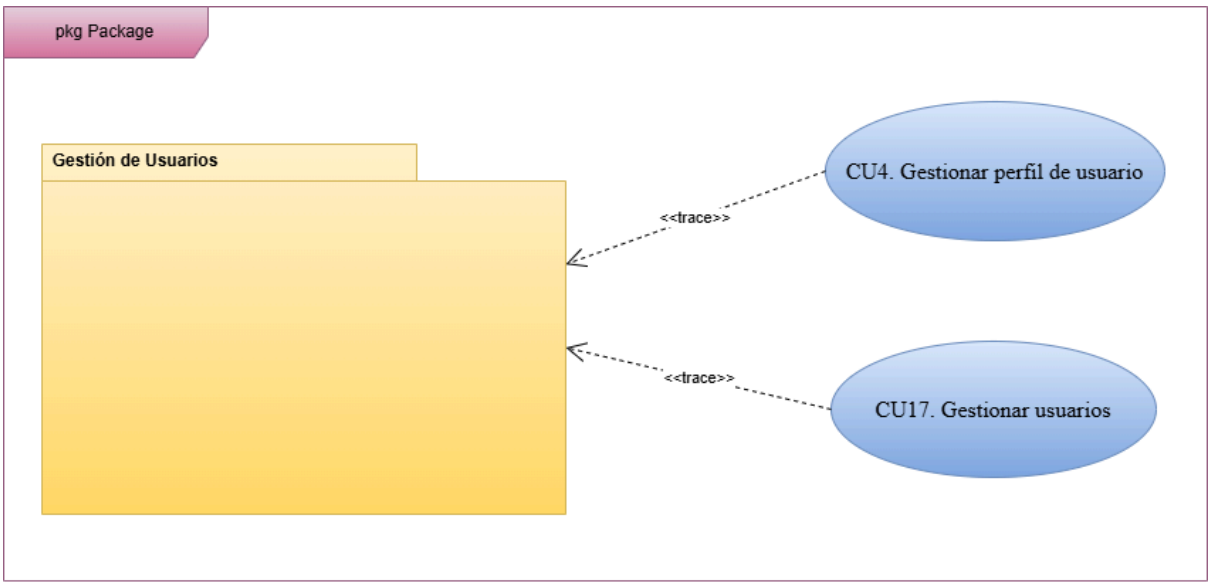
Se encargará del registro y seguimiento de todas las acciones relevantes realizadas dentro del sistema, almacenando información detallada como el usuario, la fecha, el tipo de operación y los cambios efectuados. Este paquete permite la consulta y análisis de la información registrada, facilitando la auditoría, el control y la trazabilidad de las actividades para garantizar la transparencia y el correcto funcionamiento del sistema.

3.2.1.2 Relacion de Paquetes y Casos de Uso

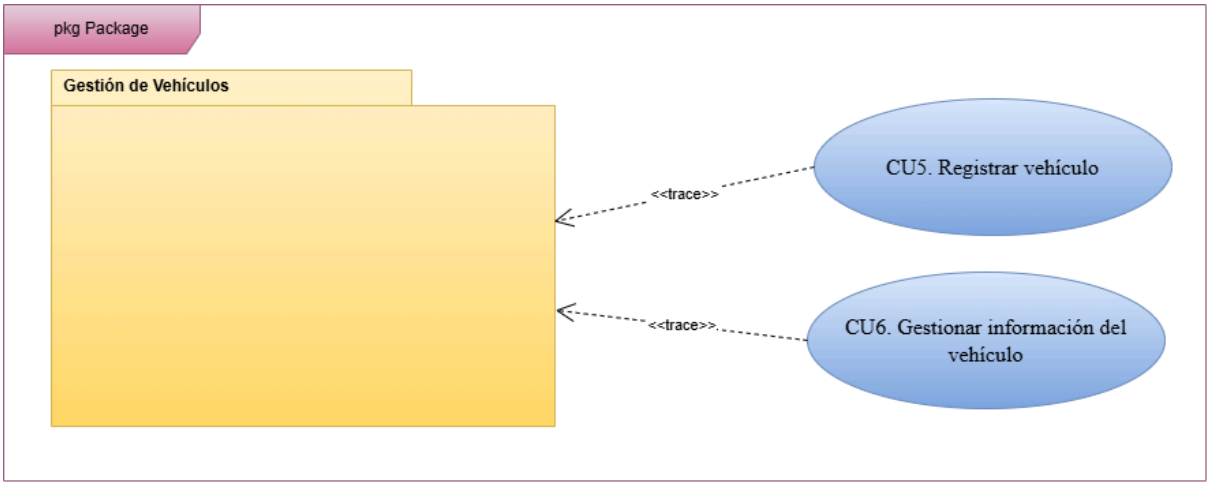
paquete Autenticación y Seguridad



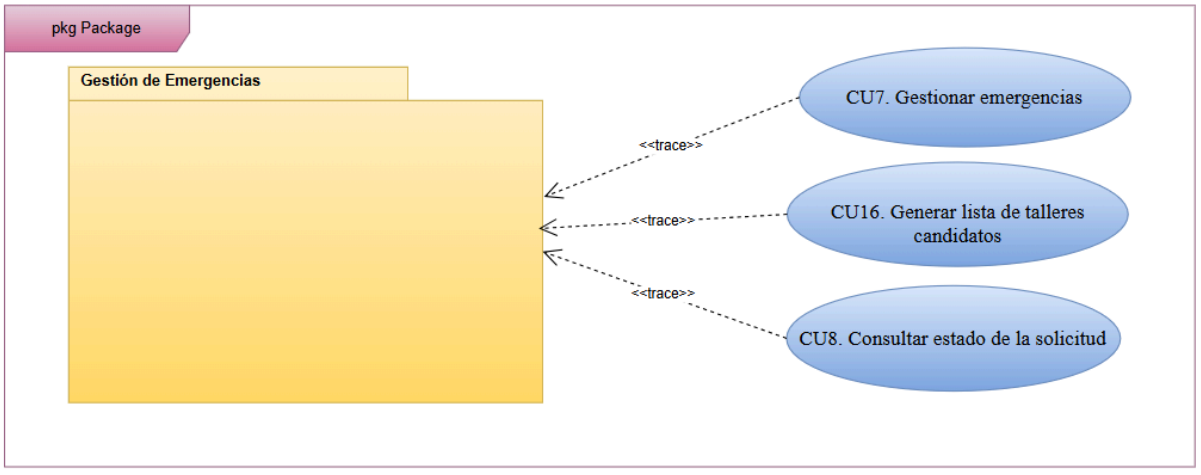
paquete Gestion de Usuarios



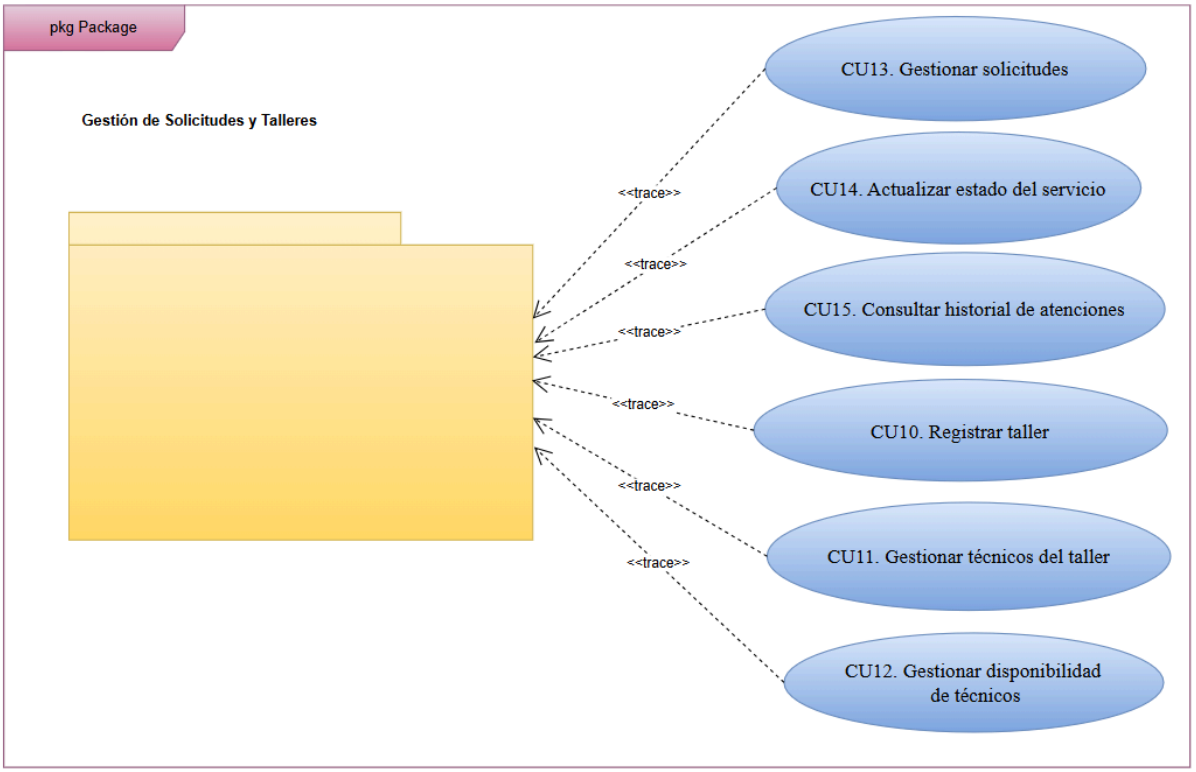
paquete Gestion de Vehiculos



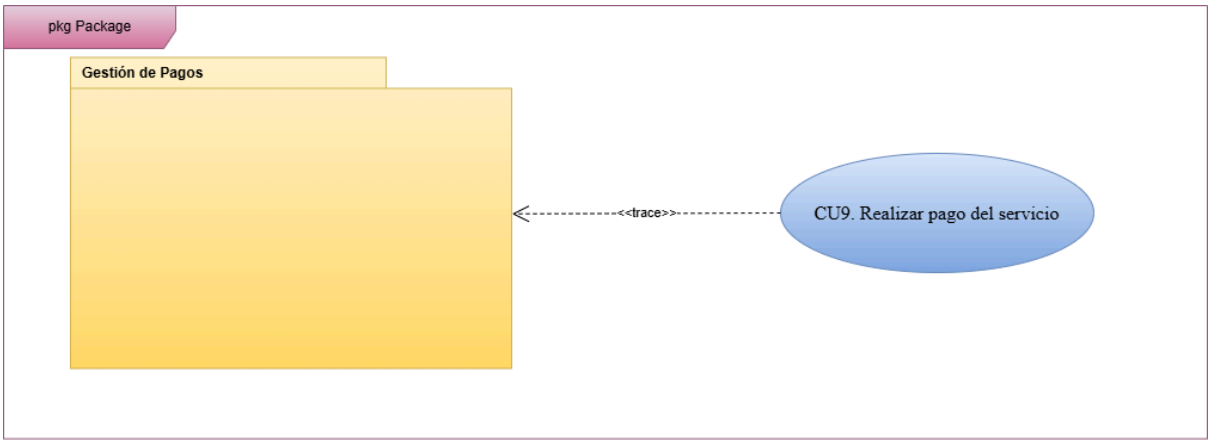
paquete Gestión de Emergencias



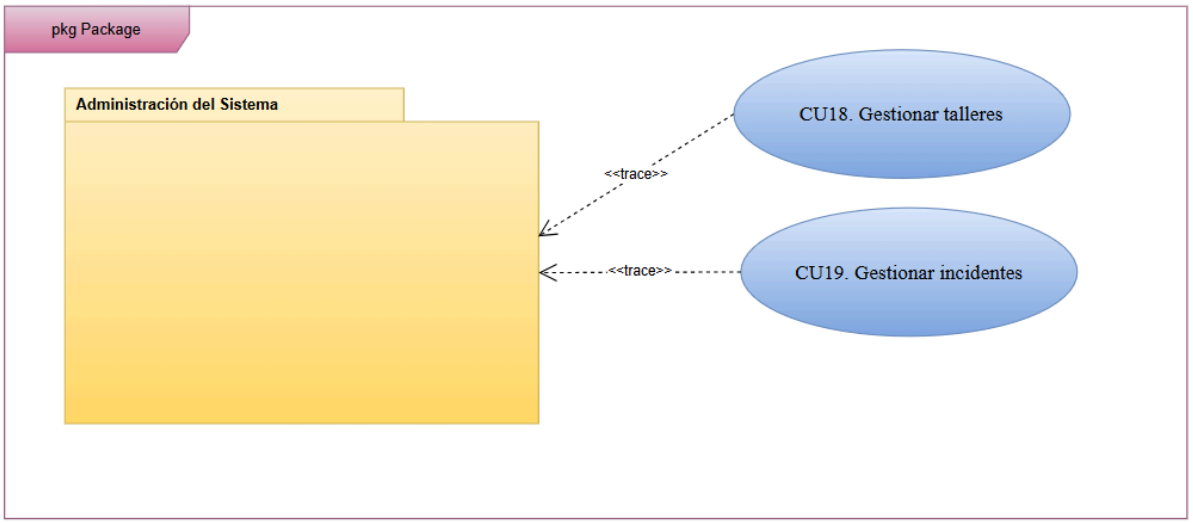
paquete Gestión de Solicitudes y Talleres



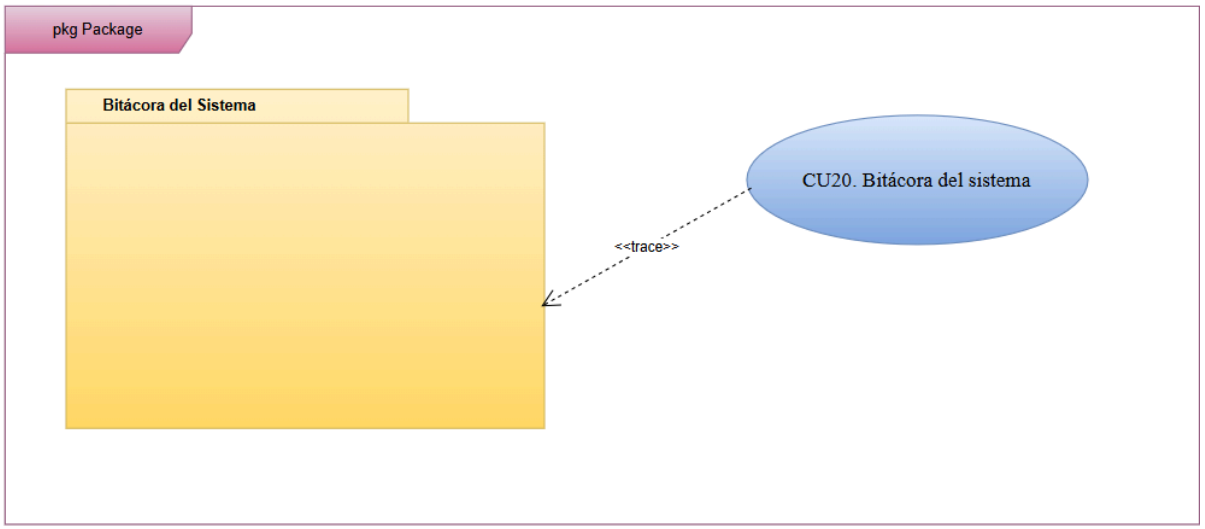
paquete Gestión de Pagos



paquete Administración del Sistema

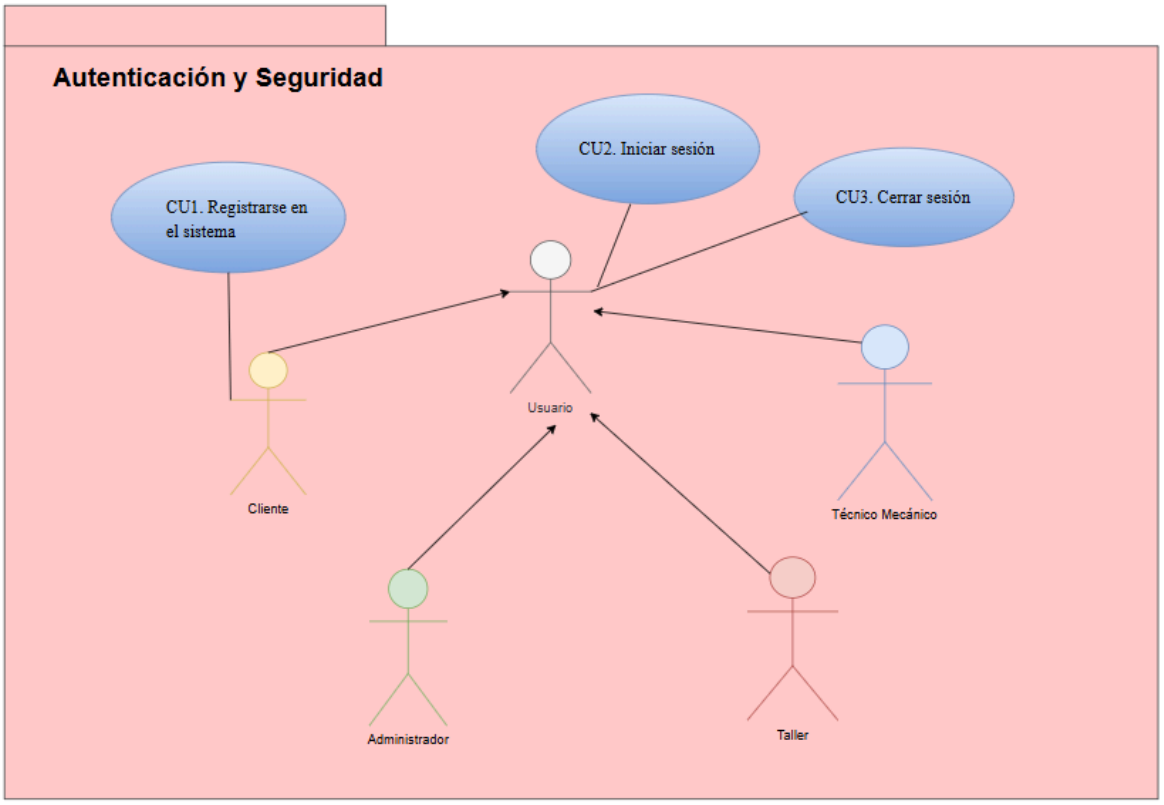


paquete Bitácora del Sistema

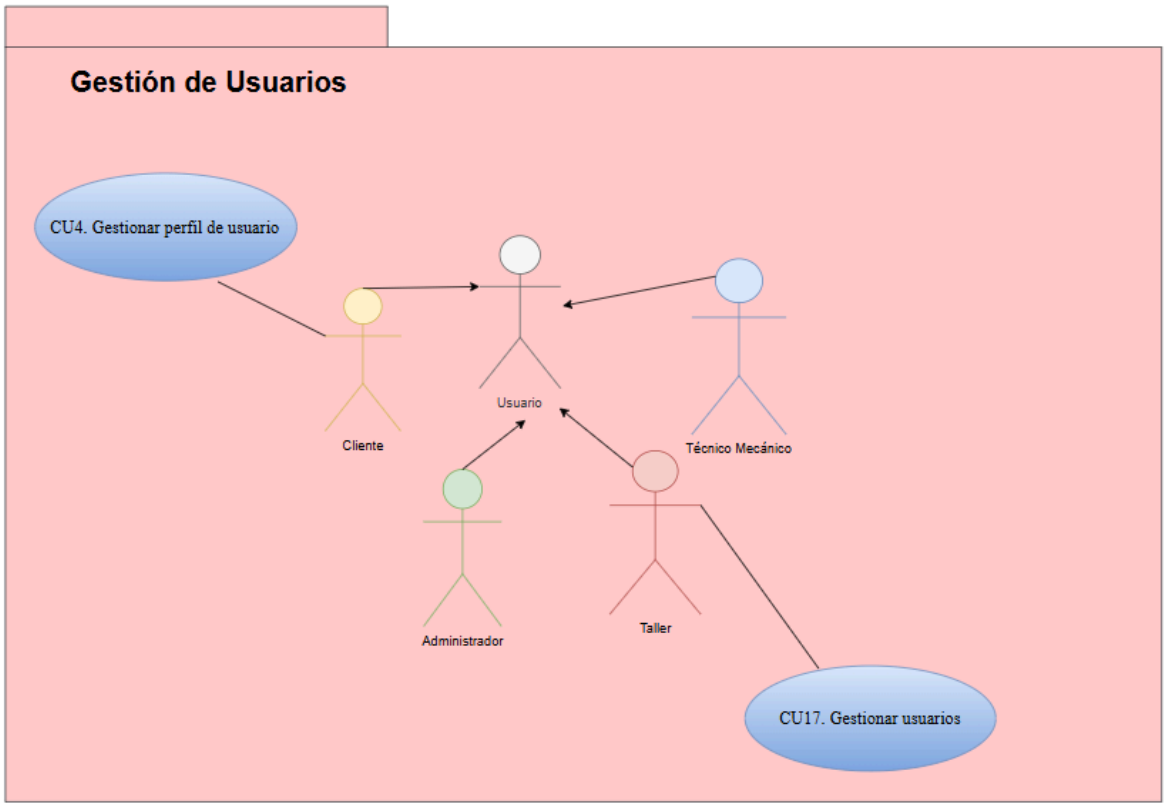


3.2.1.3 Vista de Paquete (Encapsular CU)

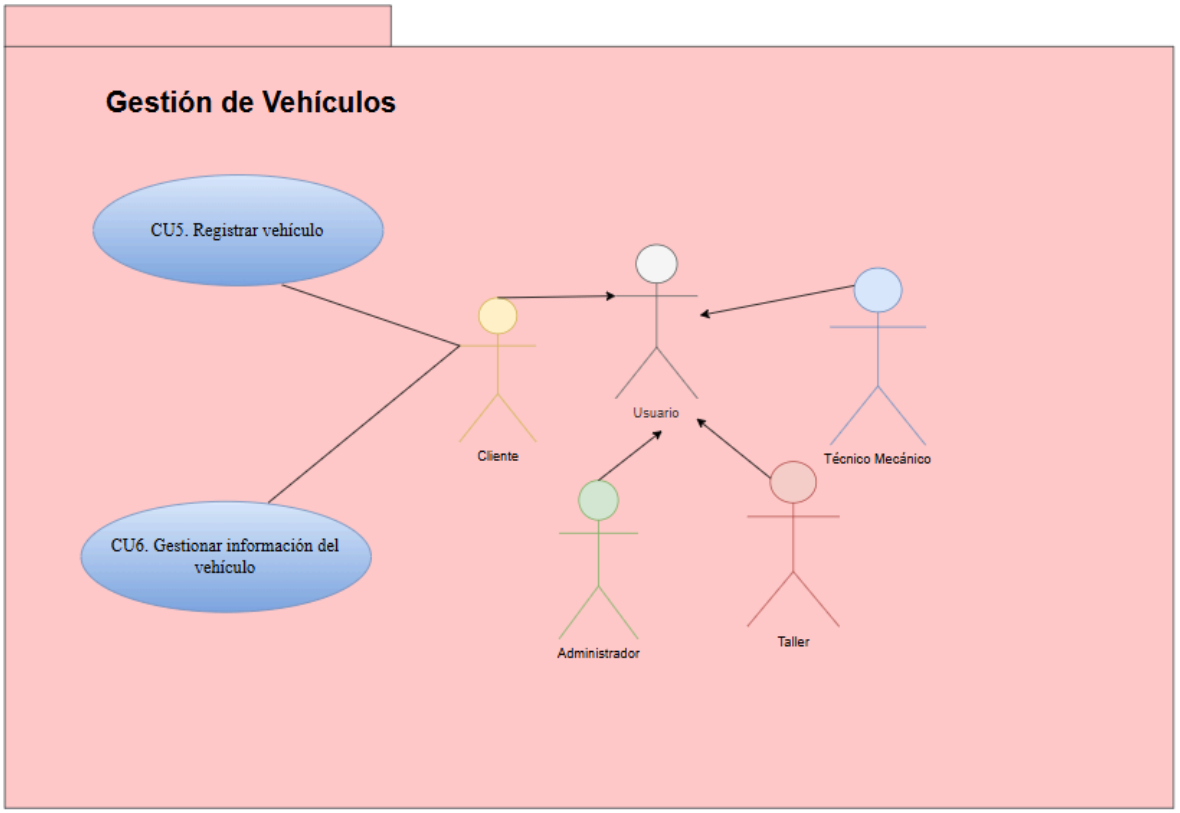
paquete Autenticación y Seguridad (auth)



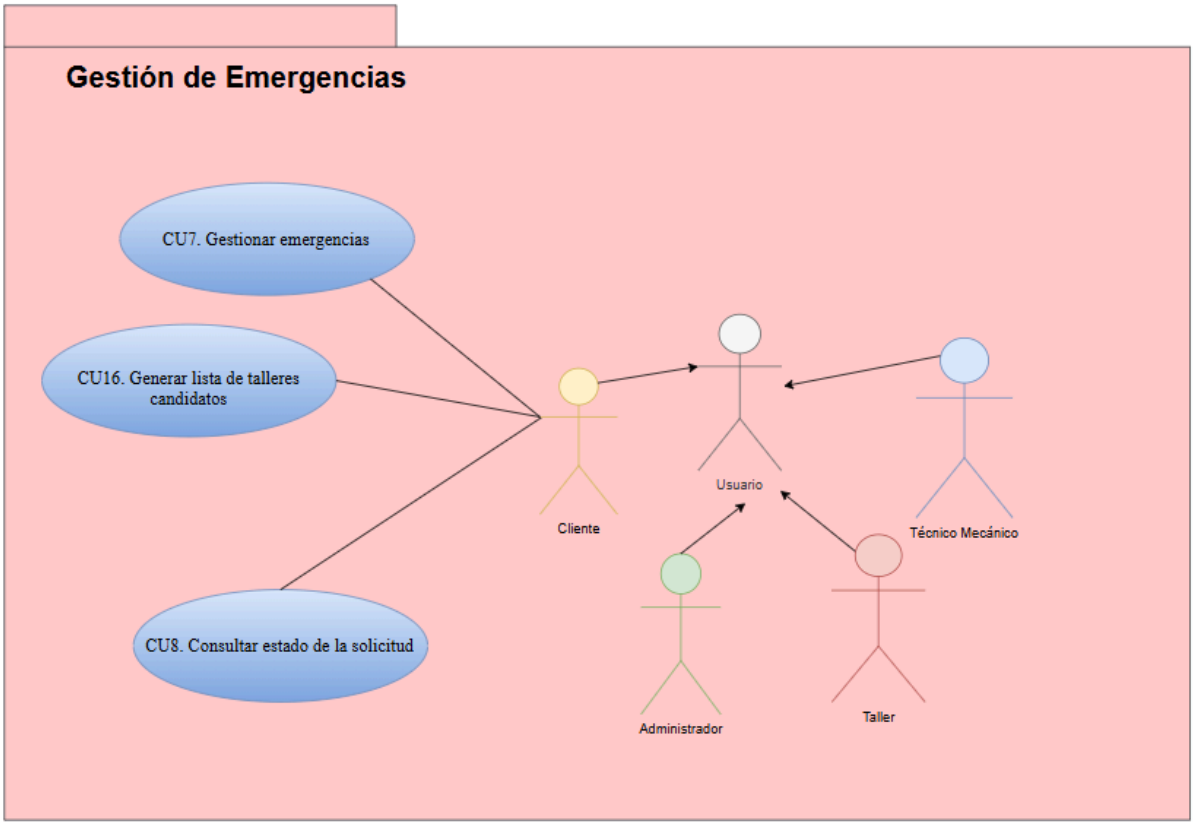
paquete Gestion de Usuarios (users)



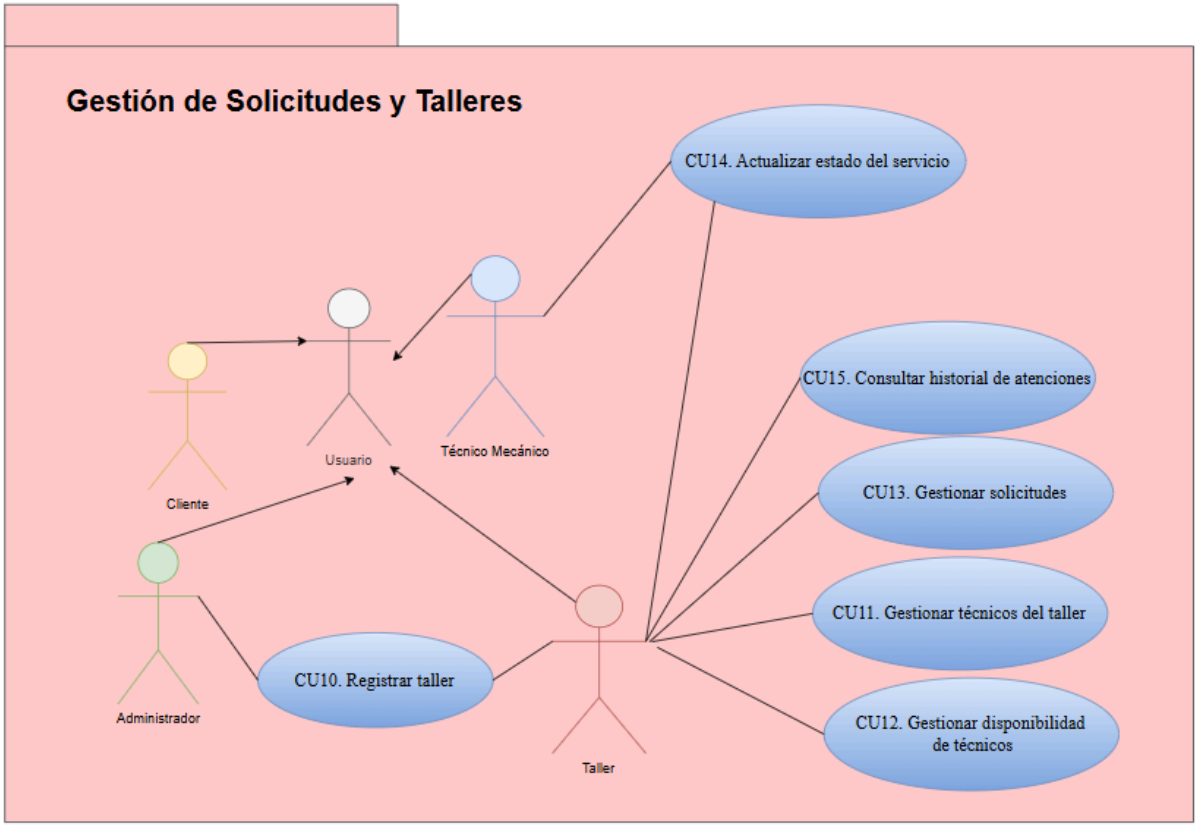
paquete Gestion de Vehiculos (vehicles)



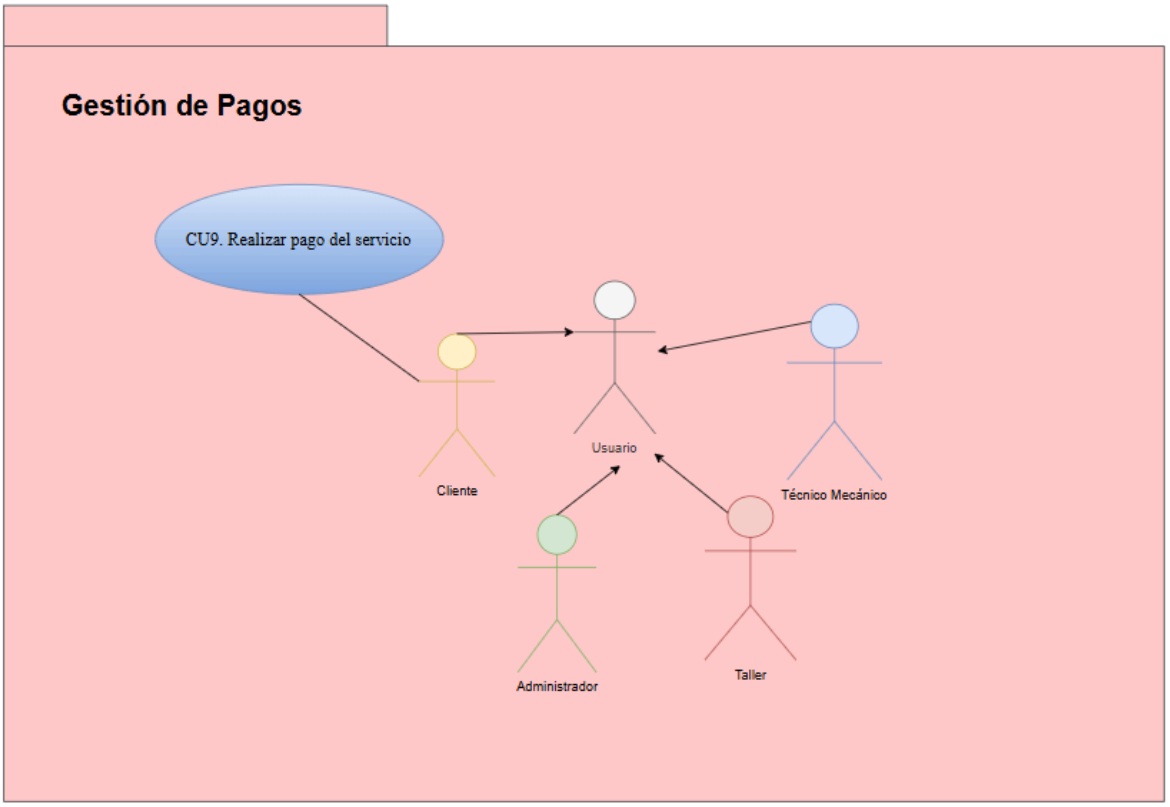
paquete Gestión de Emergencias (Incident)



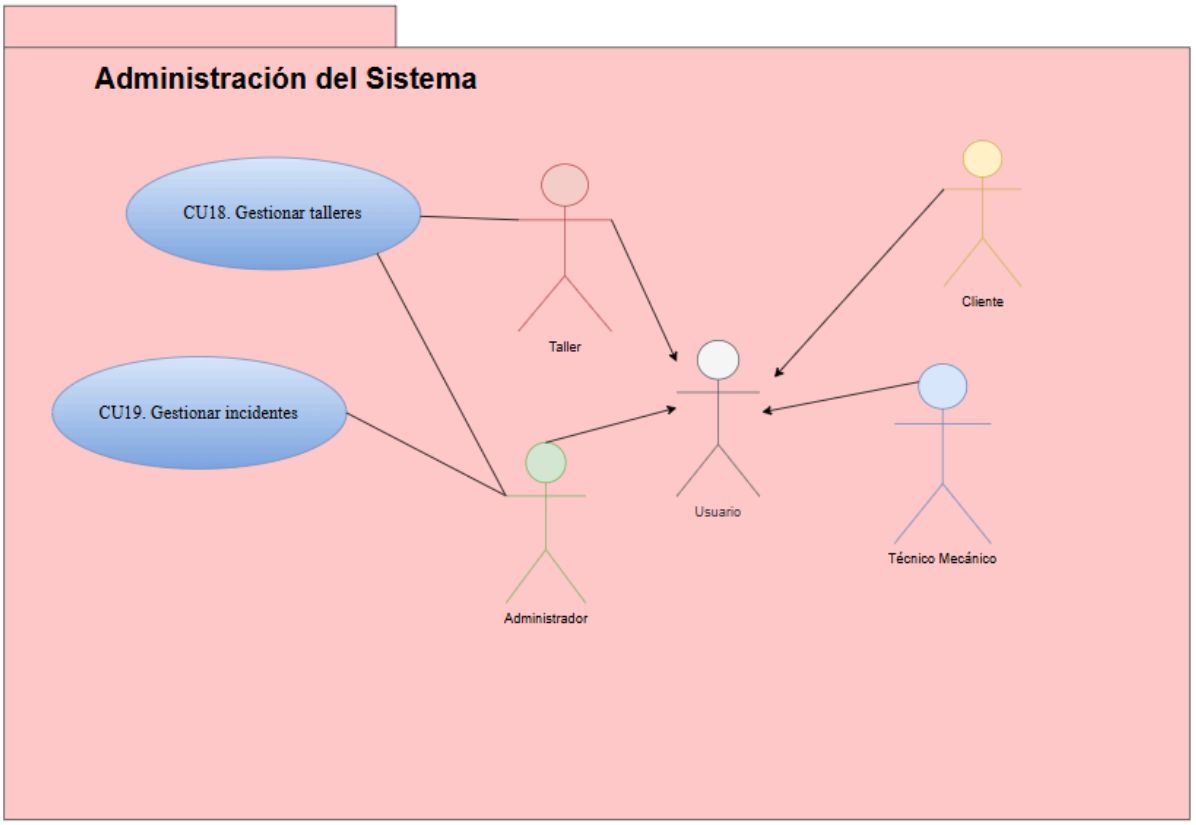
paquete Gestión de Solicitudes y Talleres (workshop)



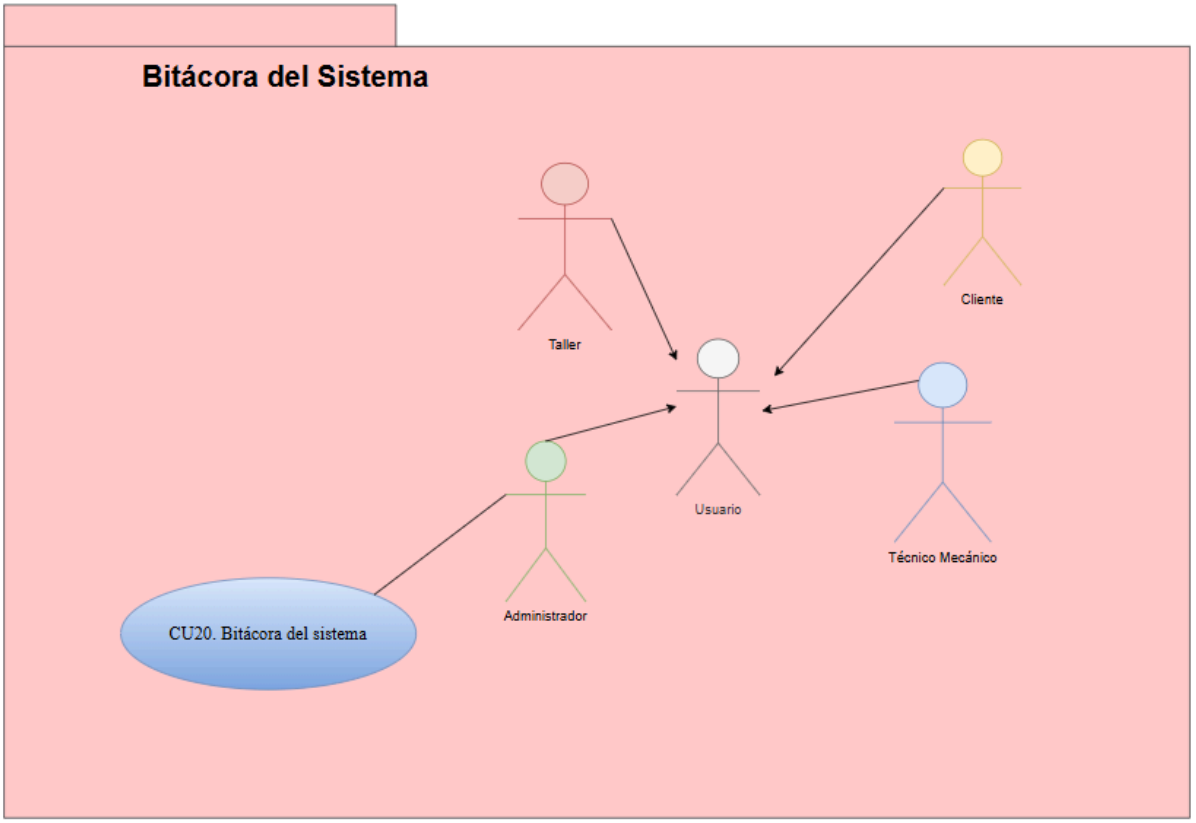
paquete Gestión de Pagos (payments)



paquete Administración del Sistema(admin)



paquete Bitácora (audit_logs)

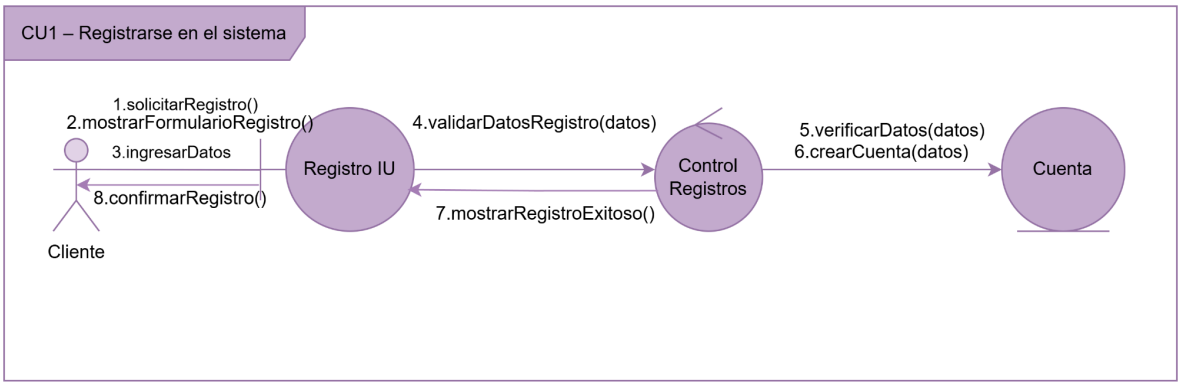


3.1.2 Analizar un Caso de Uso

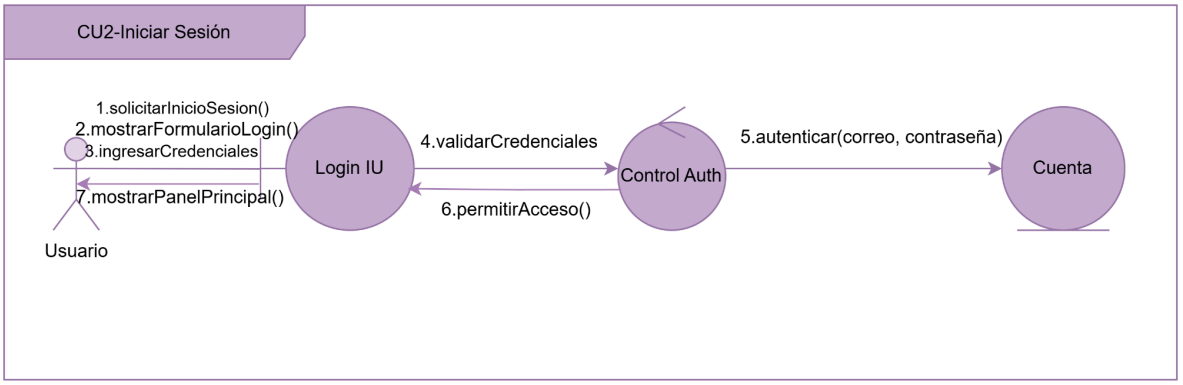
3.1.2.1 Diagrama de Comunicacion

Ciclo #1

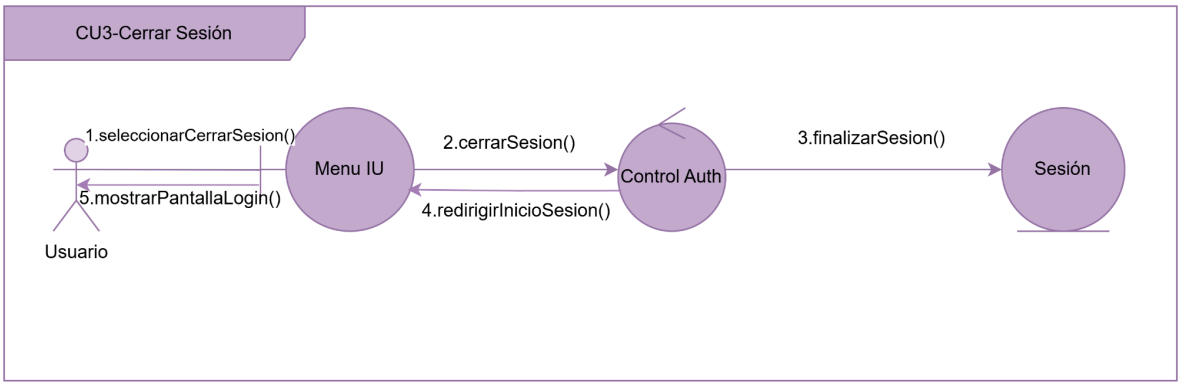
CU1 - Registrarse en el sistema



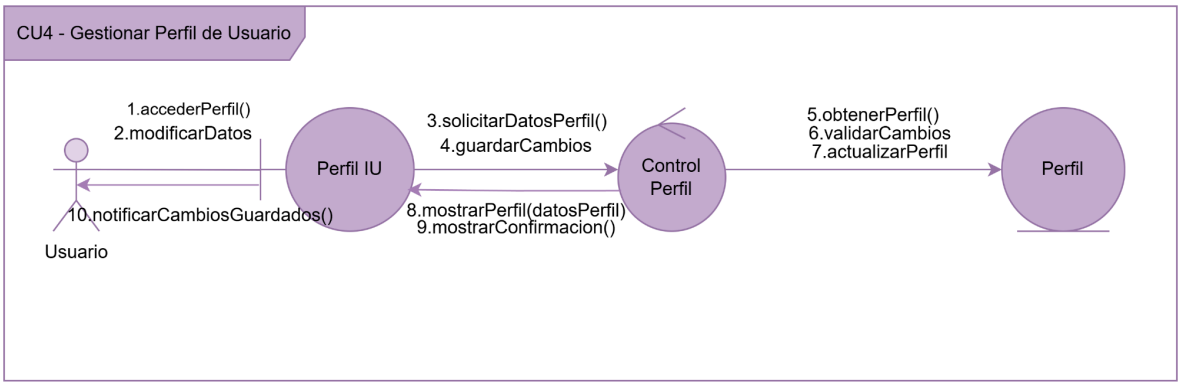
CU2 - Iniciar Sesión



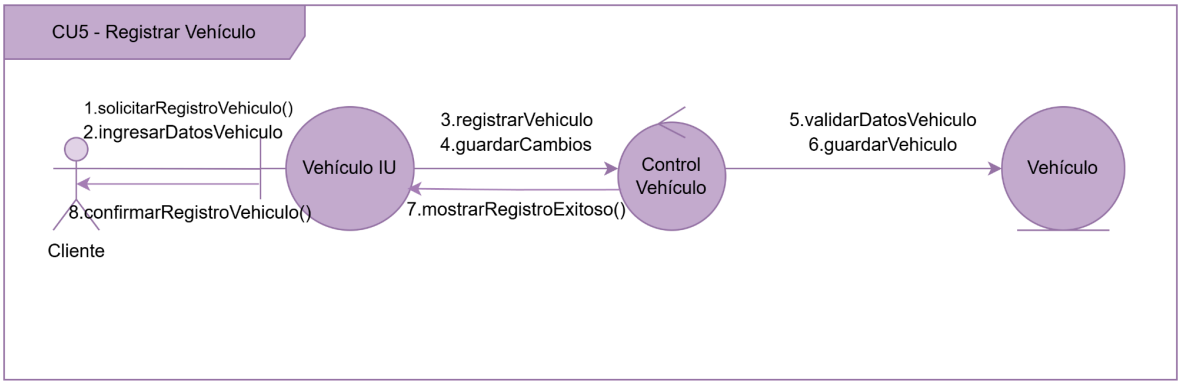
CU3 - Cerrar Sesión



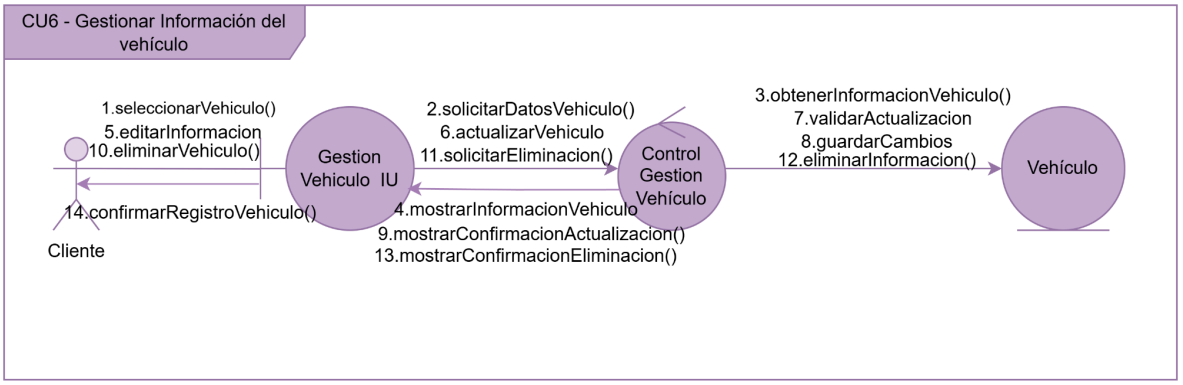
CU4 - Gestionar Perfil de Usuario



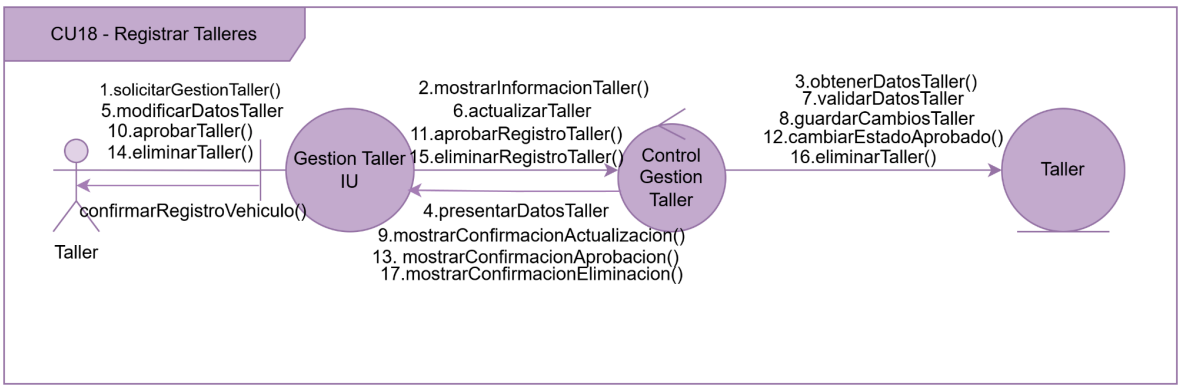
CU5 - Registrar Vehículo



CU6 – Gestionar información del vehículo

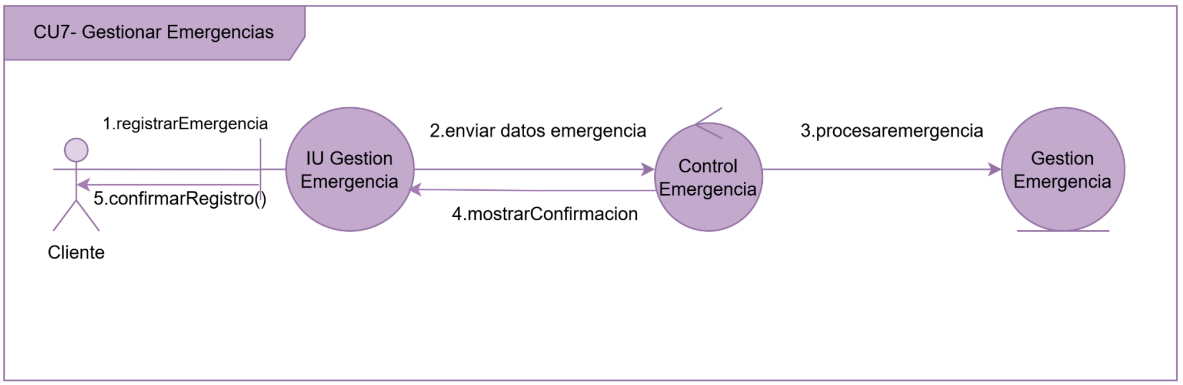


CU18 - Registrar Talleres

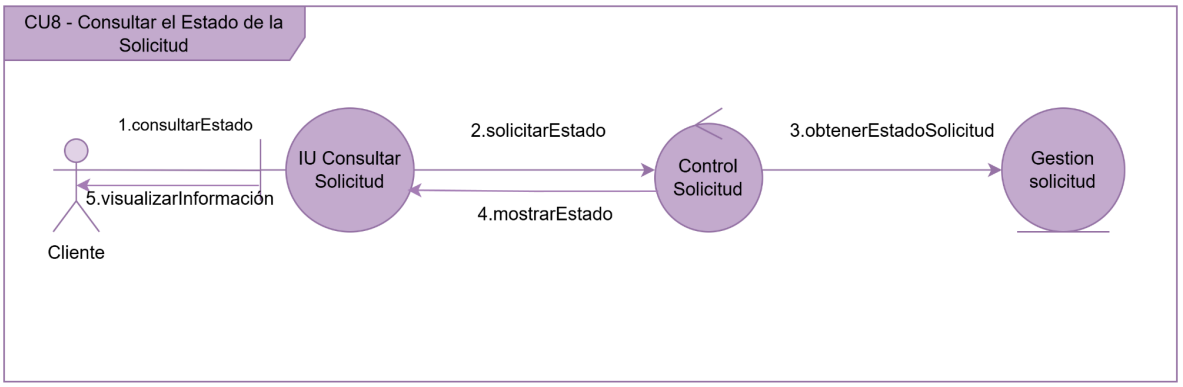


Ciclo #2

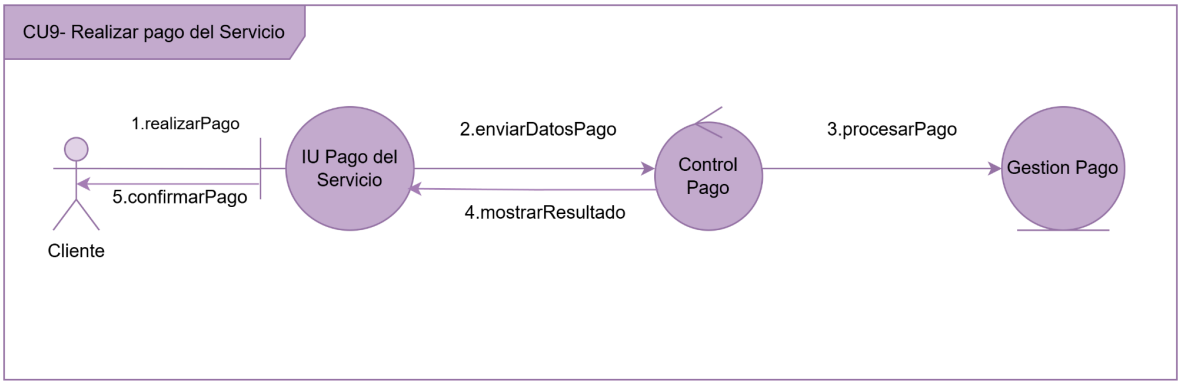
CU7- Gestionar Emergencias



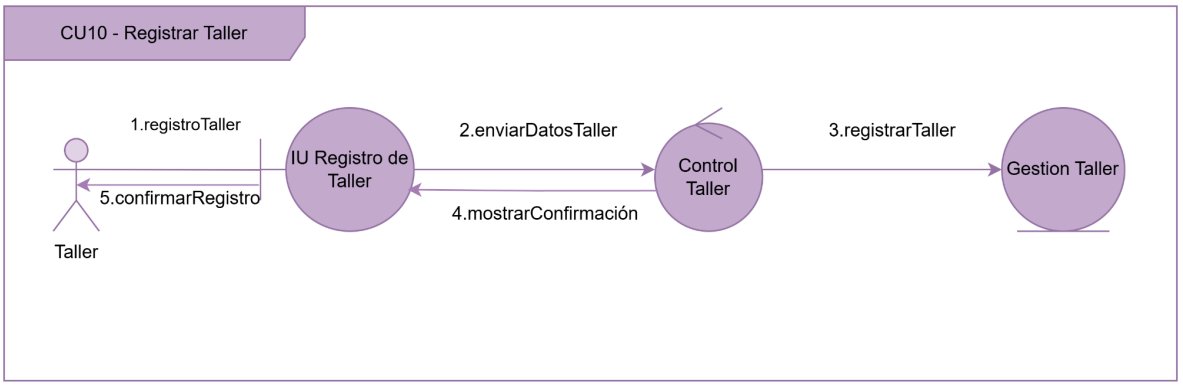
CU8 - Consultar Estado de la Solicitud



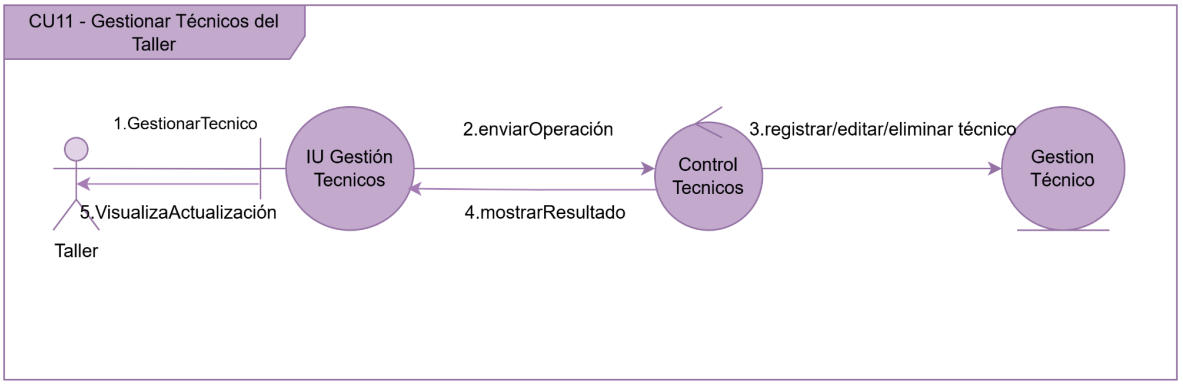
CU9 - Realizar Pago de Servicio



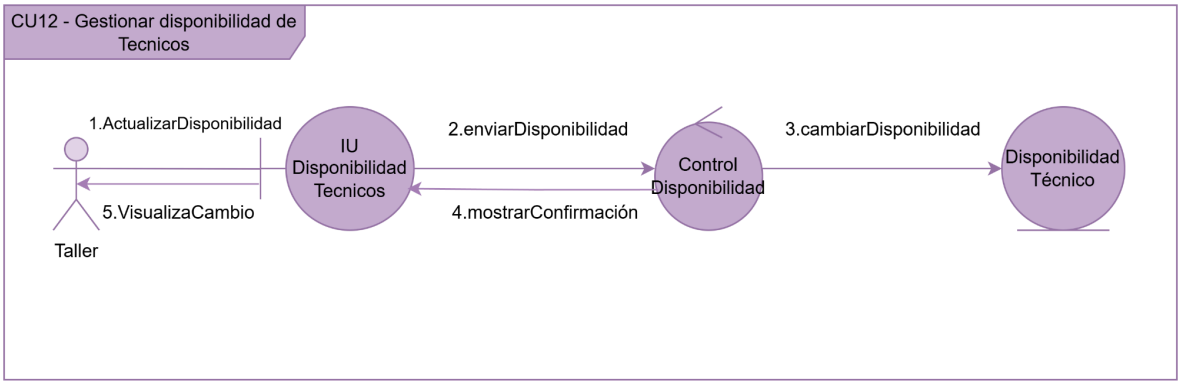
CU10 - Registrar Taller



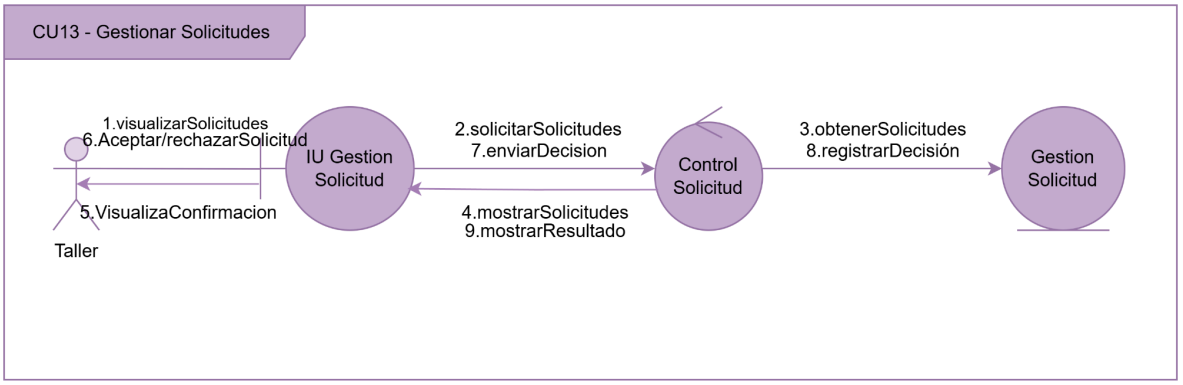
CU11 - Gestionar Técnicos del Taller



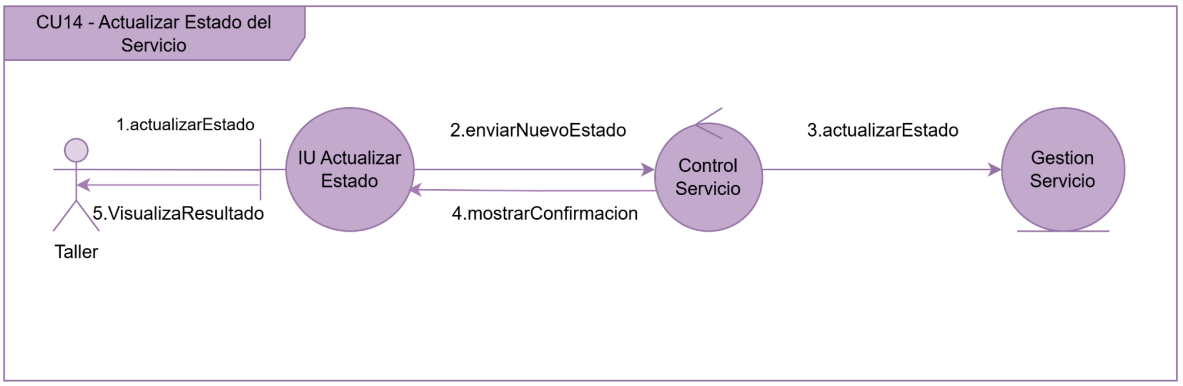
CU12-Gestionar Disponibilidad de Técnicos



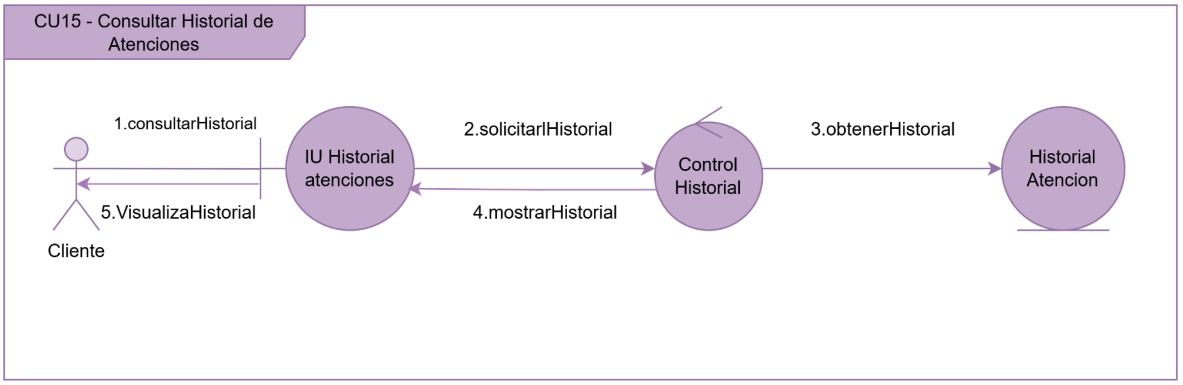
CU13- Gestionar Solicitudes



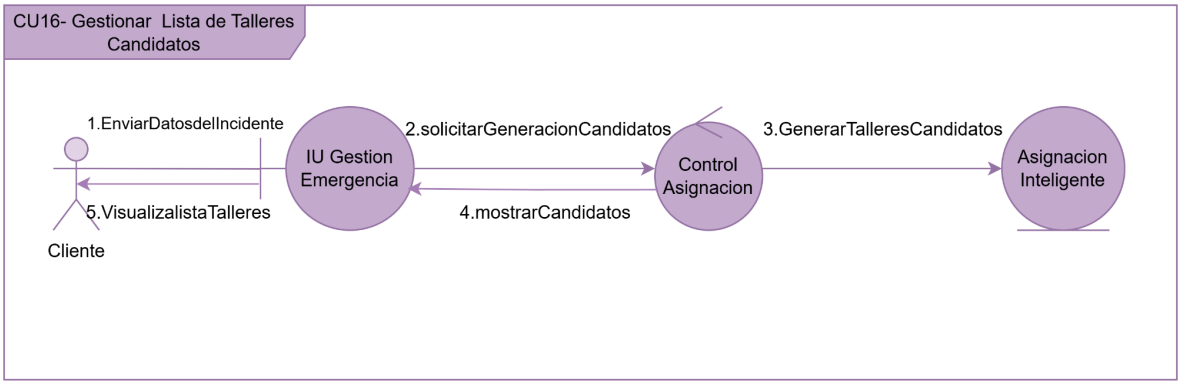
CU14- Actualizar Estado del Servicio



CU15 - Consultar Historial de Atenciones

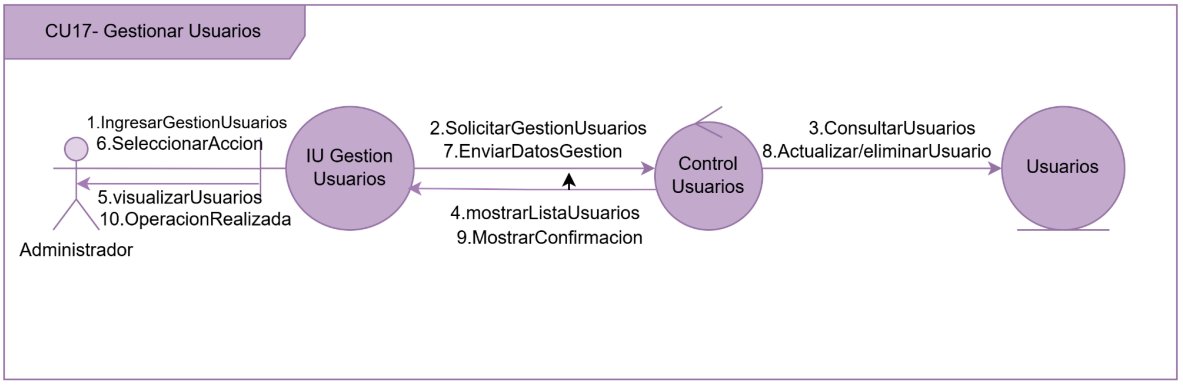


CU16 - Gestionar Lista de Talleres Candidatos

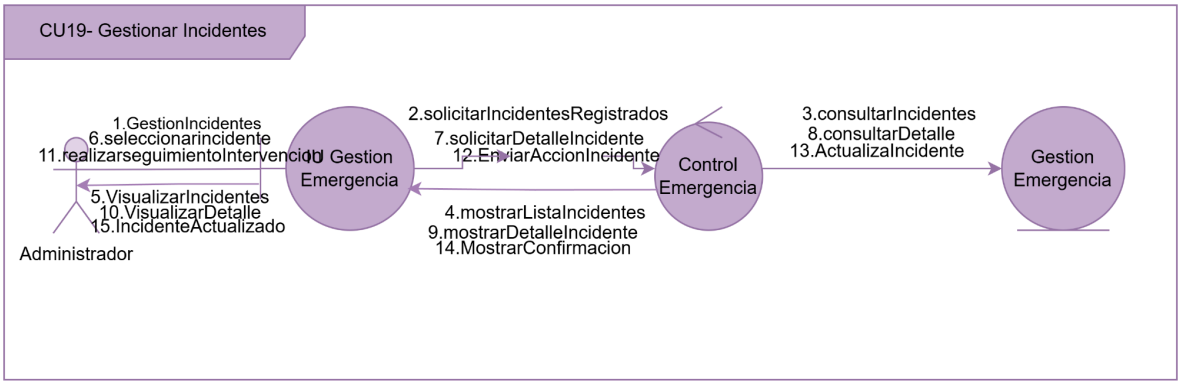


Ciclo # 3

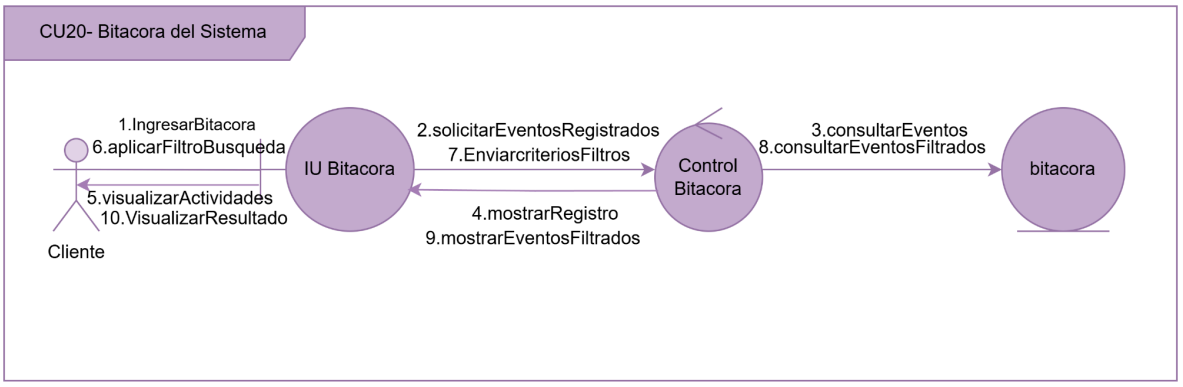
CU17. Gestionar Usuarios



CU19. Gestionar Incidentes



CU20. Bitácora del Sistema

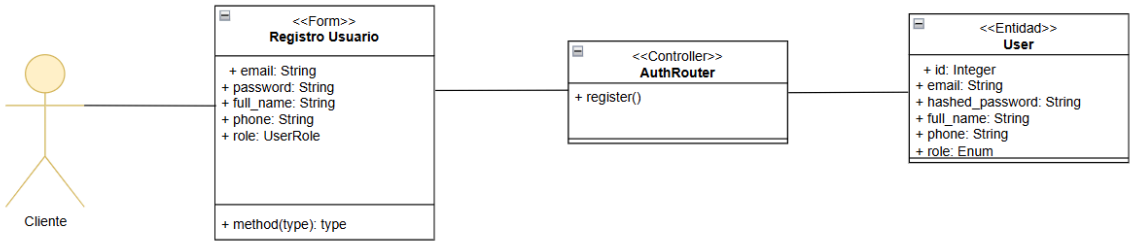


3.1.3 Análisis de Clase

Ciclo #1

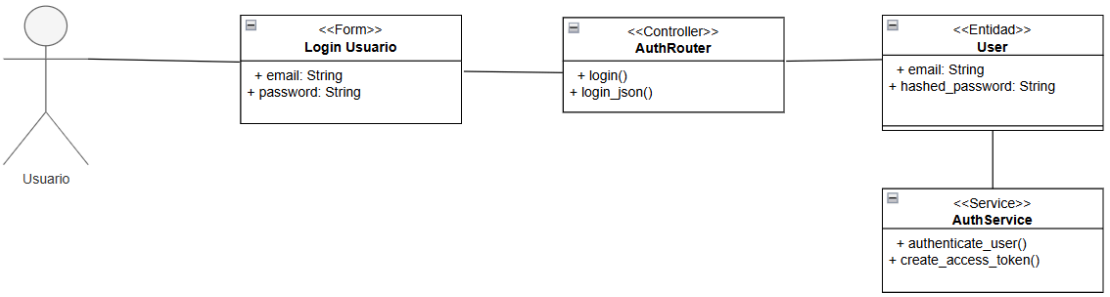
CU1. Registrarse en el sistema (cliente)

CU1. Registrarse en el sistema



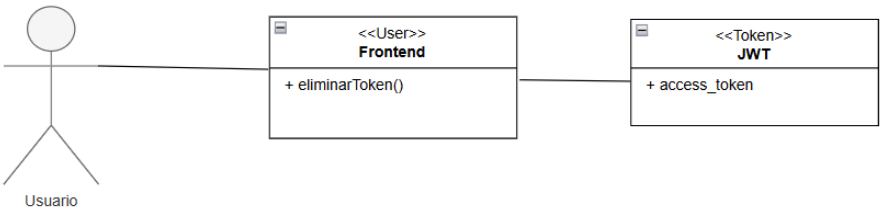
CU2. Iniciar sesión

CU2. Iniciar sesión



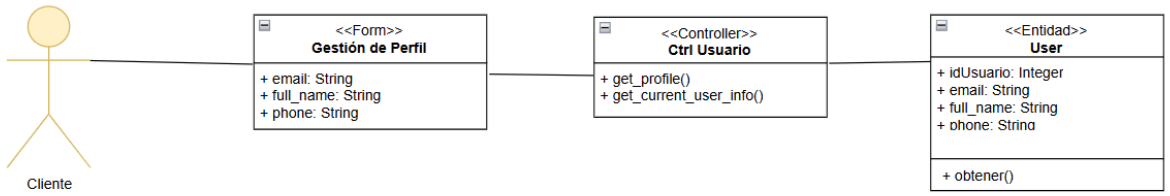
CU3. Cerrar sesión

CU3. Cerrar sesión



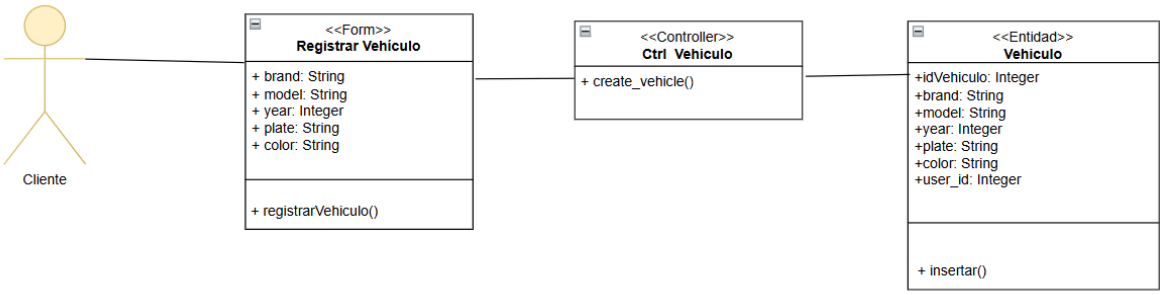
CU4. Gestionar perfil de usuario

CU4. Gestionar perfil de usuario



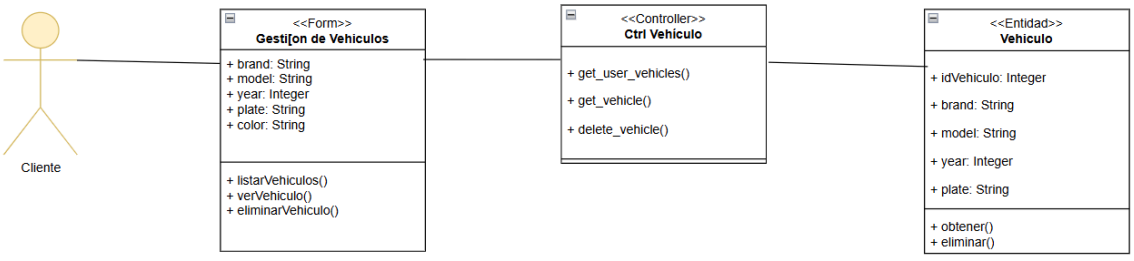
CU5. Registrar vehículo

CU5. Registrar vehículo



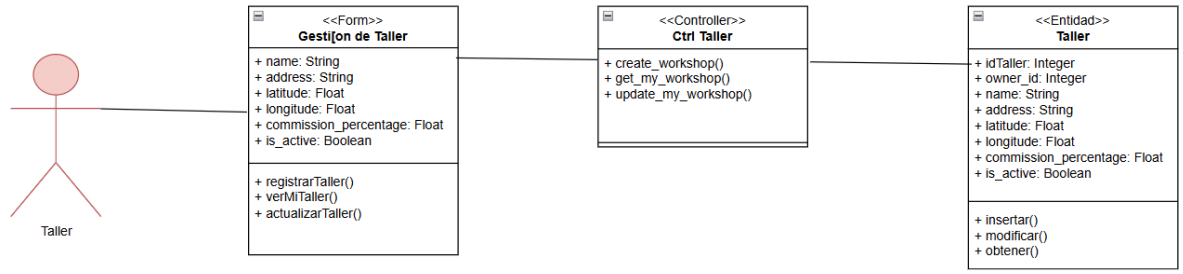
CU6. Gestionar información del vehículo

CU6. Gestionar información del vehículo



CU18. Gestionar talleres

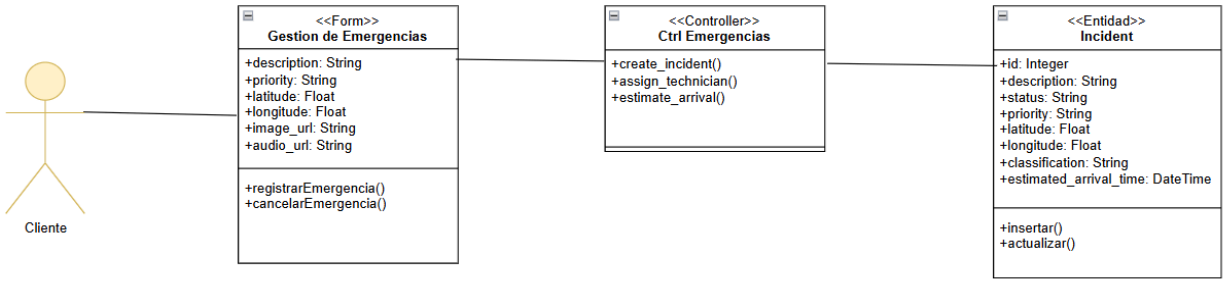
CU18. Gestionar taller



Ciclo #2

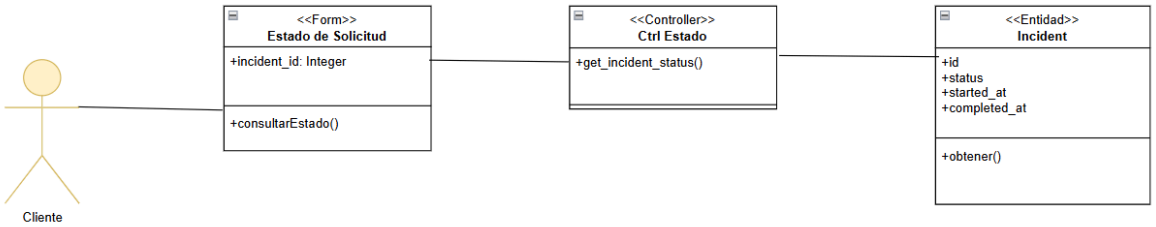
CU7- Gestionar Emergencias

CU7- Gestionar Emergencias



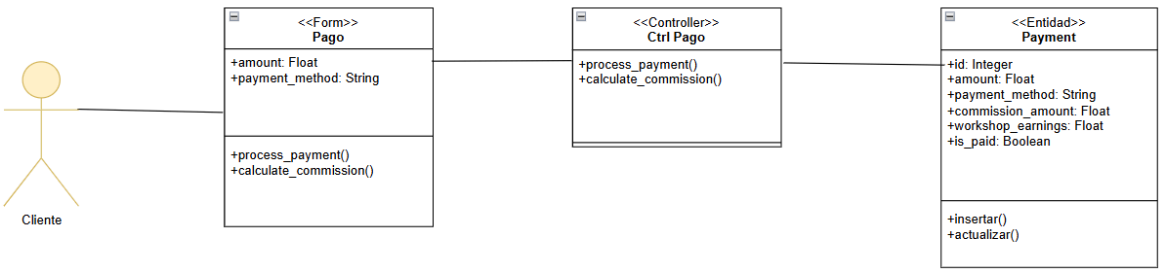
CU8 - Consultar Estado de la Solicitud

CU8 - Consultar Estado de la Solicitud



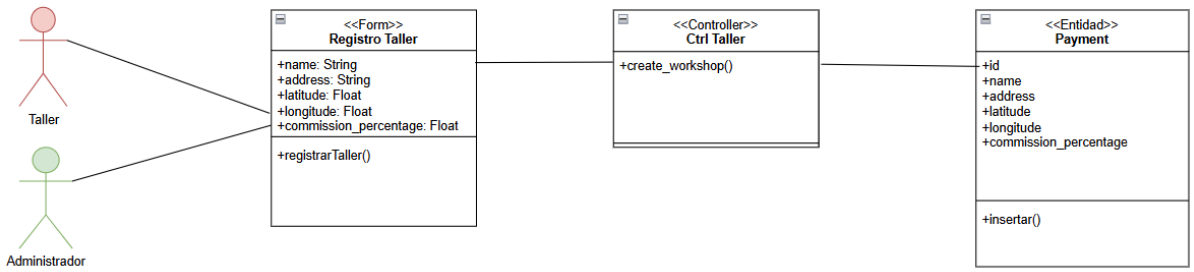
CU9 - Realizar Pago de Servicio

CU9 - Realizar Pago de Servicio



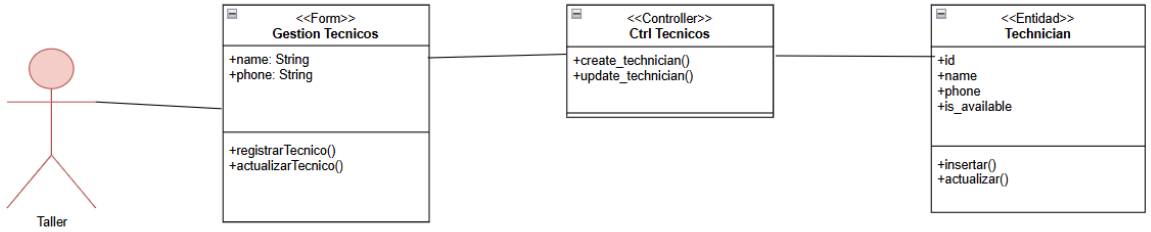
CU10 - Registrar Taller

CU10 - Registrar Taller



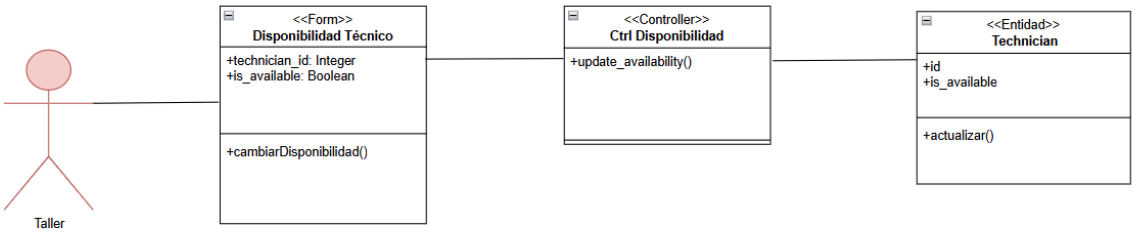
CU11 - Gestionar Técnicos del Taller

CU11 - Gestionar Técnicos del Taller



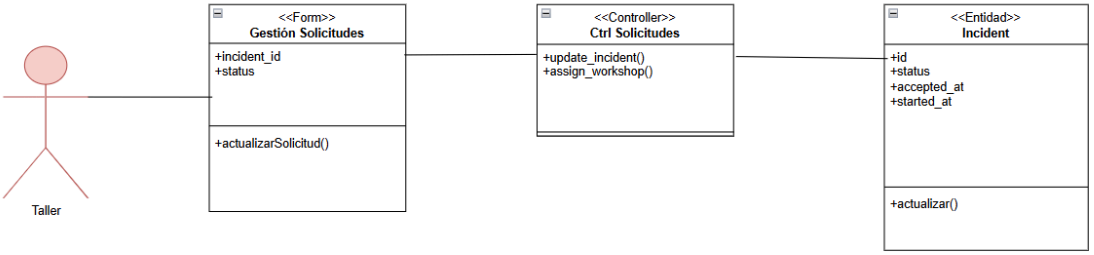
CU12-Gestionar Disponibilidad de Técnicos

CU12-Gestionar Disponibilidad de Técnicos



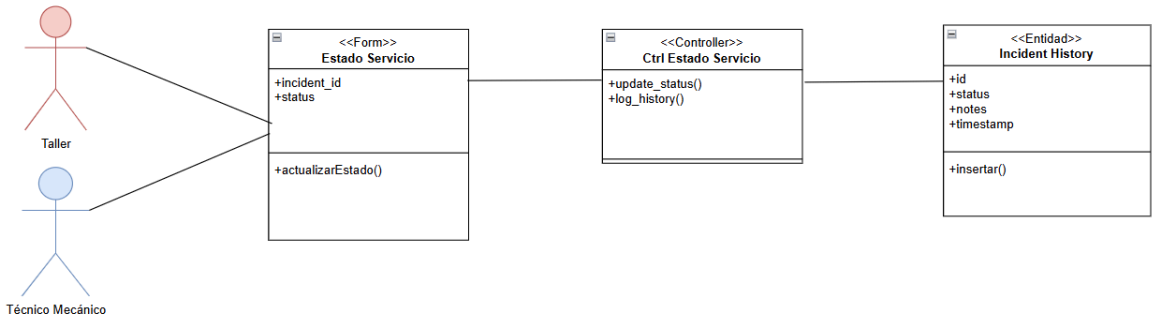
CU13- Gestionar Solicitudes

CU13- Gestionar Solicitudes



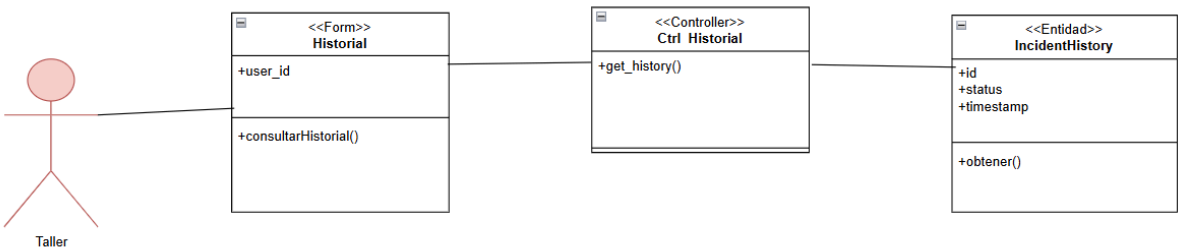
CU14- Actualizar Estado del Servicio

CU14- Actualizar Estado del Servicio



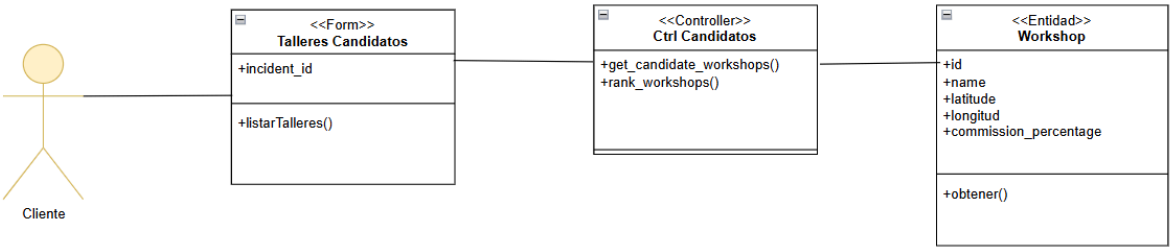
CU15 - Consultar Historial de Atenciones

CU15 - Consultar Historial de Atenciones



CU16 - Gestionar Lista de Talleres Candidatos

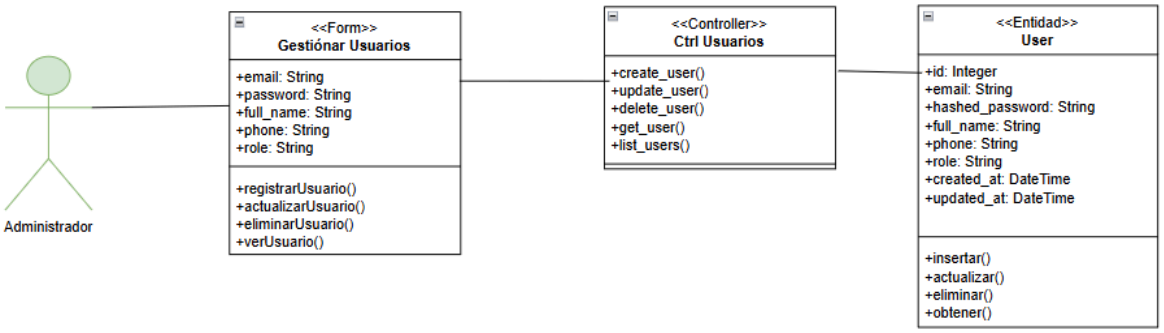
CU16 - Gestionar Lista de Talleres Candidatos



Ciclo #3

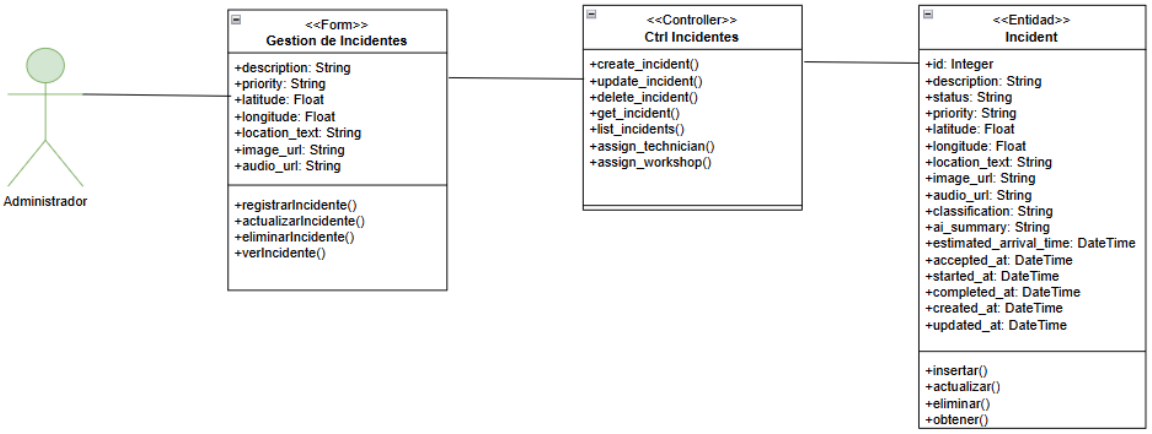
CU17. Gestionar Usuarios

CU17. Gestionar usuarios-



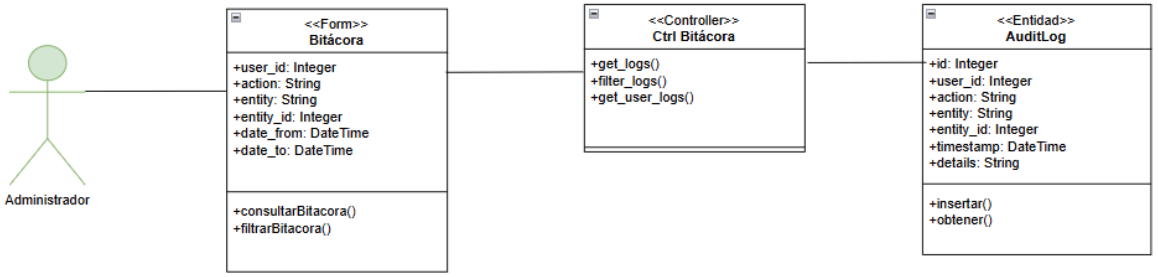
CU19. Gestionar Incidentes

CU19. Gestionar incidentes

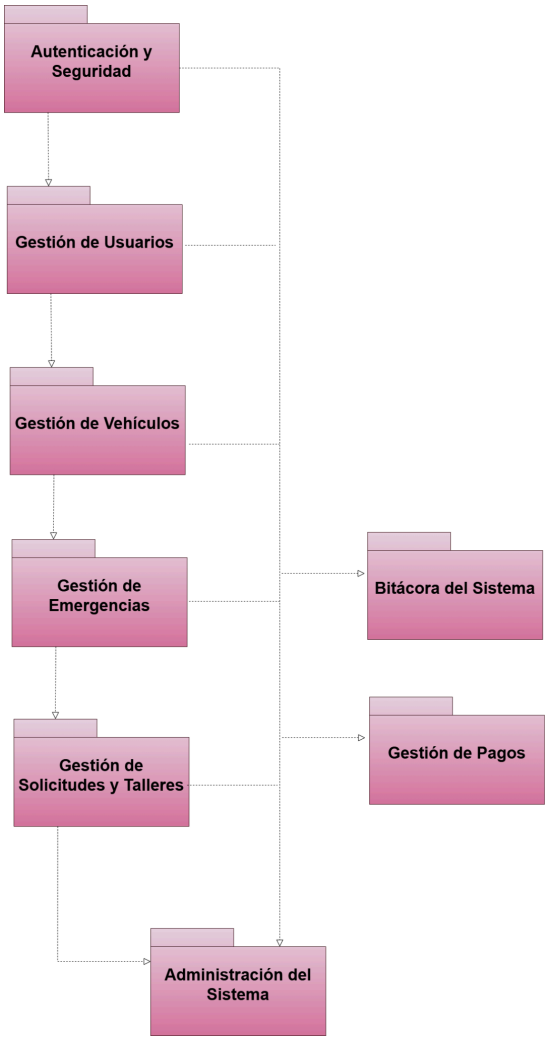


CU20. Bitácora del Sistema

CU20. Bitácora del sistema



3.1.4 Análisis de Paquetes

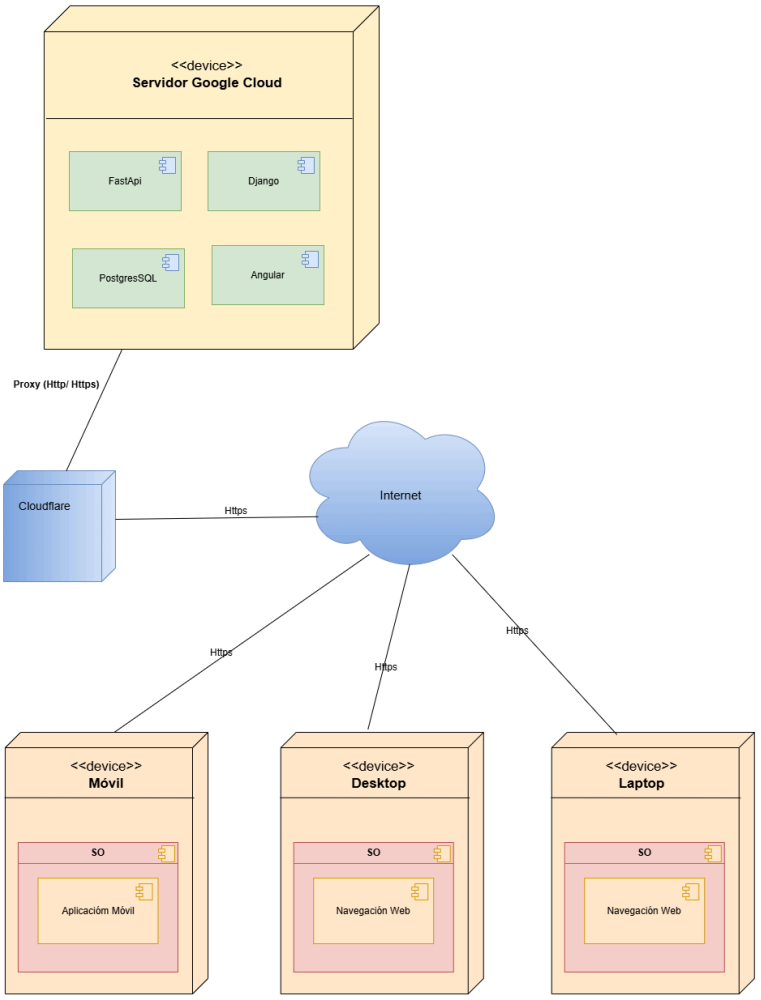


3.3) FLUJO DE TRABAJO: DISEÑO

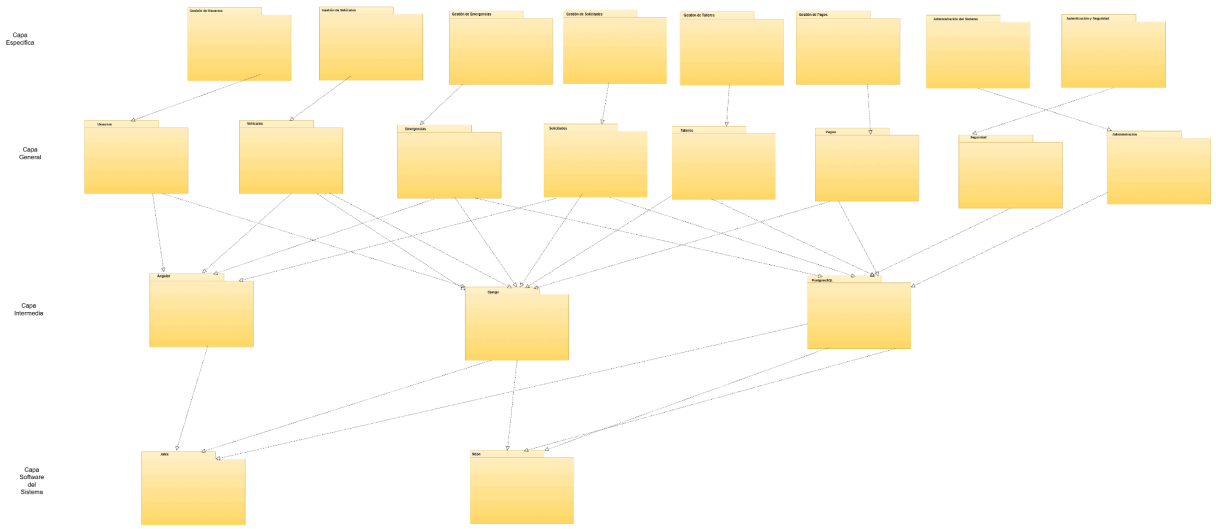
3.3.1 Diseño de Arquitectura

3.3.1.1. Diseño Físico (Diagrama de Despliegue)

deployment Diagrama de Despliegue



3.3.1.2 Diseño Lógico (Diagrama Organizado en Capas)

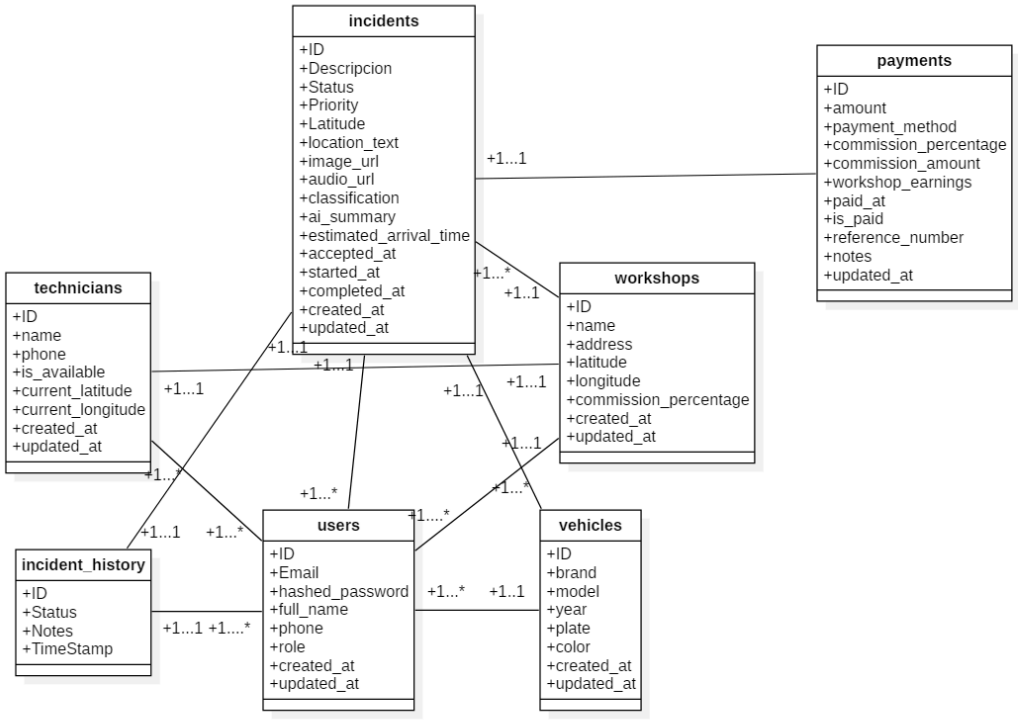


3.3.2. Diseño de Datos

3.3.2.1. Diseño de Datos Lógico

Diagrama de Clases

DIAGRAMA DE CLASE



Mapeo y Normalización

MAPEO

USERS

PK

ID	Email	hashed_password	full_name	phone	role	created_at	update_at

VEHICLES

PK FK

ID	brand	model	year	plate	color	created_at	update_at	user_ID

WORKSHOPS

PK FK

ID	name	address	latitude	longitude	commission_percentage	is_activate	created_at	update_at	owner_ID

TECHNICIANS

PK FK FK

ID	name	phone	is_available	current_latitude	current_longitude	created_at	updated_at	workshop_id	user_id

INCIDENTS

PK

ID	description	status	priority	latitude	longitude	location_text	image_url	audio_url	classification

FK FK FK FK

ai_summary	estimated_arrival_time	accepted_at	started_at	completed_at	created_at	updated_at	user_id	vehicle_id	workshop_id	technician_id

INCIDENT_HISTORY

PK FK FK

ID	Status	Notes	TimeStamp	incident_id	changed_by_user_id

PAYMENTS

PK

ID	amount	payment_method	commission_percentage	commission_amount	workshop_earnings	paid_at	is_paid

FK

reference_number	notes	created_at	updated_at	incident_id

3.3.2.2 Diseño de Datos Físico

TABLA DE VOLUMEN

USERS				
Atributo	TipoDato	Amplitud	Llave	Nulo
id	integer	-2,147,483,648 … 2,147,483,647	PK	NO
email	character varying	0 ... n caracteres		NO
hashed_password	character varying	0 ... n caracteres		NO
full_name	character varying	0 ... n caracteres		NO
phone	character varying	0 ... n caracteres		SI
role	userrole	Enum definido por el sistema		NO

created_at	timestamp without time zone	4713 a.C. … 294276 d.C. aprox.		SI
updated_at	timestamp without time zone	4713 a.C. … 294276 d.C. aprox.		SI

VEHICLES				
Atributo	TipoDato	Amplitud	Llave	Nulo
ID	integer	-2,147,483,648 … 2,147,483,647	PK	NO
user_id	integer	-2,147,483,648 … 2,147,483,647	FK	NO
Brand	character varying	0 ... n caracteres		NO
model	character varying	0 ... n caracteres		NO
year	integer	-2,147,483,648 … 2,147,483,647		NO
plate	character varying	0 ... n caracteres		NO
color	character varying	0 ... n caracteres		Sí
created_at	timestamp without time zone	4713 a.C. … 294276 d.C. aprox.		SI
updated_at	timestamp without time zone	4713 a.C. … 294276 d.C. aprox.		SI

WORKSHOPS				
Atributo	TipoDato	Amplitud	Llave	Nulo

id	integer	-2,147,483,648 … 2,147,483,647	PK	NO
owner_id	integer	-2,147,483,648 … 2,147,483,647	FK	NO
name	character varying	0 ... n caracteres		No
address	character varying	0 ... n caracteres		NO
latitude	double precision	±15 dígitos decimales aprox.		NO
longitude	double precision	±15 dígitos decimales aprox.		NO
commission_percentage	double precision	±15 dígitos decimales aprox.		NO
is_active	boolean	TRUE / FALSE		NO
created_at	timestamp without time zone	4713 a.C. … 294276 d.C. aprox.		SI
updated_at	timestamp without time zone	4713 a.C. … 294276 d.C. aprox.		SI

TECHNICIANS				
Atributo	TipoDato	Amplitud	Llave	Null
ID	INTEGER	-2,147,483,648 … 2,147,483,647	PK	NO
workshop_id	INTEGER	-2,147,483,648 … 2,147,483,647	FK	NO
user_id	INTEGER	-2,147,483,648 …	FK	SI

		2,147,483,647		
name	VARCHAR	0 ... n caracteres		NO
phone	VARCHAR	0 ... n caracteres		SI
is_available	BOOLEAN	TRUE / FALSE		NO
current_latitude	DOUBLE PRECISION	±15 dígitos decimales aprox.		SI
current_longitude	DOUBLE PRECISION	±15 dígitos decimales aprox.		SI
created_at	TIMESTAMP	4713 a.C. … 294276 d.C. aprox.		SI
updated_at	TIMESTAMP	4713 a.C. … 294276 d.C. aprox.		SI

INCIDENTS				
Atributos	TipoDato	Amplitud	Llave	Null
id	integer	-2,147,483,648 … 2,147,483,647	PK	No
user_id	integer	-2,147,483,648 … 2,147,483,647	FK	NO
vehicle_id	integer	-2,147,483,648 … 2,147,483,647	FK	NO
workshop_id	integer	-2,147,483,648 … 2,147,483,647	FK	SI
technician_id	integer	-2,147,483,648 … 2,147,483,647	FK	SI

description	TEXT	Longitud variable		NO
status	INCIDENTSTATUS	Enum del sistema		NO
priority	INCIDENTSTATUS	Enum del sistema		NO
latitude	DOUBLE PRECISION	±15 dígitos decimales aprox.		NO
longitude	DOUBLE PRECISION	±15 dígitos decimales aprox.		SI
location_text	VARCHAR	0 ... n caracteres		SI
image_url	VARCHAR	0 ... n caracteres		SI
audio_url	VARCHAR	0 ... n caracteres		SI
classification	VARCHAR	0 ... n caracteres		SI
ai_summary	TEXT	Longitud variable		SI
estimated_arrival_time	INTEGER	-2,147,483,648 … 2,147,483,647		SI
accepted_at	TIMESTAMP	4713 a.C. … 294276 d.C. aprox.		SI
started_at	TIMESTAMP	4713 a.C. … 294276 d.C. aprox.		SI
completed_at	TIMESTAMP	4713 a.C. … 294276 d.C. aprox.		SI
created_at	TIMESTAMP	4713 a.C. … 294276 d.C. aprox.		SI
updated_at	TIMESTAMP	4713 a.C. … 294276 d.C. aprox.		SI

INCIDENT_HISTORY				
Atributo	TipoDato	Amplitud	Llave	Null
id	integer	-2,147,483,648 … 2,147,483,647	PK	No
incident_id	integer	-2,147,483,648 … 2,147,483,647	Fk	NO
status	incidentstatus	Enum definido por el sistema		NO
changed_by_user_id	integer	-2,147,483,648 … 2,147,483,647	FK	NO
notes	text	Longitud variable		SI
timestamp	timestamp without time zone	4713 a.C. … 294276 d.C. aprox.		SI

PAYMENTS				
Atributos	TipeDato	Amplitud	Llave	Null
ID	INTEGER	-2,147,483,648 … 2,147,483,647	PK	NO
incident_id	INTEGER	-2,147,483,648 … 2,147,483,647	FK	NO
amount	NUMERIC(10,2)	-99,999,999.99 ... 99,999,999.99		NO
payment_method	PAYMENTMETHOD	Enum del sistema		NO
commission_percentage	DOUBLE PRECISION	±15 dígitos decimales aprox.		NO

commission_amount	NUMERIC(10,2)	-99,999,999.99 ... 99,999,999.99		NO
workshop_earnings	NUMERIC(10,2)	-99,999,999.99 ... 99,999,999.99		NO
paid_at	TIMESTAMP	4713 a.C. … 294276 d.C. aprox.		SI
is_paid	BOOLEAN	TRUE / FALSE		NO
reference_number	VARCHAR	0 ... n caracteres		SI
notes	TEXT	Longitud variable		SI
created_at	TIMESTAMP	4713 a.C. … 294276 d.C. aprox.		SI
updated_at	TIMESTAMP	4713 a.C. … 294276 d.C. aprox.		SI

SCRIPT

```
-- ========================================
-- ELIMINAR TIPOS Y TABLAS SI EXISTEN
-- ========================================

DROP TABLE IF EXISTS payments CASCADE;
DROP TABLE IF EXISTS incident_history CASCADE;
DROP TABLE IF EXISTS incidents CASCADE;
DROP TABLE IF EXISTS technicians CASCADE;
DROP TABLE IF EXISTS workshops CASCADE;
DROP TABLE IF EXISTS vehicles CASCADE;
DROP TABLE IF EXISTS users CASCADE;

DROP TYPE IF EXISTS paymentmethod CASCADE;
DROP TYPE IF EXISTS incidentpriority CASCADE;
DROP TYPE IF EXISTS incidentstatus CASCADE;
DROP TYPE IF EXISTS userrole CASCADE;

-- ========================================
-- ENUMS
-- Ajusta estos valores si en tu sistema usas otros
-- ========================================
```

```
CREATE TYPE userrole AS ENUM (
    'client',
    'workshop_owner',
    'technician',
    'admin'
);

CREATE TYPE incidentstatus AS ENUM (
    'pending',
    'assigned',
    'accepted',
    'in_progress',
    'completed',
    'cancelled'
);

CREATE TYPE incidentpriority AS ENUM (
    'low',
    'medium',
    'high',
    'critical'
);

CREATE TYPE paymentmethod AS ENUM (
    'cash',
    'qr',
    'transfer',
    'card'
);

-- ========================================
-- TABLA: users
-- ========================================

CREATE TABLE users (
    id SERIAL PRIMARY KEY,
    email VARCHAR NOT NULL,
    hashed_password VARCHAR NOT NULL,
    full_name VARCHAR NOT NULL,
    phone VARCHAR NULL,
    role userrole NOT NULL,
    created_at TIMESTAMP WITHOUT TIME ZONE NULL,
    updated_at TIMESTAMP WITHOUT TIME ZONE NULL
);

-- Opcional pero recomendable
-- ALTER TABLE users ADD CONSTRAINT users_email_key UNIQUE (email);
```

```
-- ========================================
-- TABLA: vehicles
-- ========================================

CREATE TABLE vehicles (
    id SERIAL PRIMARY KEY,
    user_id INTEGER NOT NULL,
    brand VARCHAR NOT NULL,
    model VARCHAR NOT NULL,
    year INTEGER NOT NULL,
    plate VARCHAR NOT NULL,
    color VARCHAR NULL,
    created_at TIMESTAMP WITHOUT TIME ZONE NULL,
    updated_at TIMESTAMP WITHOUT TIME ZONE NULL,
    CONSTRAINT vehicles_user_id_fkey
        FOREIGN KEY (user_id) REFERENCES users(id)
);

-- Opcional pero recomendable
-- ALTER TABLE vehicles ADD CONSTRAINT vehicles_plate_key UNIQUE (plate);

-- ========================================
-- TABLA: workshops
-- ========================================

CREATE TABLE workshops (
    id SERIAL PRIMARY KEY,
    owner_id INTEGER NOT NULL,
    name VARCHAR NOT NULL,
    address VARCHAR NOT NULL,
    latitude DOUBLE PRECISION NOT NULL,
    longitude DOUBLE PRECISION NOT NULL,
    commission_percentage DOUBLE PRECISION NOT NULL,
    is_active BOOLEAN NOT NULL,
    created_at TIMESTAMP WITHOUT TIME ZONE NULL,
    updated_at TIMESTAMP WITHOUT TIME ZONE NULL,
    CONSTRAINT workshops_owner_id_fkey
        FOREIGN KEY (owner_id) REFERENCES users(id)
);

-- ========================================
-- TABLA: technicians
-- ========================================

CREATE TABLE technicians (
    id SERIAL PRIMARY KEY,
    workshop_id INTEGER NOT NULL,
    user_id INTEGER NULL,
```

```
    name VARCHAR NOT NULL,
    phone VARCHAR NULL,
    is_available BOOLEAN NOT NULL,
    current_latitude DOUBLE PRECISION NULL,
    current_longitude DOUBLE PRECISION NULL,
    created_at TIMESTAMP WITHOUT TIME ZONE NULL,
    updated_at TIMESTAMP WITHOUT TIME ZONE NULL,
    CONSTRAINT technicians_workshop_id_fkey
        FOREIGN KEY (workshop_id) REFERENCES workshops(id),
    CONSTRAINT technicians_user_id_fkey
        FOREIGN KEY (user_id) REFERENCES users(id)
);

-- ========================================
-- TABLA: incidents
-- ========================================

CREATE TABLE incidents (
    id SERIAL PRIMARY KEY,
    user_id INTEGER NOT NULL,
    vehicle_id INTEGER NOT NULL,
    workshop_id INTEGER NULL,
    technician_id INTEGER NULL,
    description TEXT NOT NULL,
    status incidentstatus NOT NULL,
    priority incidentpriority NOT NULL,
    latitude DOUBLE PRECISION NULL,
    longitude DOUBLE PRECISION NULL,
    location_text VARCHAR NULL,
    image_url VARCHAR NULL,
    audio_url VARCHAR NULL,
    classification VARCHAR NULL,
    ai_summary TEXT NULL,
    estimated_arrival_time INTEGER NULL,
    accepted_at TIMESTAMP WITHOUT TIME ZONE NULL,
    started_at TIMESTAMP WITHOUT TIME ZONE NULL,
    completed_at TIMESTAMP WITHOUT TIME ZONE NULL,
    created_at TIMESTAMP WITHOUT TIME ZONE NULL,
    updated_at TIMESTAMP WITHOUT TIME ZONE NULL,
    CONSTRAINT incidents_user_id_fkey
        FOREIGN KEY (user_id) REFERENCES users(id),
    CONSTRAINT incidents_vehicle_id_fkey
        FOREIGN KEY (vehicle_id) REFERENCES vehicles(id),
    CONSTRAINT incidents_workshop_id_fkey
        FOREIGN KEY (workshop_id) REFERENCES workshops(id),
    CONSTRAINT incidents_technician_id_fkey
        FOREIGN KEY (technician_id) REFERENCES technicians(id)
);
```

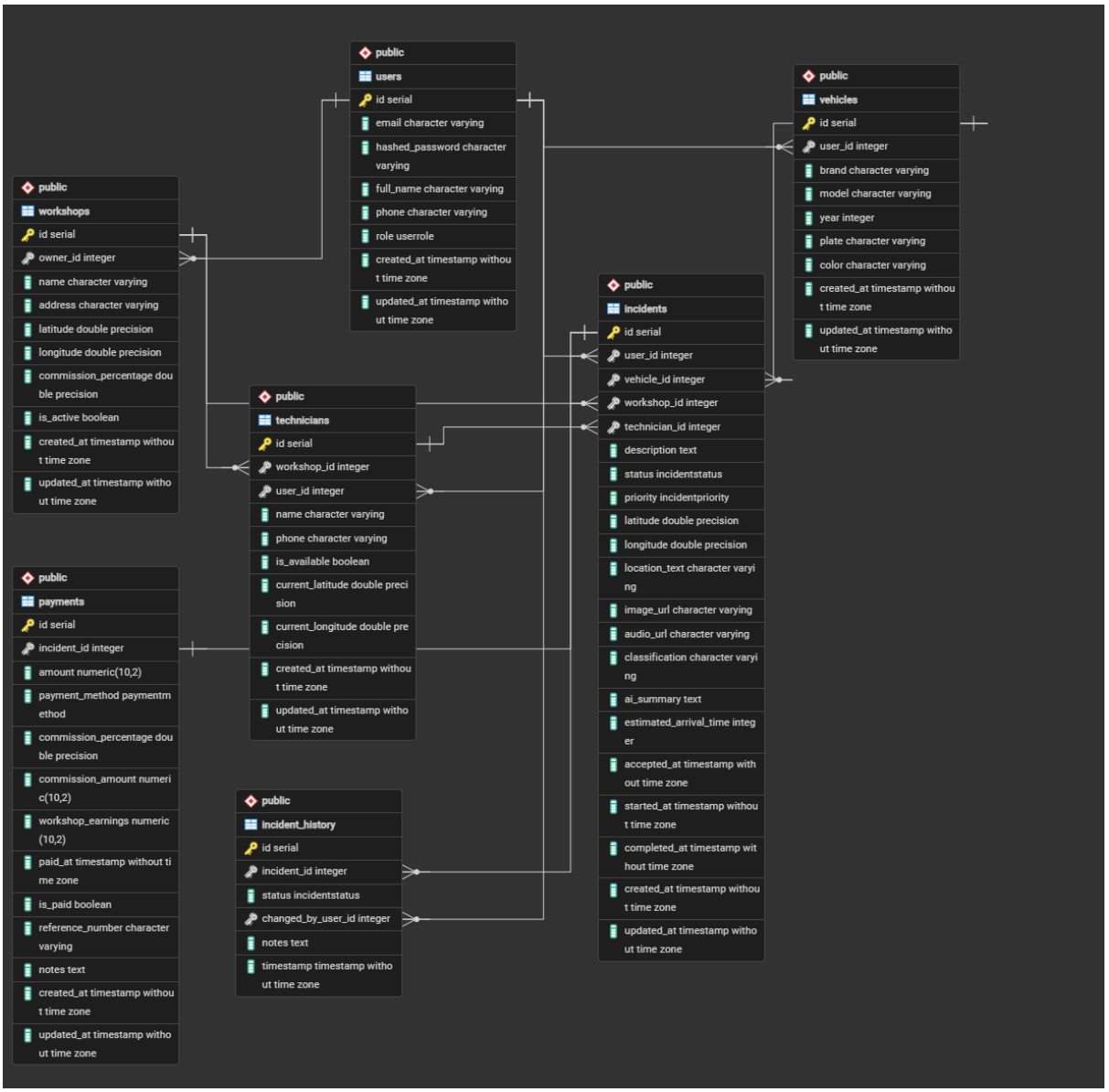
```
-- ========================================
-- TABLA: incident_history
-- ========================================

CREATE TABLE incident_history (
    id SERIAL PRIMARY KEY,
    incident_id INTEGER NOT NULL,
    status incidentstatus NOT NULL,
    changed_by_user_id INTEGER NOT NULL,
    notes TEXT NULL,
    "timestamp" TIMESTAMP WITHOUT TIME ZONE NULL,
    CONSTRAINT incident_history_incident_id_fkey
        FOREIGN KEY (incident_id) REFERENCES incidents(id),
    CONSTRAINT incident_history_changed_by_user_id_fkey
        FOREIGN KEY (changed_by_user_id) REFERENCES users(id)
);

-- ========================================
-- TABLA: payments
-- ========================================

CREATE TABLE payments (
    id SERIAL PRIMARY KEY,
    incident_id INTEGER NOT NULL,
    amount NUMERIC(10,2) NOT NULL,
    payment_method paymentmethod NOT NULL,
    commission_percentage DOUBLE PRECISION NOT NULL,
    commission_amount NUMERIC(10,2) NOT NULL,
    workshop_earnings NUMERIC(10,2) NOT NULL,
    paid_at TIMESTAMP WITHOUT TIME ZONE NULL,
    is_paid BOOLEAN NOT NULL,
    reference_number VARCHAR NULL,
    notes TEXT NULL,
    created_at TIMESTAMP WITHOUT TIME ZONE NULL,
    updated_at TIMESTAMP WITHOUT TIME ZONE NULL,
    CONSTRAINT payments_incident_id_fkey
        FOREIGN KEY (incident_id) REFERENCES incidents(id),
    CONSTRAINT payments_incident_id_key UNIQUE (incident_id)
);
```

DIAGRAMA RELACIONAL



ACTUALIZACIÓN DE TUPLAS

```
-- ========================================
-- INSERTS EN users
-- ========================================
INSERT INTO users (email, hashed_password, full_name, phone, role, created_at, updated_at)
VALUES
('admin@auxiliovehicular.com', 'hash_admin_123', 'Carlos Mendoza', '70000001', 'admin', '2026-01-10 08:00:00', '2026-01-10 08:00:00'),
('dueno1@taller.com', 'hash_owner_123', 'Juan Pérez', '70000002', 'workshop_owner', '2026-01-11 09:00:00', '2026-01-11 09:00:00'),
('dueno2@taller.com', 'hash_owner_456', 'María López', '70000003', 'workshop_owner', '2026-01-11 09:30:00', '2026-01-11 09:30:00'),
('tecnico1@taller.com', 'hash_tec_123', 'Luis Herrera', '70000004', 'technician', '2026-01-12 07:00:00', '2026-01-12 07:00:00'),
```

('tecnico2@taller.com', 'hash_tec_456', 'Pedro Rojas', '70000005', 'technician', '2026-01-12 07:15:00', '2026-01-12 07:15:00'),
('cliente1@gmail.com', 'hash_cli_123', 'Ana Fernández', '70000006', 'client', '2026-01-13 10:00:00', '2026-01-13 10:00:00'),
('cliente2@gmail.com', 'hash_cli_456', 'Diego Suárez', '70000007', '2026-01-13 10:30:00', '2026-01-13 10:30:00'),
('cliente3@gmail.com', 'hash_cli_789', 'Valeria Castro', NULL, 'client', '2026-01-14 11:00:00', '2026-01-14 11:00:00');

-- ==
-- INSERTS EN vehicles
-- user_id debe apuntar a usuarios con rol client
-- ==
INSERT INTO vehicles (user_id, brand, model, year, plate, color, created_at, updated_at)
VALUES
(6, 'Toyota', 'Corolla', 2018, '1234ABC', 'Blanco', '2026-01-15 09:00:00', '2026-01-15 09:00:00'),
(6, 'Suzuki', 'Vitara', 2020, '2345BCD', 'Gris', '2026-01-15 09:10:00', '2026-01-15 09:10:00'),
(7, 'Nissan', 'Sentra', 2017, '3456CDE', 'Negro', '2026-01-15 09:20:00', '2026-01-15 09:20:00'),
(8, 'Hyundai', 'Accent', 2019, '4567DEF', NULL, '2026-01-15 09:30:00', '2026-01-15 09:30:00');

-- ==
-- INSERTS EN workshops
-- owner_id apunta a usuarios con rol workshop_owner
-- ==
INSERT INTO workshops (owner_id, name, address, latitude, longitude, commission_percentage, is_active, created_at, updated_at)
VALUES
(2, 'Taller Express Norte', 'Av. Banzer 4to anillo', -17.750100, -63.176500, 10.0, TRUE, '2026-01-16 08:00:00', '2026-01-16 08:00:00'),
(3, 'Auxilio Mecánico Sur', 'Av. Santos Dumont 6to anillo', -17.830400, -63.210300, 12.5, TRUE, '2026-01-16 08:20:00', '2026-01-16 08:20:00');

-- ==
-- INSERTS EN technicians
-- workshop_id apunta a workshops
-- user_id apunta a users con rol technician
-- ==
INSERT INTO technicians (workshop_id, user_id, name, phone, is_available, current_latitude, current_longitude, created_at, updated_at)
VALUES
(1, 4, 'Luis Herrera', '70000004', TRUE, -17.760000, -63.170000, '2026-01-17 07:00:00', '2026-01-17 07:00:00'),
(2, 5, 'Pedro Rojas', '70000005', FALSE, -17.825000, -63.205000, '2026-01-17 07:10:00', '2026-01-17 07:10:00'),

```
(1, NULL, 'Carlos Vargas', '70000008', TRUE, NULL, NULL, '2026-01-17 07:20:00',
'2026-01-17 07:20:00');

-- ========================================
-- INSERTS EN incidents
-- user_id apunta a cliente
-- vehicle_id apunta a vehicles
-- workshop_id y technician_id pueden ser NULL
-- ========================================
INSERT INTO incidents (
    user_id, vehicle_id, workshop_id, technician_id,
    description, status, priority,
    latitude, longitude, location_text,
    image_url, audio_url, classification, ai_summary,
    estimated_arrival_time, accepted_at, started_at, completed_at,
    created_at, updated_at
)
VALUES
(6, 1, 1, 1,
 'El vehículo no enciende en el estacionamiento del centro comercial.',
 'assigned', 'medium',
 -17.783300, -63.182200, 'Centro comercial, ingreso principal',
 'https://ejemplo.com/img/incidente1.jpg', 'https://ejemplo.com/audio/incidente1.mp3',
 'battery', 'Posible problema de batería según descripción e imagen.',
 20, '2026-01-18 09:10:00', NULL, NULL,
 '2026-01-18 09:00:00', '2026-01-18 09:10:00'),

(7, 3, 2, 2,
 'Pinchazo de llanta en carretera al norte.',
 'in_progress', 'high',
 -17.690000, -63.100000, 'Carretera al norte, km 12',
 'https://ejemplo.com/img/incidente2.jpg', NULL,
 'tire', 'Se detecta posible pinchazo en rueda delantera.',
 15, '2026-01-18 10:05:00', '2026-01-18 10:20:00', NULL,
 '2026-01-18 10:00:00', '2026-01-18 10:20:00'),

(8, 4, NULL, NULL,
 'Choque leve con daños en el parachoques delantero.',
 'pending', 'high',
 -17.810000, -63.190000, 'Av. Alemania y 2do anillo',
 NULL, 'https://ejemplo.com/audio/incidente3.mp3',
 'collision', 'Incidente pendiente de asignación.',
 NULL, NULL, NULL, NULL,
 '2026-01-18 11:00:00', '2026-01-18 11:00:00'),

(6, 2, 1, 3,
 'Sobrecalentamiento del motor después de 30 minutos de viaje.',
 'completed', 'critical',
```

```
 -17.770000, -63.175000, 'Av. Beni, cerca del 5to anillo',
 'https://ejemplo.com/img/incidente4.jpg', 'https://ejemplo.com/audio/incidente4.mp3',
 'engine', 'Posible falla de refrigeración o radiador.',
 10, '2026-01-18 08:15:00', '2026-01-18 08:25:00', '2026-01-18 09:05:00',
 '2026-01-18 08:00:00', '2026-01-18 09:05:00');

-- ========================================
-- INSERTS EN incident_history
-- ========================================
INSERT INTO incident_history (incident_id, status, changed_by_user_id, notes,
"timestamp")
VALUES
(1, 'pending', 6, 'Cliente registró la emergencia.', '2026-01-18 09:00:00'),
(1, 'assigned', 2, 'Taller Express Norte aceptó la solicitud.', '2026-01-18 09:10:00'),

(2, 'pending', 7, 'Cliente reportó pinchazo de llanta.', '2026-01-18 10:00:00'),
(2, 'accepted', 3, 'Taller asignado y técnico confirmado.', '2026-01-18 10:05:00'),
(2, 'in_progress', 5, 'Técnico en camino al punto de asistencia.', '2026-01-18 10:20:00'),

(3, 'pending', 8, 'Incidente registrado y en espera de taller.', '2026-01-18 11:00:00'),

(4, 'pending', 6, 'Cliente reportó sobrecalentamiento.', '2026-01-18 08:00:00'),
(4, 'accepted', 2, 'Taller confirmó atención inmediata.', '2026-01-18 08:15:00'),
(4, 'in_progress', 4, 'Técnico revisando sistema de refrigeración.', '2026-01-18 08:25:00'),
(4, 'completed', 4, 'Incidente resuelto, vehículo operativo.', '2026-01-18 09:05:00');

-- ========================================
-- INSERTS EN payments
-- incident_id es UNIQUE, por lo que solo puede haber uno por incidente
-- ========================================
INSERT INTO payments (
    incident_id, amount, payment_method,
    commission_percentage, commission_amount, workshop_earnings,
    paid_at, is_paid, reference_number, notes,
    created_at, updated_at
)
VALUES
(1, 120.00, 'qr', 10.0, 12.00, 108.00,
 '2026-01-18 09:45:00', TRUE, 'QR-0001', 'Pago realizado por QR al finalizar el servicio.',
 '2026-01-18 09:40:00', '2026-01-18 09:45:00'),

(2, 180.00, 'transfer', 12.5, 22.50, 157.50,
 NULL, FALSE, 'TR-0002', 'Pago pendiente de confirmación bancaria.',
 '2026-01-18 10:30:00', '2026-01-18 10:30:00'),

(4, 250.00, 'cash', 10.0, 25.00, 225.00,
 '2026-01-18 09:10:00', TRUE, NULL, 'Pago en efectivo contra entrega del servicio.',
 '2026-01-18 09:05:00', '2026-01-18 09:10:00');
```

CONSULTAS

1. Listar todos los usuarios

```
SELECT *
FROM users;
```

2. Listar usuarios clientes

```
SELECT id, full_name, email, phone
FROM users
WHERE role = 'client';
```

3. Listar vehículos con el nombre del propietario

```
SELECT
    v.id,
    u.full_name AS propietario,
    v.brand,
    v.model,
    v.year,
    v.plate,
    v.color
FROM vehicles v
JOIN users u ON v.user_id = u.id;
```

4. Listar talleres con su dueño

```
SELECT
    w.id,
    w.name AS taller,
    u.full_name AS propietario,
    w.address,
    w.commission_percentage,
    w.is_active
FROM workshops w
JOIN users u ON w.owner_id = u.id;
```

5. Listar técnicos con su taller

```
SELECT
    t.id,
    t.name AS tecnico,
    w.name AS taller,
    t.phone,
    t.is_available
FROM technicians t
JOIN workshops w ON t.workshop_id = w.id;
```

6. Mostrar solo técnicos disponibles

```
SELECT
    id,
    name,
    phone,
```

```
    current_latitude,
    current_longitude
FROM technicians
WHERE is_available = TRUE;
7. Listar todos los incidentes con cliente, vehículo y taller
SELECT
    i.id,
    u.full_name AS cliente,
    v.brand || ' ' || v.model AS vehiculo,
    w.name AS taller,
    i.status,
    i.priority,
    i.description,
    i.created_at
FROM incidents i
JOIN users u ON i.user_id = u.id
JOIN vehicles v ON i.vehicle_id = v.id
LEFT JOIN workshops w ON i.workshop_id = w.id;
8. Mostrar incidentes pendientes
SELECT
    id,
    description,
    priority,
    location_text,
    created_at
FROM incidents
WHERE status = 'pending';
9. Mostrar incidentes completados
SELECT
    id,
    description,
    priority,
    completed_at
FROM incidents
WHERE status = 'completed';

10. Ver historial de un incidente específico
SELECT
    ih.id,
    ih.incident_id,
    ih.status,
    u.full_name AS cambiado_por,
    ih.notes,
    ih."timestamp"
FROM incident_history ih
JOIN users u ON ih.changed_by_user_id = u.id
WHERE ih.incident_id = 1
ORDER BY ih."timestamp";
```

11. Contar cuántos incidentes tiene cada cliente

```
SELECT
    u.full_name AS cliente,
    COUNT(i.id) AS cantidad_incidentes
FROM users u
LEFT JOIN incidents i ON u.id = i.user_id
WHERE u.role = 'client'
GROUP BY u.full_name
ORDER BY cantidad_incidentes DESC;
```

12. Contar cuántos incidentes atendió cada taller

```
SELECT
    w.name AS taller,
    COUNT(i.id) AS total_incidentes
FROM workshops w
LEFT JOIN incidents i ON w.id = i.workshop_id
GROUP BY w.name
ORDER BY total_incidentes DESC;
```

13. Contar cuántos incidentes atendió cada técnico

```
SELECT
    t.name AS tecnico,
    COUNT(i.id) AS total_atendidos
FROM technicians t
LEFT JOIN incidents i ON t.id = i.technician_id
GROUP BY t.name
ORDER BY total_atendidos DESC;
```

14. Ver pagos realizados

```
SELECT
    p.id,
    p.incident_id,
    p.amount,
    p.payment_method,
    p.commission_amount,
    p.workshop_earnings,
    p.paid_at
FROM payments p
WHERE p.is_paid = TRUE;
```

PROCEDIMIENTOS ALMACENADOS

1) Procedimiento para registrar un vehículo

```
CREATE OR REPLACE PROCEDURE sp_registrar_vehiculo(
    p_user_id INTEGER,
    p_brand VARCHAR,
    p_model VARCHAR,
    p_year INTEGER,
    p_plate VARCHAR,
    p_color VARCHAR
```

```
)
LANGUAGE plpgsql
AS $$
BEGIN
    INSERT INTO vehicles (
        user_id, brand, model, year, plate, color, created_at, updated_at
    )
    VALUES (
        p_user_id, p_brand, p_model, p_year, p_plate, p_color, NOW(), NOW()
    );
END;
$$;
2) Procedimiento para registrar un taller
CREATE OR REPLACE PROCEDURE sp_registrar_taller(
    p_owner_id INTEGER,
    p_name VARCHAR,
    p_address VARCHAR,
    p_latitude DOUBLE PRECISION,
    p_longitude DOUBLE PRECISION,
    p_commission_percentage DOUBLE PRECISION
)
LANGUAGE plpgsql
AS $$
BEGIN
    INSERT INTO workshops (
        owner_id, name, address, latitude, longitude,
        commission_percentage, is_active, created_at, updated_at
    )
    VALUES (
        p_owner_id, p_name, p_address, p_latitude, p_longitude,
        p_commission_percentage, TRUE, NOW(), NOW()
    );
END;
$$;
3) Procedimiento para registrar un técnico
CREATE OR REPLACE PROCEDURE sp_registrar_tecnico(
    p_workshop_id INTEGER,
    p_user_id INTEGER,
    p_name VARCHAR,
    p_phone VARCHAR
)
LANGUAGE plpgsql
AS $$
BEGIN
    INSERT INTO technicians (
        workshop_id, user_id, name, phone,
        is_available, created_at, updated_at
    )
```

```
    VALUES (
        p_workshop_id, p_user_id, p_name, p_phone,
        TRUE, NOW(), NOW()
    );
END;
$$;
4) Procedimiento para registrar un incidente
CREATE OR REPLACE PROCEDURE sp_registrar_incidente(
    p_user_id INTEGER,
    p_vehicle_id INTEGER,
    p_description TEXT,
    p_priority incidentpriority,
    p_latitude DOUBLE PRECISION,
    p_longitude DOUBLE PRECISION,
    p_location_text VARCHAR,
    p_image_url VARCHAR,
    p_audio_url VARCHAR
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_incident_id INTEGER;
BEGIN
    INSERT INTO incidents (
        user_id, vehicle_id, workshop_id, technician_id,
        description, status, priority,
        latitude, longitude, location_text,
        image_url, audio_url, classification, ai_summary,
        estimated_arrival_time, accepted_at, started_at, completed_at,
        created_at, updated_at
    )
    VALUES (
        p_user_id, p_vehicle_id, NULL, NULL,
        p_description, 'pending', p_priority,
        p_latitude, p_longitude, p_location_text,
        p_image_url, p_audio_url, NULL, NULL,
        NULL, NULL, NULL, NULL,
        NOW(), NOW()
    )
    RETURNING id INTO v_incident_id;

    INSERT INTO incident_history (
        incident_id, status, changed_by_user_id, notes, "timestamp"
    )
    VALUES (
        v_incident_id, 'pending', p_user_id,
        'Incidente registrado por el cliente', NOW()
    );
```

```
END;
$$;
5) Procedimiento para asignar taller y técnico a un incidente
CREATE OR REPLACE PROCEDURE sp_asignar_incidente(
    p_incident_id INTEGER,
    p_workshop_id INTEGER,
    p_technician_id INTEGER,
    p_user_id INTEGER
)
LANGUAGE plpgsql
AS $$
BEGIN
    UPDATE incidents
    SET
        workshop_id = p_workshop_id,
        technician_id = p_technician_id,
        status = 'assigned',
        accepted_at = NOW(),
        updated_at = NOW()
    WHERE id = p_incident_id;

    INSERT INTO incident_history (
        incident_id, status, changed_by_user_id, notes, "timestamp"
    )
    VALUES (
        p_incident_id,
        'assigned',
        p_user_id,
        'Incidente asignado a taller y técnico',
        NOW()
    );
END;
$$;
6) Procedimiento para actualizar el estado de un incidente
CREATE OR REPLACE PROCEDURE sp_actualizar_estado_incidente(
    p_incident_id INTEGER,
    p_new_status incidentstatus,
    p_user_id INTEGER,
    p_notes TEXT
)
LANGUAGE plpgsql
AS $$
BEGIN
    UPDATE incidents
    SET
        status = p_new_status,
        started_at = CASE WHEN p_new_status = 'in_progress' THEN NOW() ELSE started_at
END,
```

```
        completed_at = CASE WHEN p_new_status = 'completed' THEN NOW() ELSE
completed_at END,
        updated_at = NOW()
    WHERE id = p_incident_id;

    INSERT INTO incident_history (
        incident_id, status, changed_by_user_id, notes, "timestamp"
    )
    VALUES (
        p_incident_id,
        p_new_status,
        p_user_id,
        p_notes,
        NOW()
    );
END;
$$;
7) Procedimiento para registrar pago
CREATE OR REPLACE PROCEDURE sp_registrar_pago(
    p_incident_id INTEGER,
    p_amount NUMERIC(10,2),
    p_payment_method paymentmethod,
    p_reference_number VARCHAR,
    p_notes TEXT
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_commission_percentage DOUBLE PRECISION;
    v_commission_amount NUMERIC(10,2);
    v_workshop_earnings NUMERIC(10,2);
BEGIN
    SELECT w.commission_percentage
    INTO v_commission_percentage
    FROM incidents i
    JOIN workshops w ON i.workshop_id = w.id
    WHERE i.id = p_incident_id;

    v_commission_amount := p_amount * (v_commission_percentage / 100.0);
    v_workshop_earnings := p_amount - v_commission_amount;

    INSERT INTO payments (
        incident_id, amount, payment_method,
        commission_percentage, commission_amount, workshop_earnings,
        paid_at, is_paid, reference_number, notes,
        created_at, updated_at
    )
    VALUES (
```

```
        p_incident_id, p_amount, p_payment_method,
        v_commission_percentage, v_commission_amount, v_workshop_earnings,
        NOW(), TRUE, p_reference_number, p_notes,
        NOW(), NOW()
    );
END;
$$;
8) Procedimiento para cambiar disponibilidad de técnico
CREATE OR REPLACE PROCEDURE sp_actualizar_disponibilidad_tecnico(
    p_technician_id INTEGER,
    p_is_available BOOLEAN
)
LANGUAGE plpgsql
AS $$
BEGIN
    UPDATE technicians
    SET
        is_available = p_is_available,
        updated_at = NOW()
    WHERE id = p_technician_id;
END;
$$;
9) Procedimiento para actualizar ubicación del técnico
CREATE OR REPLACE PROCEDURE sp_actualizar_ubicacion_tecnico(
    p_technician_id INTEGER,
    p_latitude DOUBLE PRECISION,
    p_longitude DOUBLE PRECISION
)
LANGUAGE plpgsql
AS $$
BEGIN
    UPDATE technicians
    SET
        current_latitude = p_latitude,
        current_longitude = p_longitude,
        updated_at = NOW()
    WHERE id = p_technician_id;
END;
$$;
10) Procedimiento para desactivar taller
CREATE OR REPLACE PROCEDURE sp_desactivar_taller(
    p_workshop_id INTEGER
)
LANGUAGE plpgsql
AS $$
BEGIN
    UPDATE workshops
    SET
```

```
    is_active = FALSE,
    updated_at = NOW()
  WHERE id = p_workshop_id;
END;
$$;
```

TRIGGERS

1) Trigger para actualizar `updated_at`

```
CREATE OR REPLACE FUNCTION fn_actualizar_updated_at()
RETURNS TRIGGER AS $$
BEGIN
    NEW.updated_at = NOW();
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
CREATE TRIGGER trg_users_updated_at
BEFORE UPDATE ON users
FOR EACH ROW
EXECUTE FUNCTION fn_actualizar_updated_at();

CREATE TRIGGER trg_vehicles_updated_at
BEFORE UPDATE ON vehicles
FOR EACH ROW
EXECUTE FUNCTION fn_actualizar_updated_at();

CREATE TRIGGER trg_workshops_updated_at
BEFORE UPDATE ON workshops
FOR EACH ROW
EXECUTE FUNCTION fn_actualizar_updated_at();

CREATE TRIGGER trg_technicians_updated_at
BEFORE UPDATE ON technicians
FOR EACH ROW
EXECUTE FUNCTION fn_actualizar_updated_at();

CREATE TRIGGER trg_incidents_updated_at
BEFORE UPDATE ON incidents
FOR EACH ROW
EXECUTE FUNCTION fn_actualizar_updated_at();
```

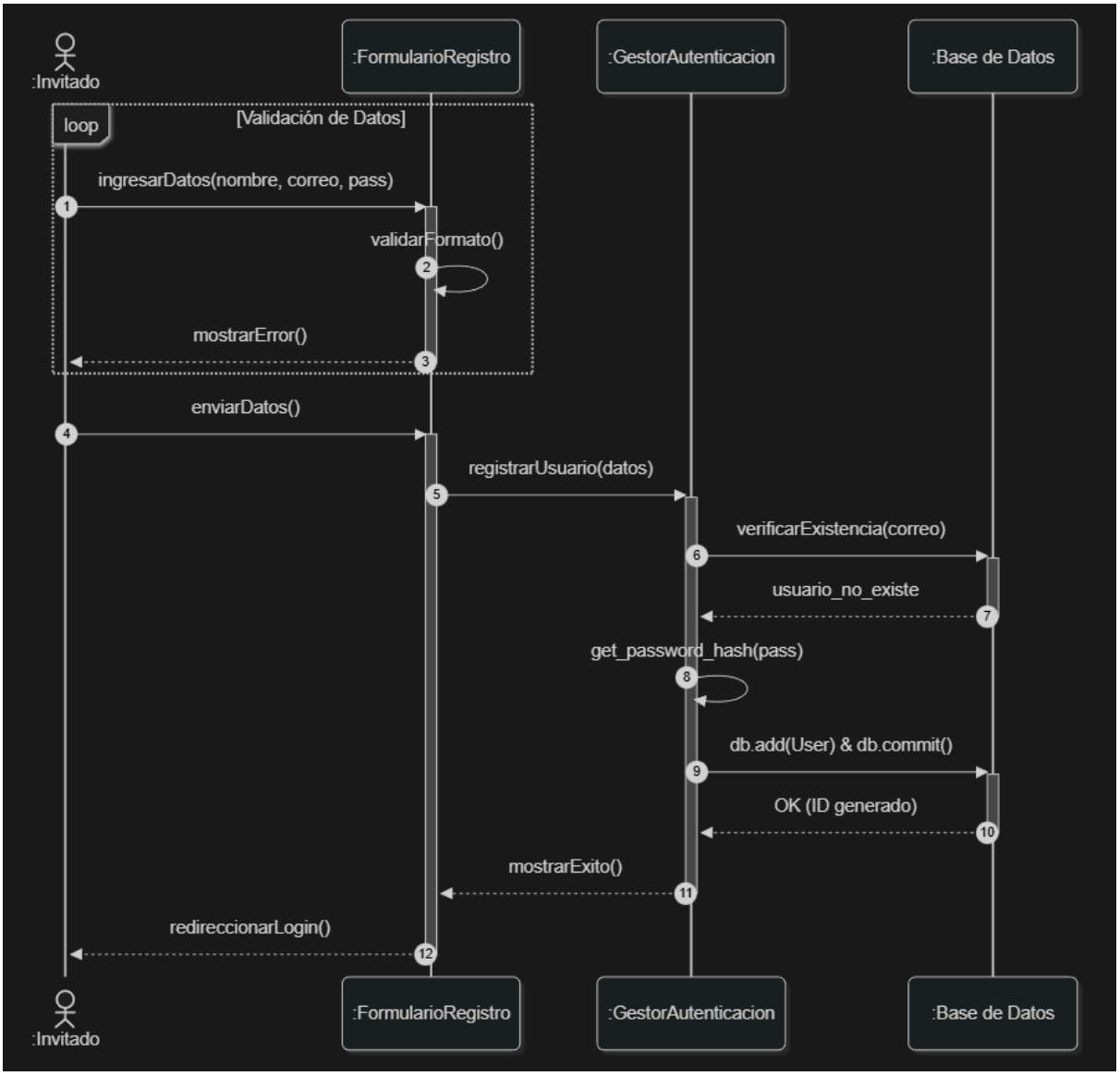
```
CREATE TRIGGER trg_payments_updated_at
BEFORE UPDATE ON payments
FOR EACH ROW
EXECUTE FUNCTION fn_actualizar_updated_at();
2) Trigger para registrar automáticamente en incident_history cuando
cambia el estado
CREATE OR REPLACE FUNCTION fn_registrar_historial_incidente()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.status IS DISTINCT FROM OLD.status THEN
        INSERT INTO incident_history (
            incident_id,
            status,
            changed_by_user_id,
            notes,
            "timestamp"
        )
        VALUES (
            NEW.id,
            NEW.status,
            COALESCE(NEW.user_id, OLD.user_id),
            'Cambio automático de estado del incidente',
            NOW()
        );
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
CREATE TRIGGER trg_incidents_historial_estado
AFTER UPDATE ON incidents
FOR EACH ROW
EXECUTE FUNCTION fn_registrar_historial_incidente();
```

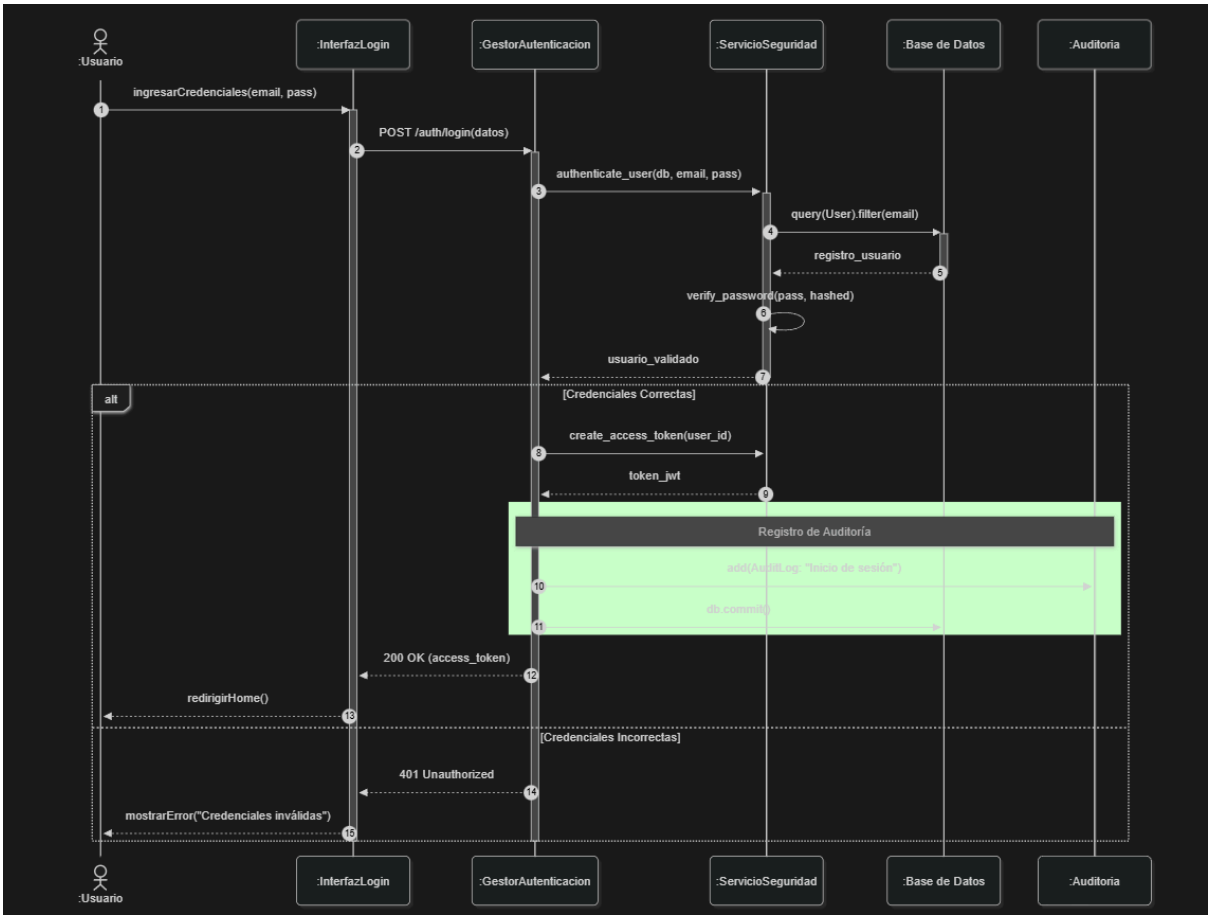
3.3.1.3 Diseñar Casos de Uso (Diagramas de Secuencia)

CICLO #1

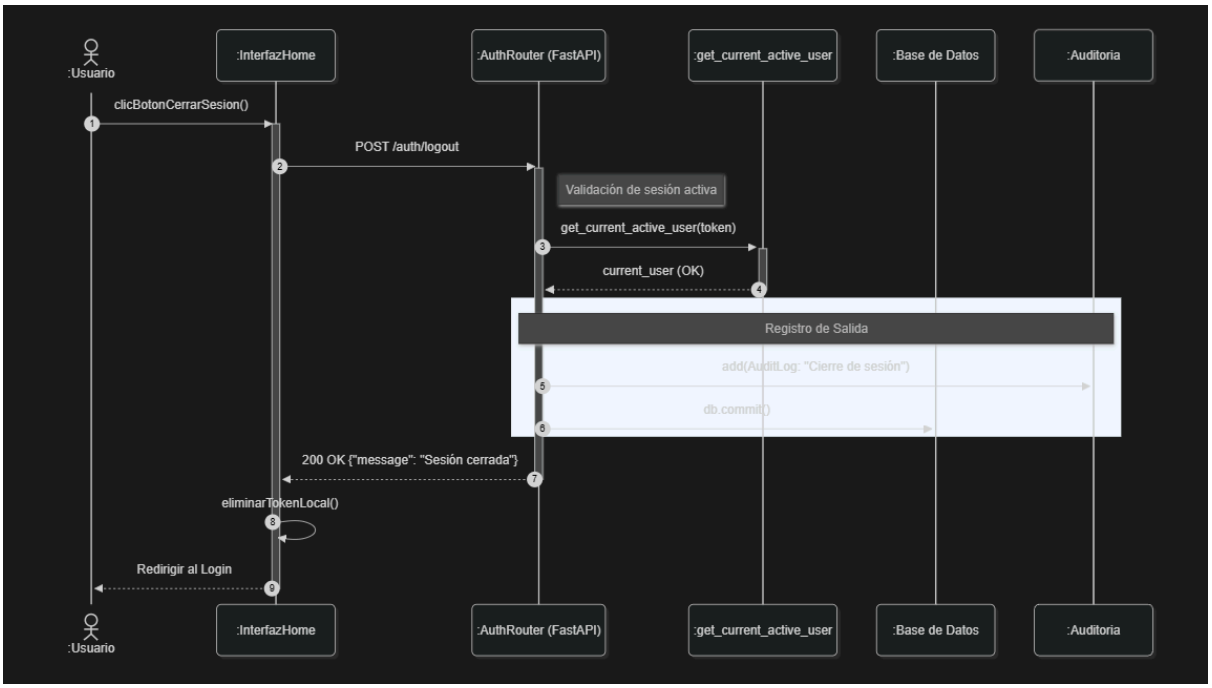
CU1-Registrarse en el Sistema



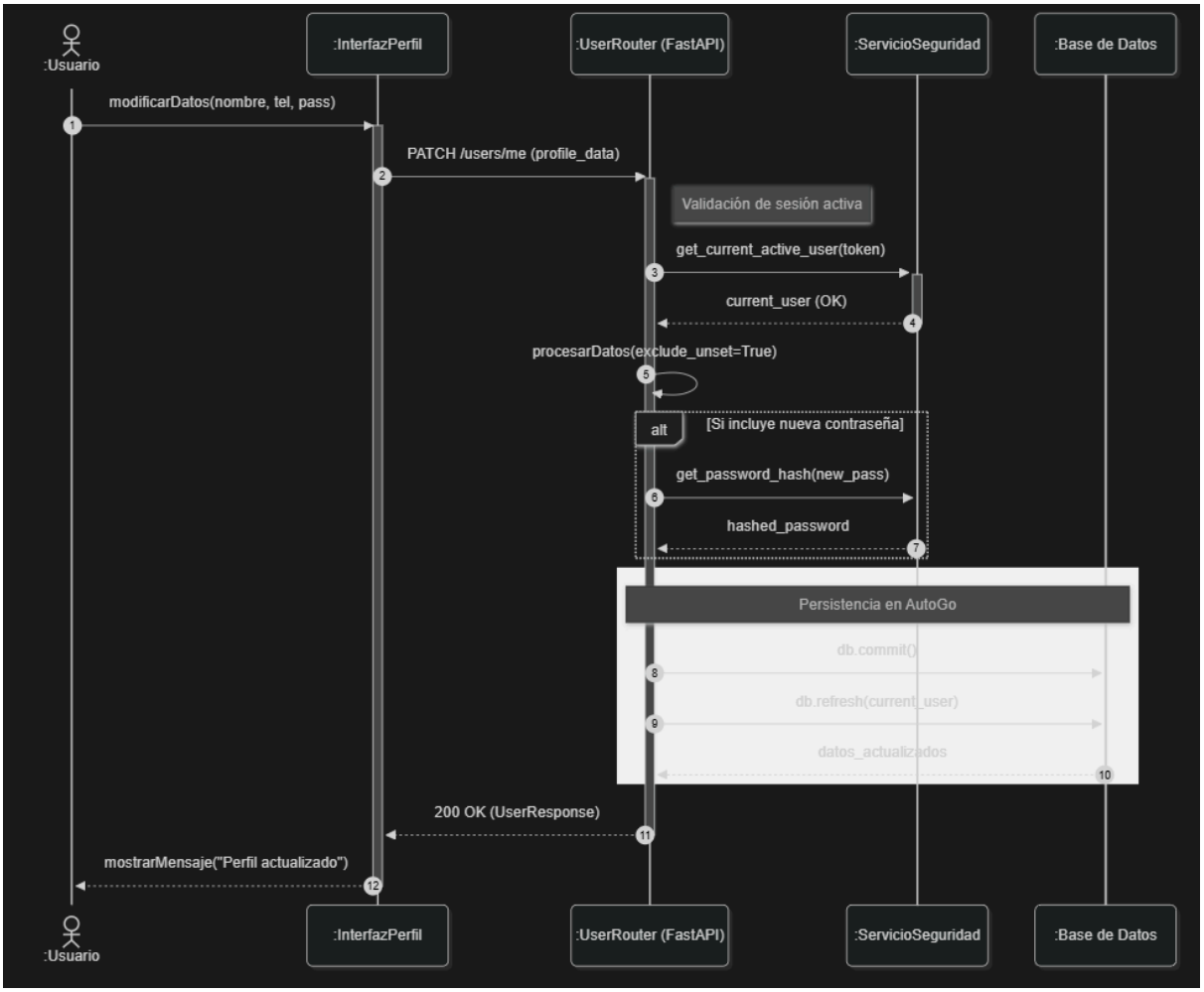
CU2- Iniciar Sesión



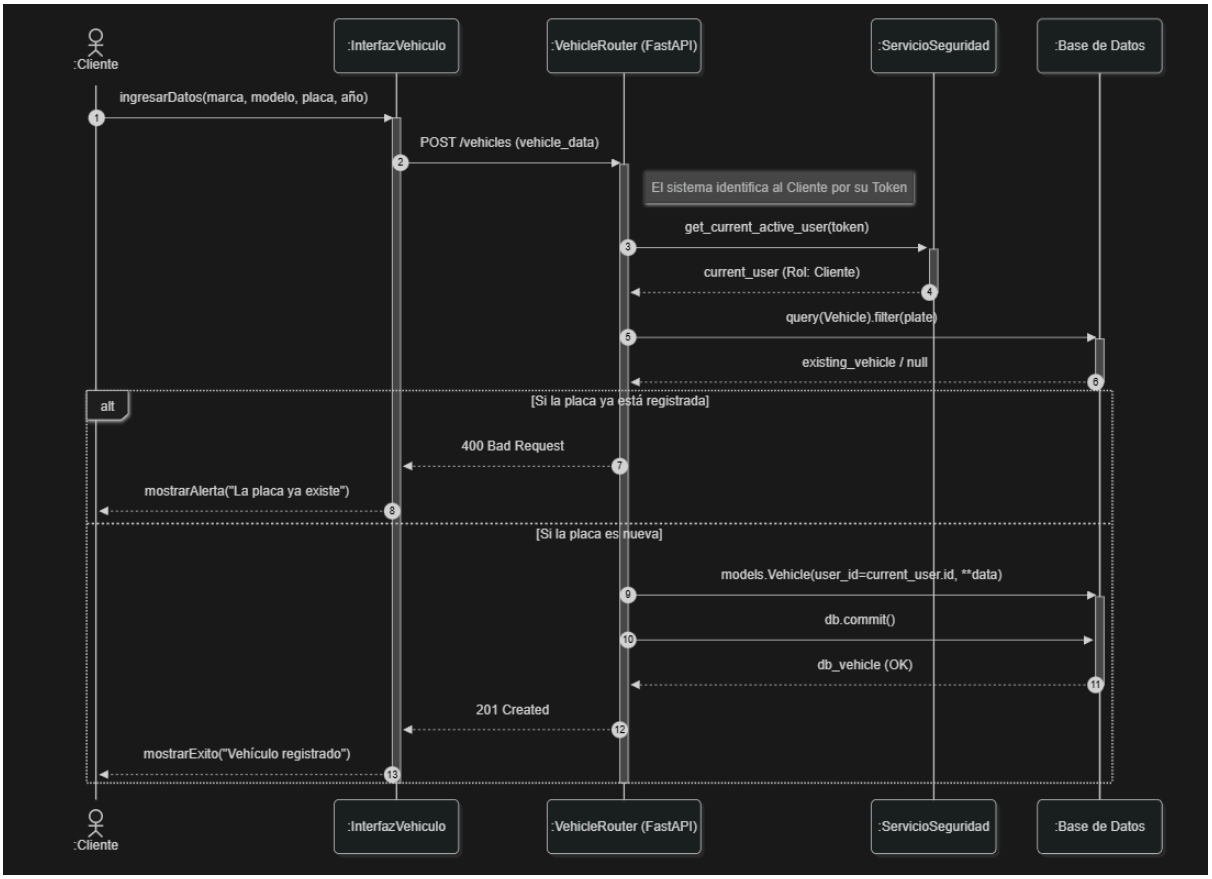
CU3 - Cerrar Sesión



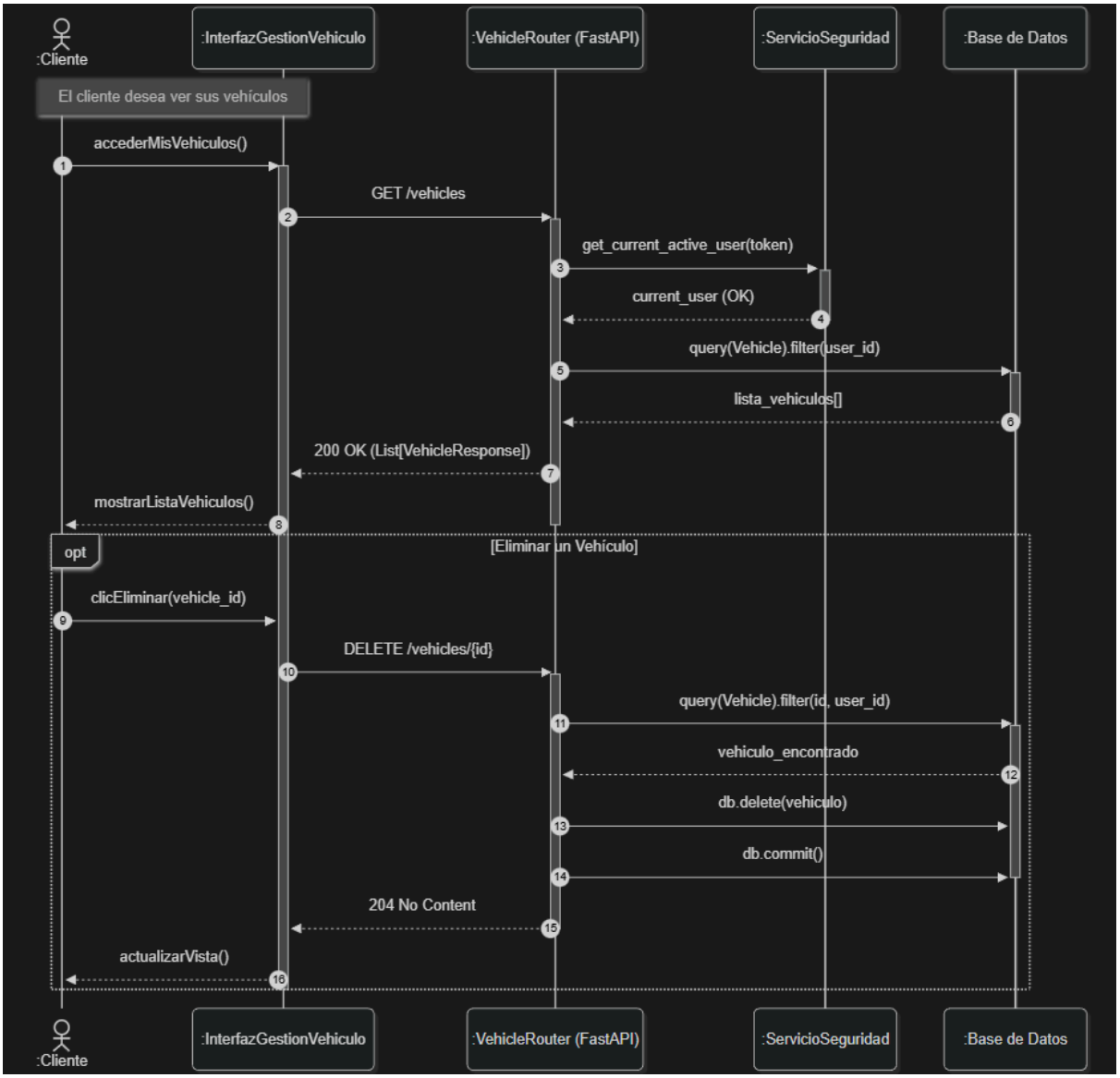
CU4- Gestionar Perfil de Usuario



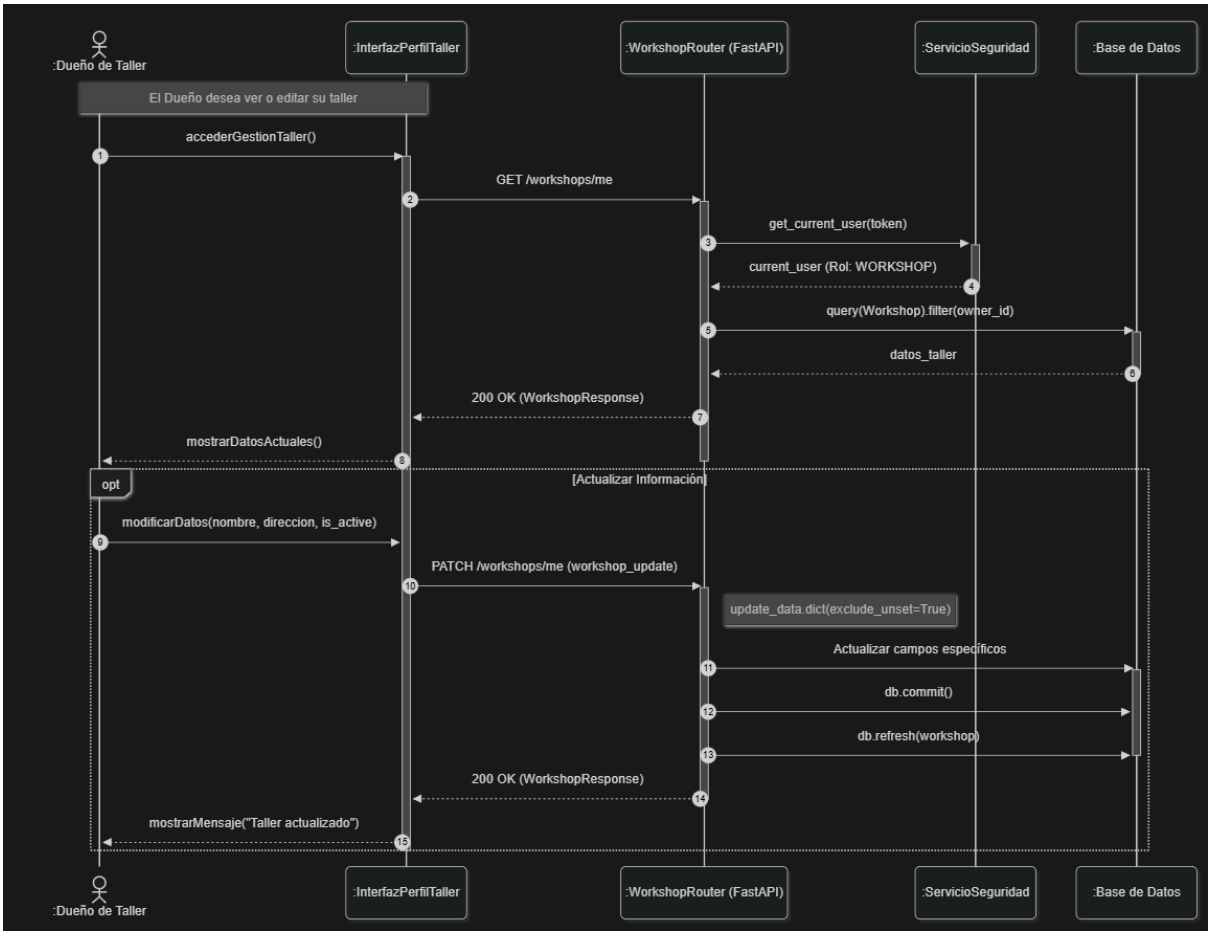
CU5- Registrar Vehículo



CU6- Gestionar Información del Vehículo

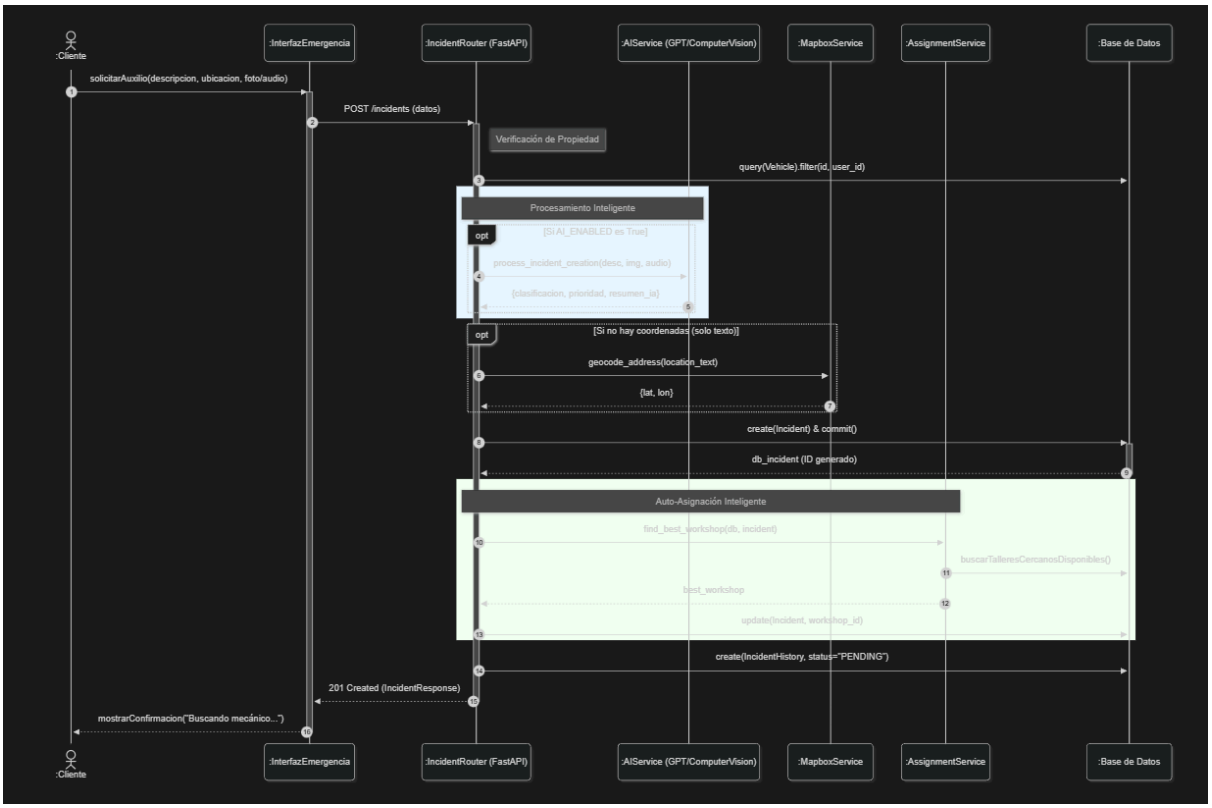


CU18- Gestionar Taller

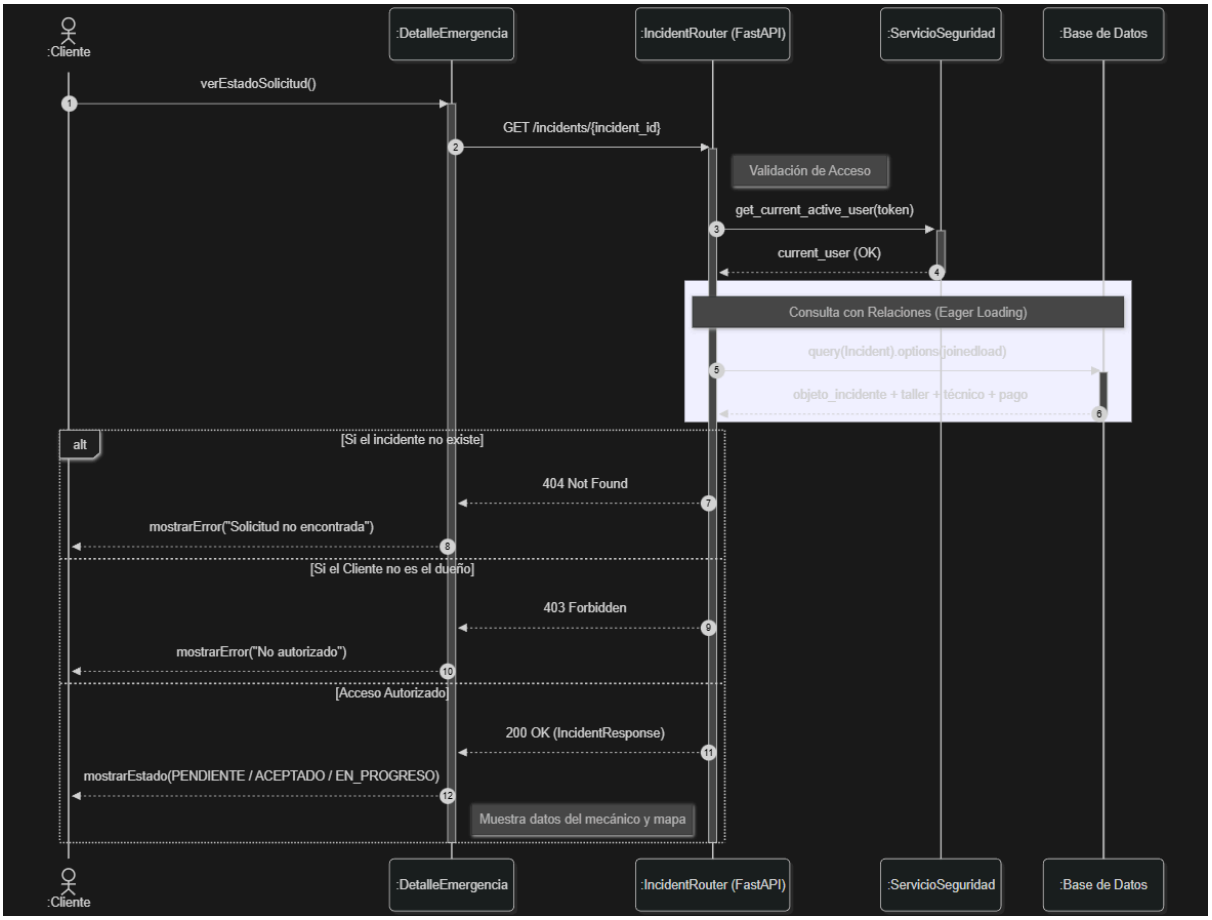


CICLO #2

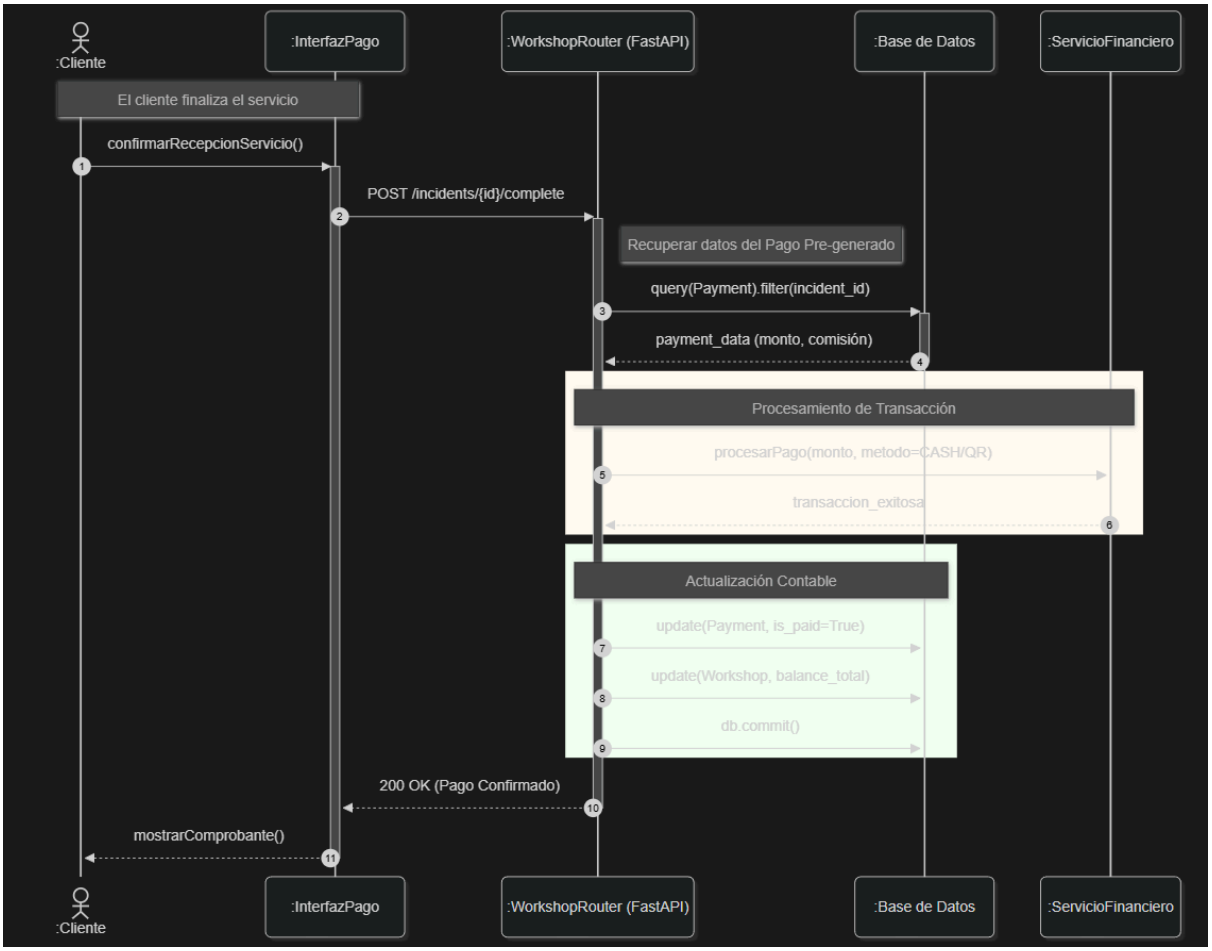
CU7 - Gestionar Emergencias



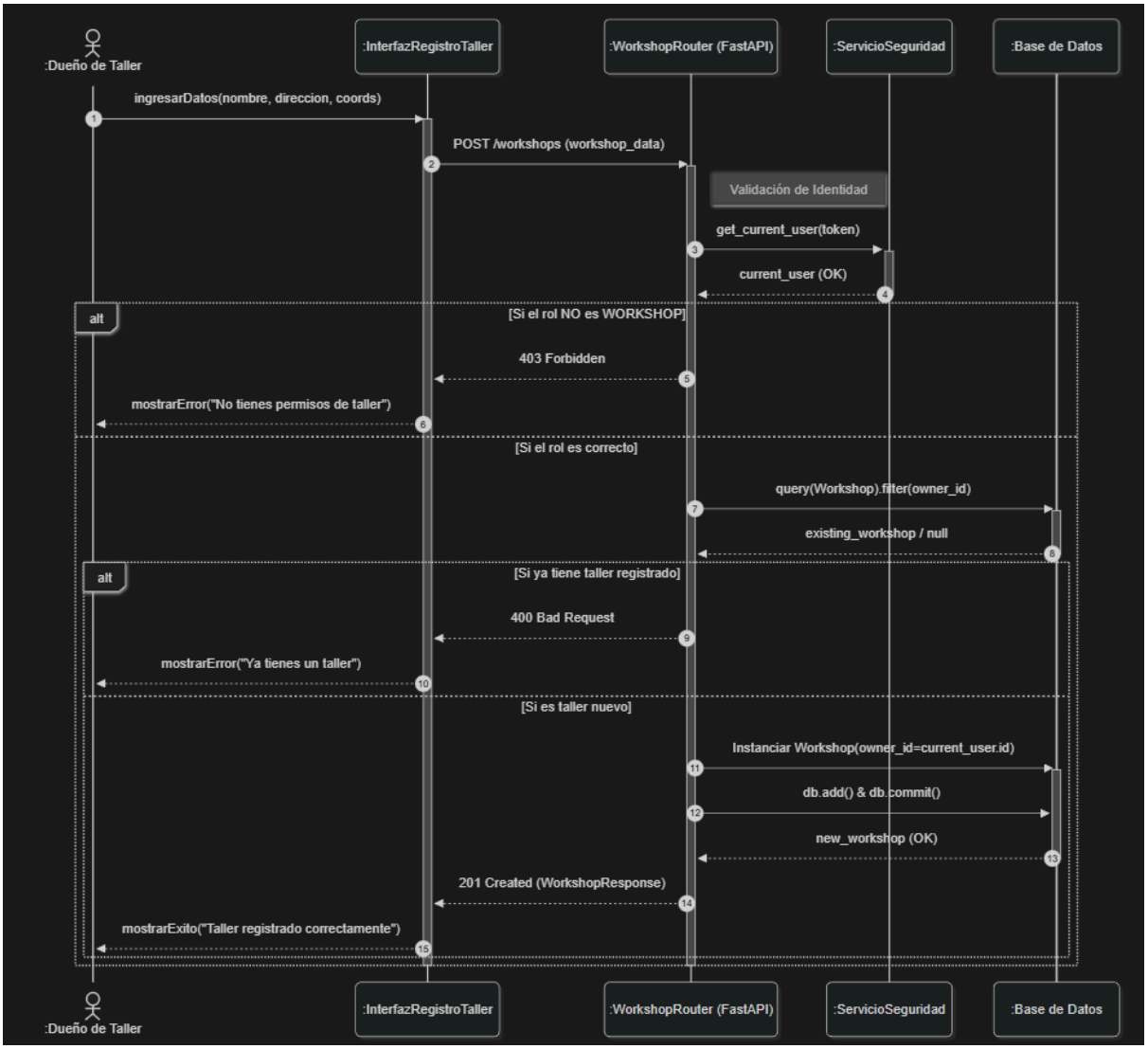
CU8. Consultar estado de la solicitud



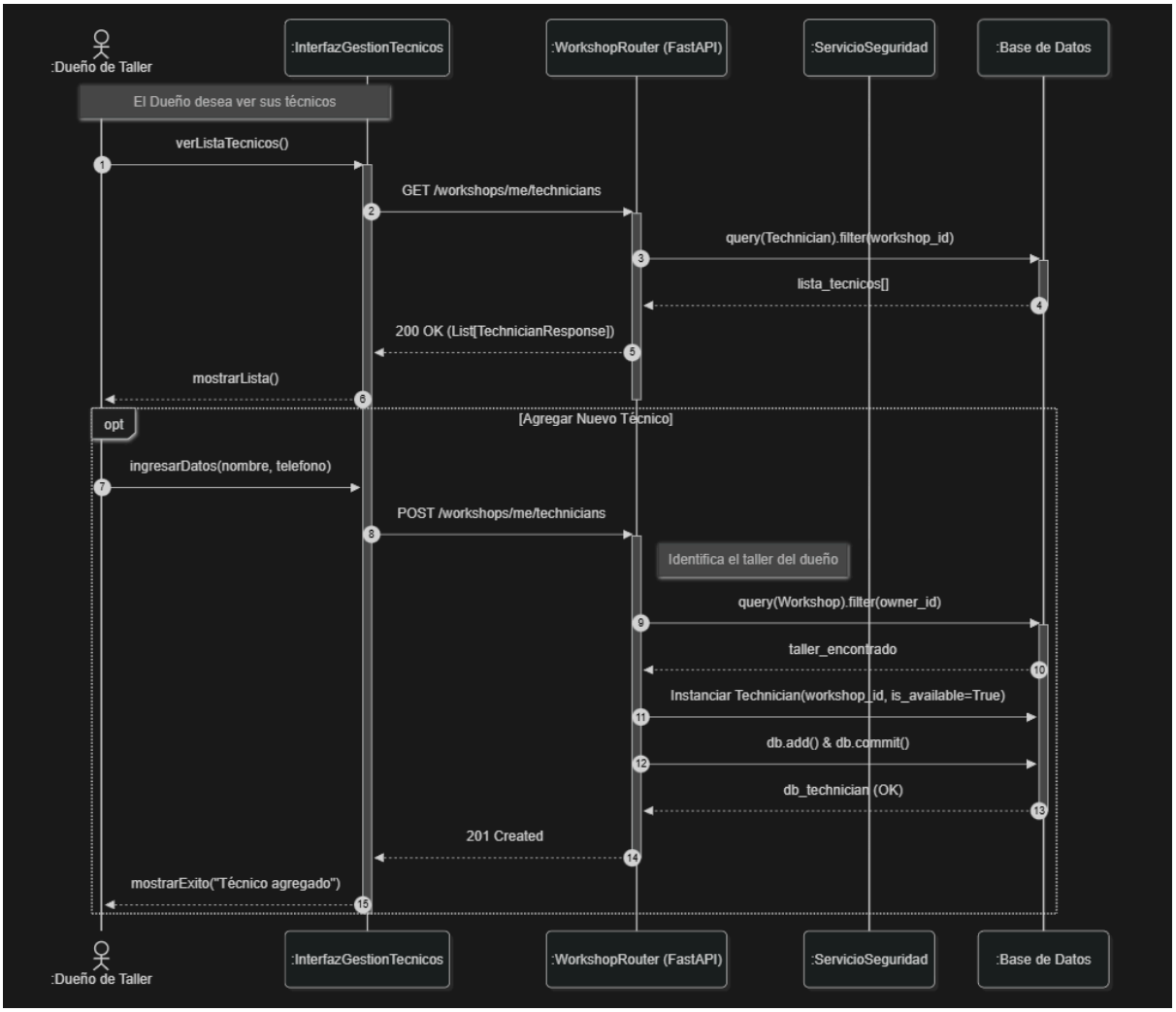
CU9. Realizar pago del servicio



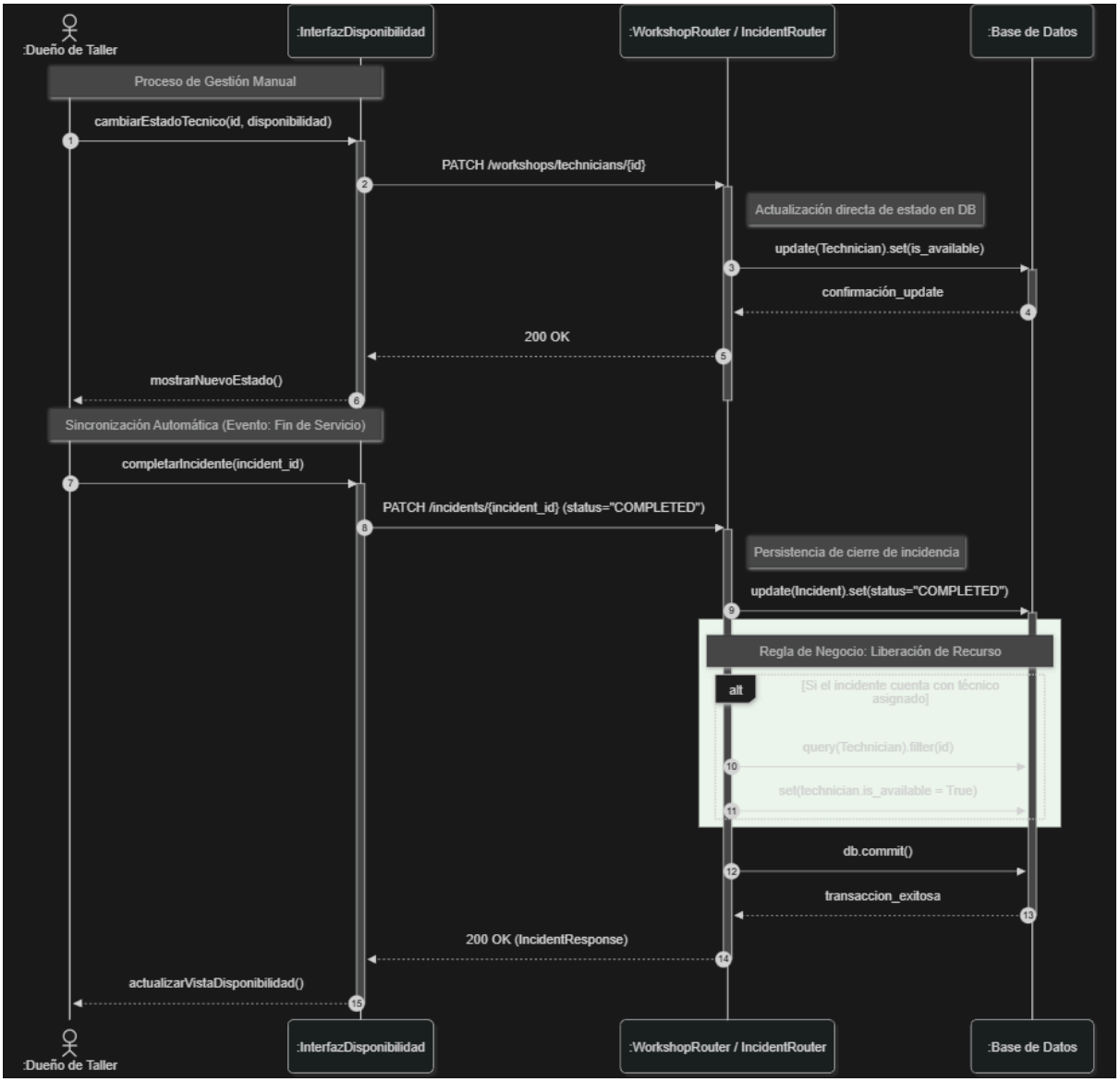
CU10. Registrar taller



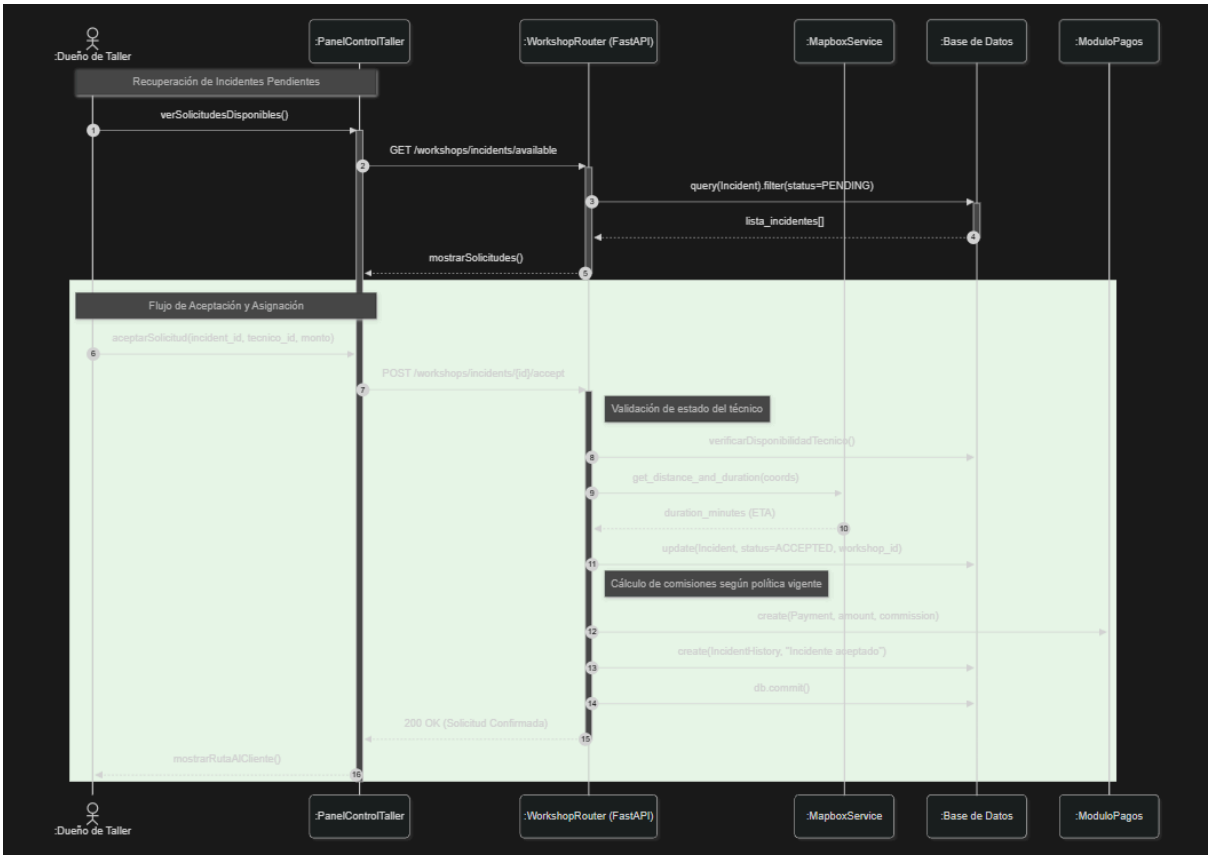
CU11. Gestionar técnicos del taller



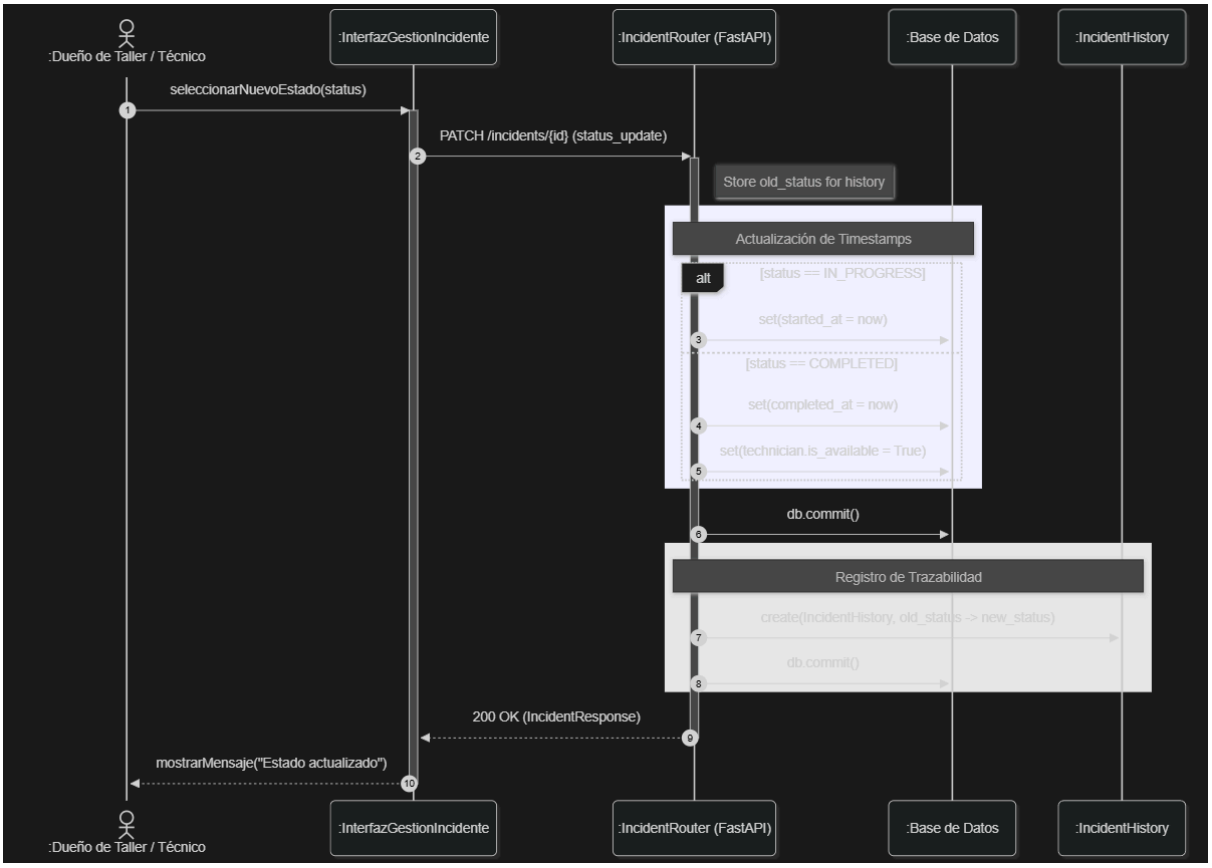
CU12. Gestionar disponibilidad de técnicos



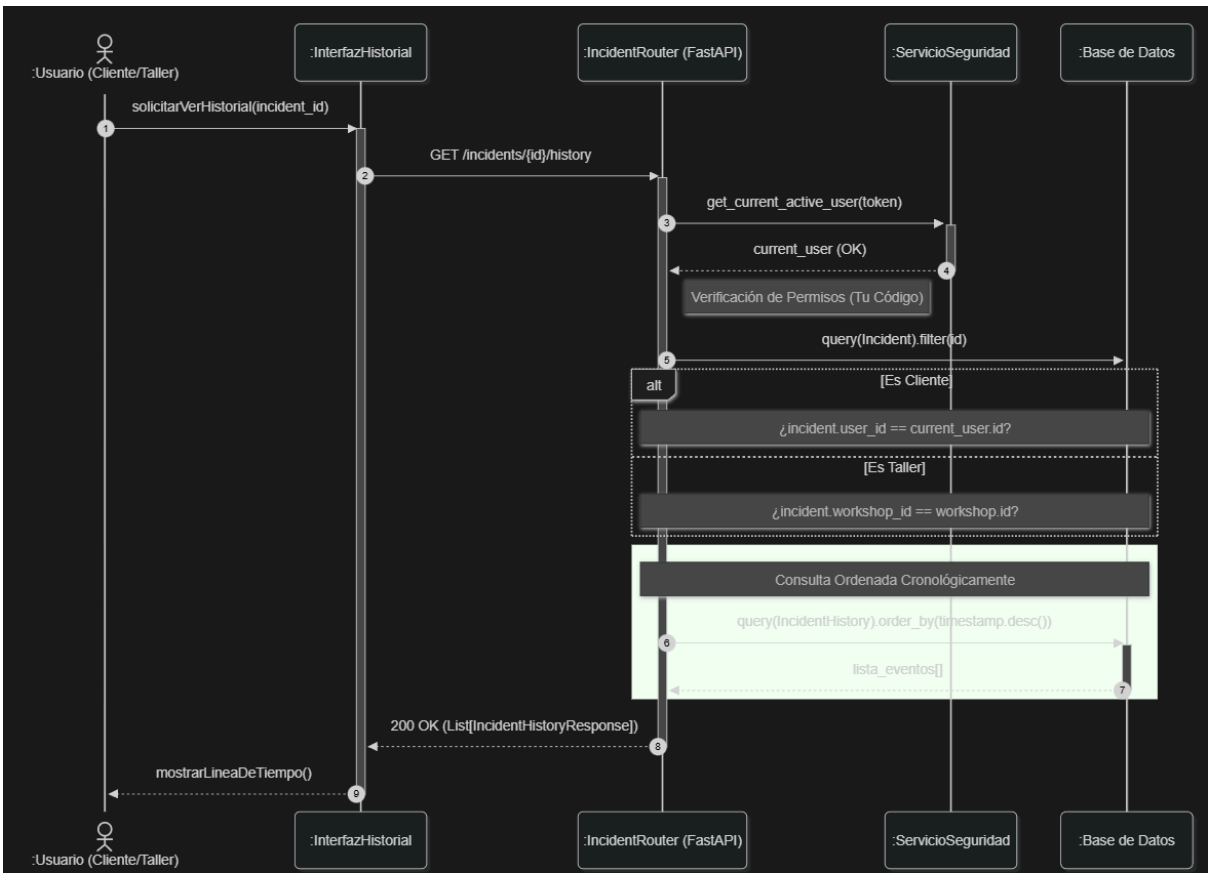
CU13. Gestionar solicitudes (Taller)



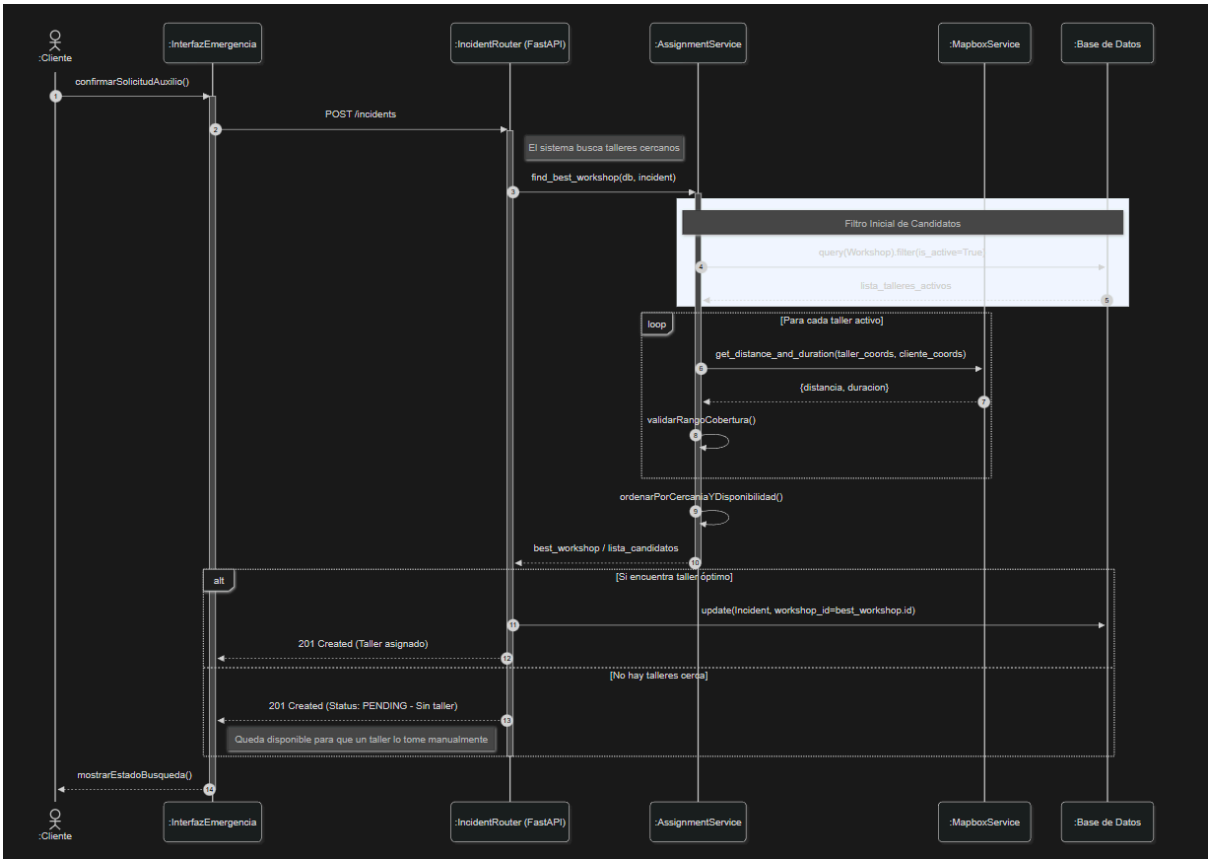
CU14. Actualizar estado del servicio



CU15. Consultar historial de atenciones

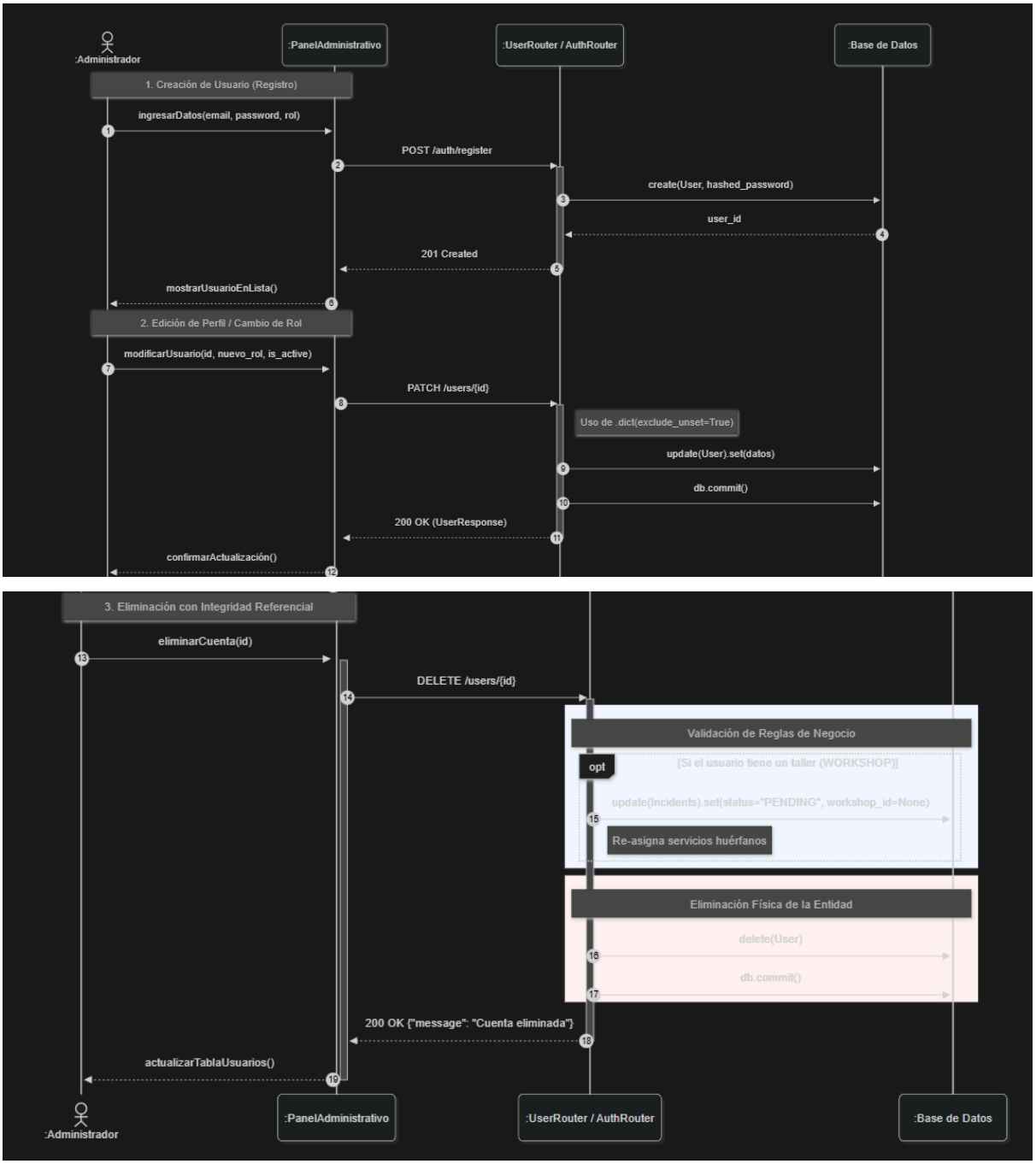


CU16. Generar lista de talleres candidatos

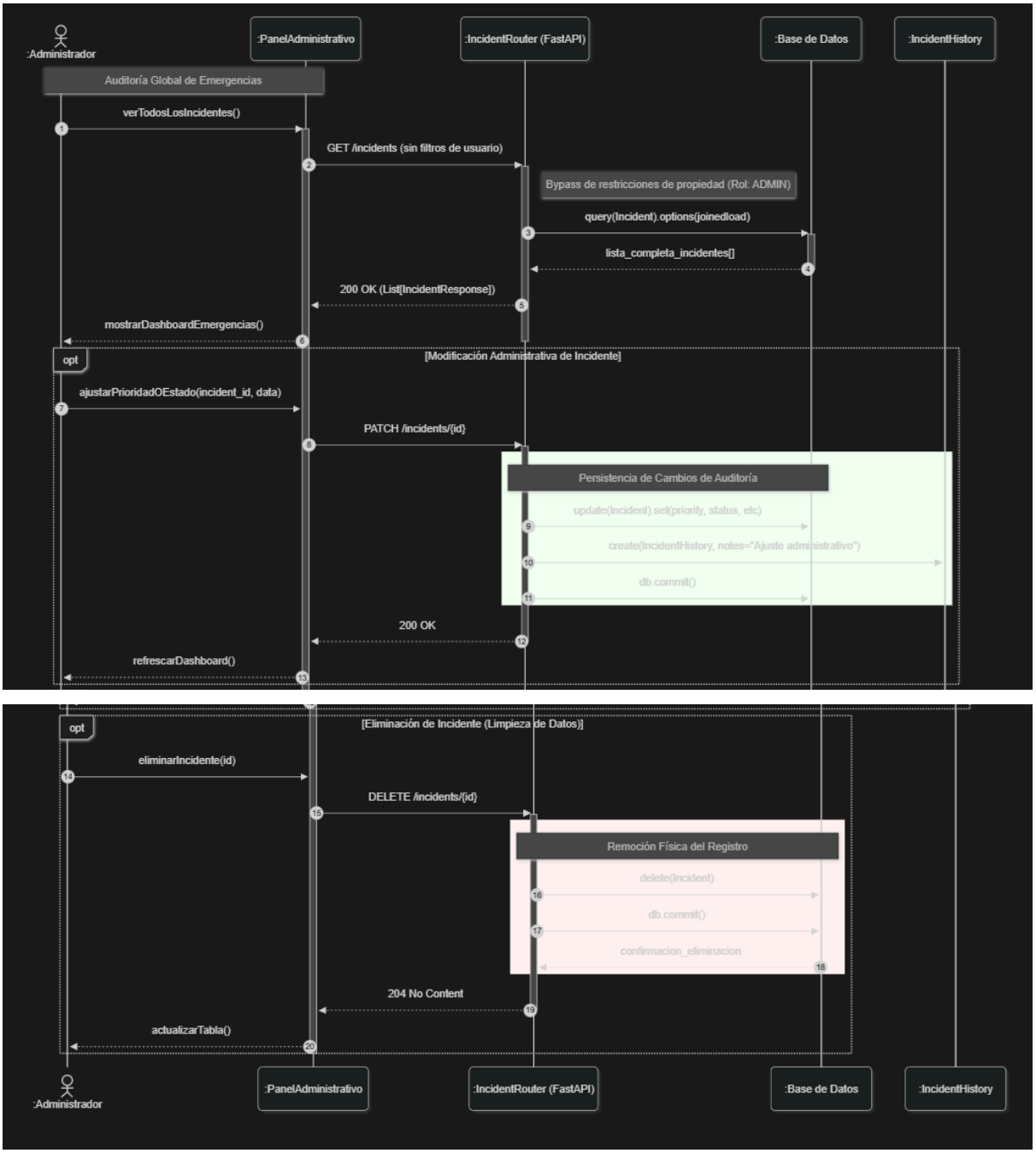


Ciclo #3

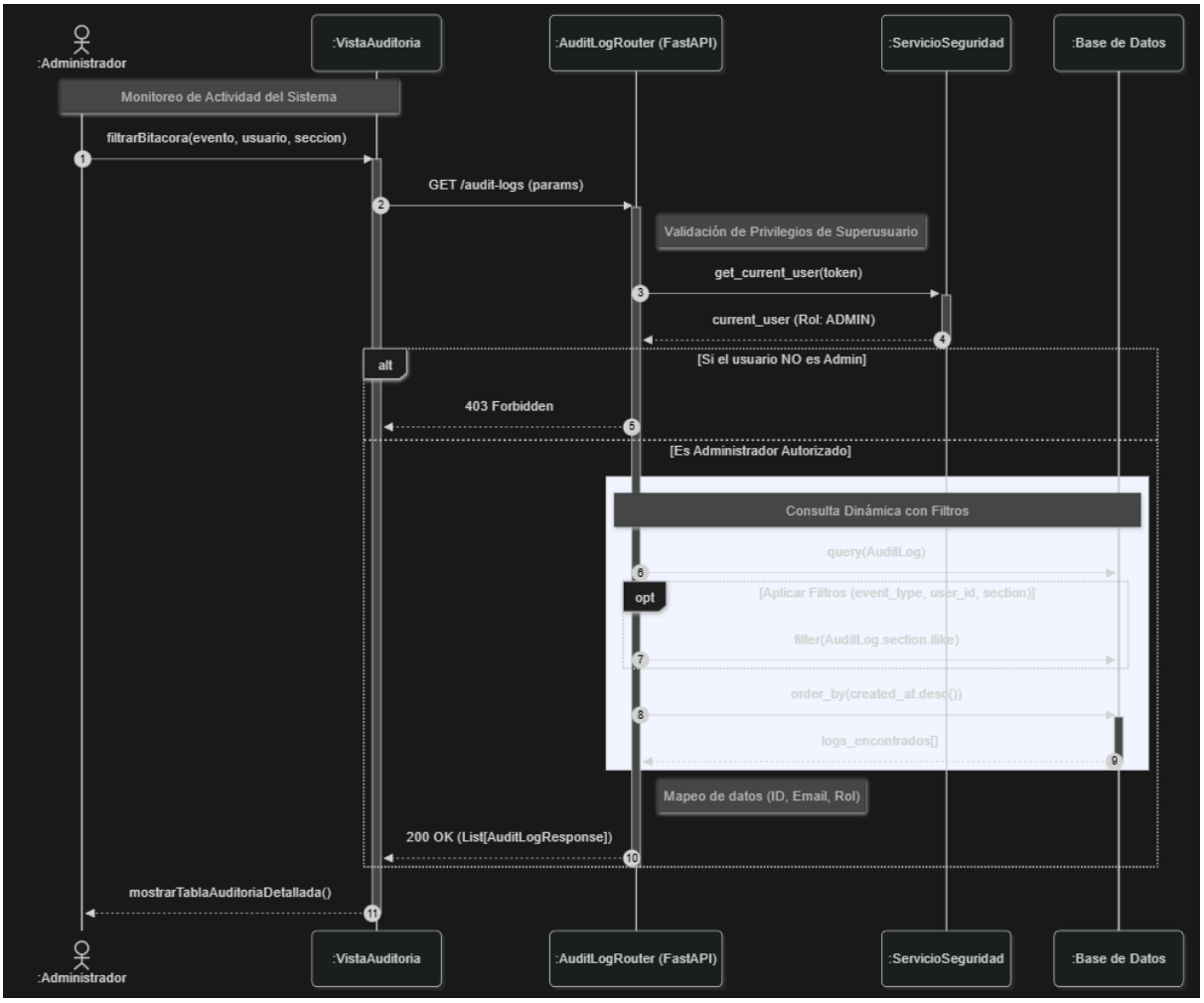
CU17. Gestionar Usuarios



CU19. Gestionar Incidentes



CU20. Bitácora del Sistema



3.4) FLUJO DE TRABAJO: IMPLEMENTACIÓN

3.4.1 Elección de Plataforma de Desarrollo de Software

Lenguaje de Programación

El sistema de información para la **Plataforma Inteligente de Atención de Emergencias Vehiculares** se desarrolló utilizando una combinación de lenguajes modernos que permiten manejar procesamiento en tiempo real, integración de inteligencia artificial y escalabilidad del sistema.

Python se emplea en la capa del servidor debido a su versatilidad y amplio soporte para desarrollo de APIs e integración con modelos de inteligencia artificial. Se utiliza junto con el framework **FastAPI**, que permite construir servicios REST eficientes, rápidos y altamente escalables. Python facilita además el procesamiento de datos multimodales (audio, imagen y texto), requerido para el análisis automático de incidentes.

En la capa del cliente web, se utiliza **TypeScript**, que mejora la robustez del desarrollo al añadir tipado estático a JavaScript. Este lenguaje se integra con Angular para construir interfaces dinámicas, estructuradas y mantenibles.

Para la aplicación móvil, se utiliza **Dart**, el lenguaje del framework Flutter, que permite desarrollar aplicaciones multiplataforma con alto rendimiento y una interfaz moderna, facilitando la interacción del usuario en situaciones de emergencia.

Framework Backend: FastApi

FastAPI es un framework moderno de alto rendimiento para la construcción de APIs con Python.

En este sistema, FastAPI se encarga de:

- La creación de APIs REST para comunicación entre cliente móvil, web y servidor.
- La gestión de autenticación y autorización de usuarios (clientes y talleres).
- El procesamiento de datos enviados por el usuario (audio, imágenes, texto y ubicación).
- La integración con módulos de inteligencia artificial para clasificación de incidentes.
- La implementación del sistema de asignación inteligente de talleres.
- La gestión de notificaciones en tiempo real.

Gracias a su rendimiento basado en **ASGI** y su compatibilidad con herramientas modernas, FastAPI permite manejar múltiples solicitudes simultáneamente, lo cual es esencial en un sistema de emergencias.

Librería Frontend: Angular

Angular es un framework de desarrollo frontend basado en TypeScript que permite construir aplicaciones web robustas y escalables.

En la plataforma, Angular facilita:

- La construcción de interfaces estructuradas para la gestión de solicitudes por parte de talleres.
- La implementación de componentes reutilizables (panel de control, historial, gestión de incidentes).
- La navegación fluida entre vistas mediante routing.
- La comunicación con el backend mediante servicios HTTP.
- La organización del código en módulos, mejorando la mantenibilidad del sistema.

El uso de Angular permite ofrecer una experiencia profesional para los talleres que gestionan múltiples solicitudes en tiempo real.

Framework Móvil: Flutter

Flutter es un framework multiplataforma que permite desarrollar aplicaciones móviles nativas desde una sola base de código.

En este sistema, Flutter permite:

- Registrar emergencias de forma rápida e intuitiva.
- Capturar y enviar datos multimodales (imágenes, audio, texto).
- Obtener y enviar la ubicación en tiempo real del usuario.
- Mostrar el estado de las solicitudes (pendiente, en proceso, atendido).
- Recibir notificaciones push en tiempo real.

Flutter garantiza una experiencia fluida y responsiva, fundamental en contextos donde el usuario necesita asistencia inmediata.

Bases de Datos

La aplicación utiliza **PostgreSQL**, un sistema de gestión de bases de datos relacional robusto y altamente confiable.

Entre sus ventajas destacan:

- Soporte para relaciones complejas entre entidades como usuarios, talleres, vehículos e incidentes.
- Integridad referencial y manejo de transacciones.
- Alto rendimiento en consultas complejas.
- Escalabilidad para grandes volúmenes de datos.

El modelo de datos fue diseñado para garantizar la trazabilidad completa de cada incidente, incluyendo historial, estados, evidencias y métricas.

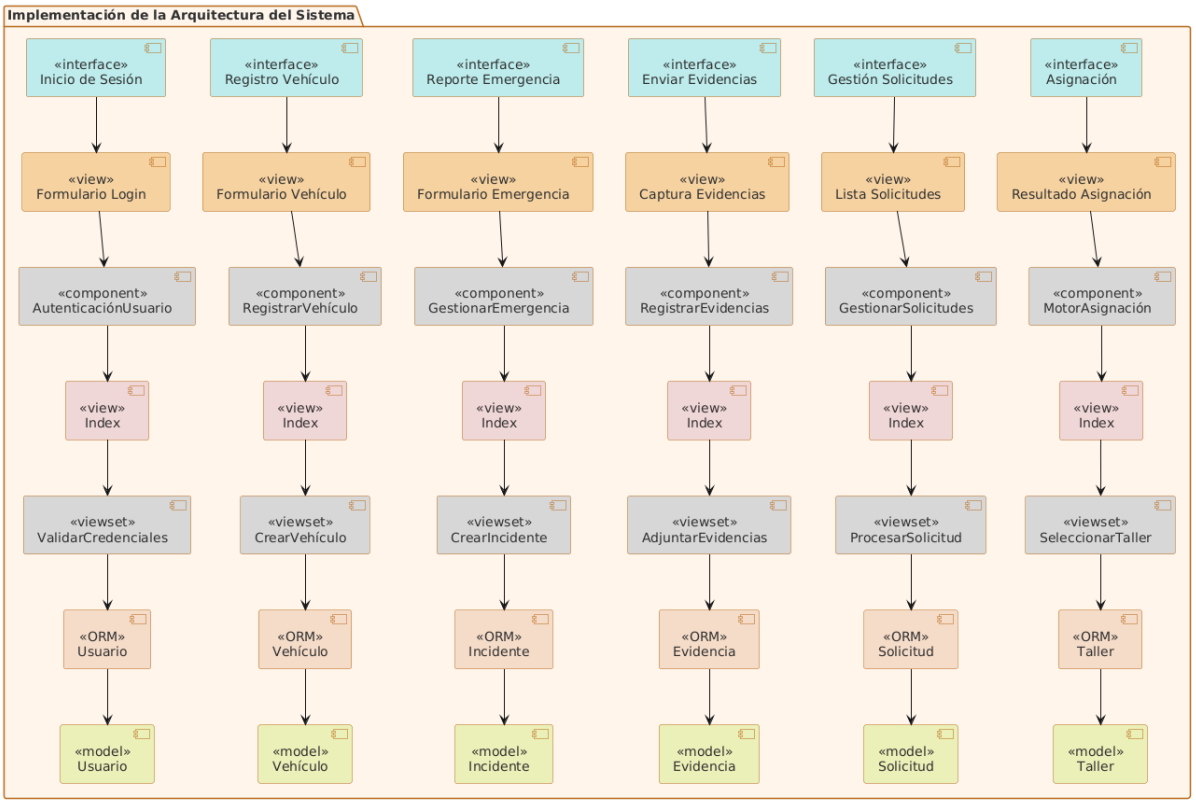
Entorno de Desarrollo

El desarrollo del sistema se realizó en entornos Windows, utilizando herramientas modernas que facilitan la colaboración y despliegue:

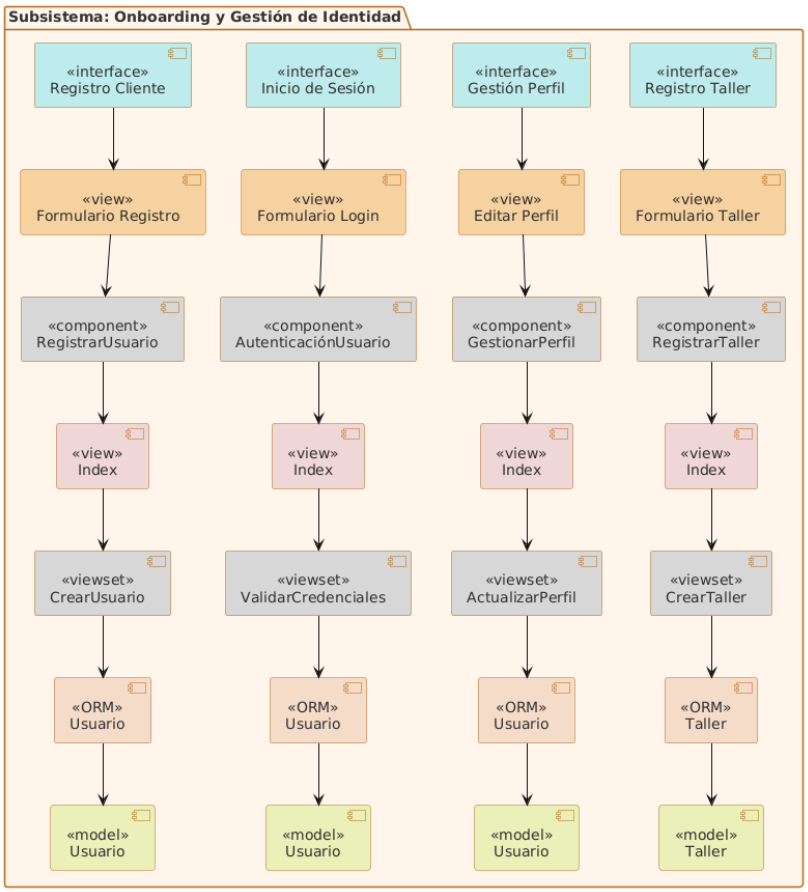
- Editor de código: Visual Studio Code
- Control de versiones: Git y GitHub
- Backend desplegable en servicios cloud
- Frontend web y app móvil preparados para despliegue multiplataforma

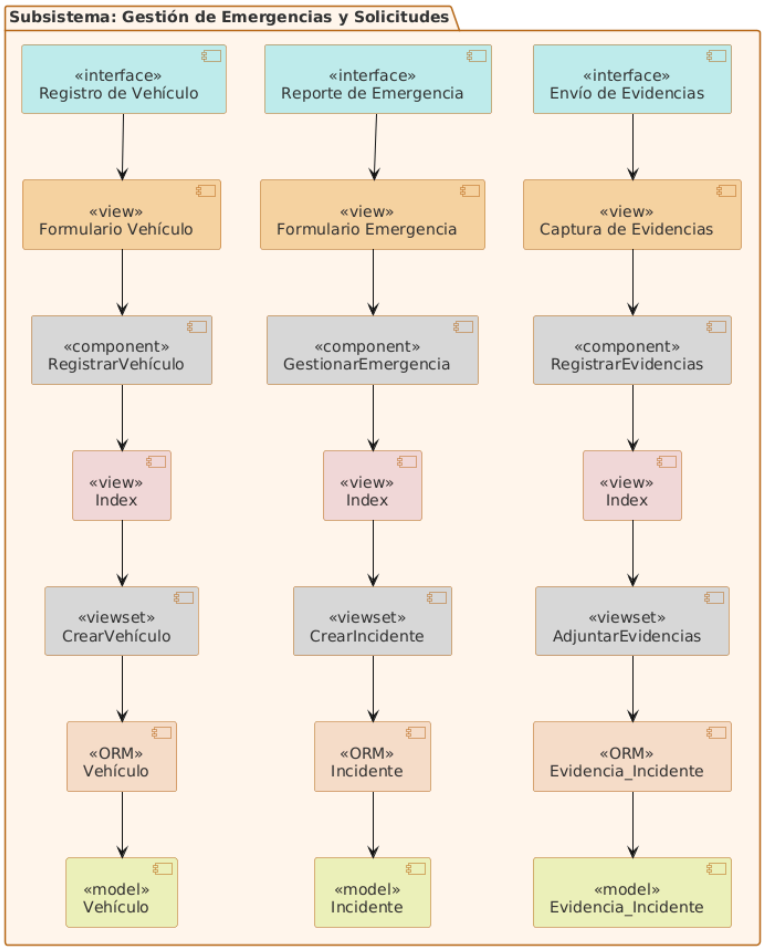
Este entorno permite mantener un flujo de trabajo organizado, colaborativo y escalable.

3.4.2 Implementación de la Arquitectura del Sistema



3.4.3 Implementación de la Arquitectura de Subsistemas





3.5) FLUJO DE TRABAJO: PRUEBAS

3.5.1 Planificar Pruebas

3.5.1.1 Objetivo de las Pruebas

El objetivo principal de esta fase es verificar y validar que la **Plataforma Inteligente de Atención de Emergencias Vehiculares** cumple con los requisitos funcionales y no funcionales definidos en el análisis del sistema.

Se busca garantizar que los módulos críticos funcionen correctamente, tales como:

- Registro y gestión de emergencias
- Procesamiento de datos multimodales (audio, imagen, texto)
- Clasificación automática de incidentes mediante inteligencia artificial
- Sistema de asignación de talleres
- Notificaciones en tiempo real

Asimismo, se pretende asegurar que la integración entre la aplicación móvil, la aplicación web, el backend y la base de datos sea consistente, confiable y eficiente, garantizando la trazabilidad completa de cada incidente.

3.5.1.2 Estrategia de Pruebas

Para garantizar la calidad del software, se aplicará una estrategia híbrida que combina enfoques complementarios:

Pruebas de Validación (Caja Negra)

Se centran en evaluar el comportamiento externo del sistema desde la perspectiva del usuario (cliente o taller), sin considerar la lógica interna del código.

Se validarán aspectos como:

- Registro de usuarios y vehículos
- Envío de emergencias con ubicación, imágenes y audio
- Visualización del estado de la solicitud
- Recepción de notificaciones
- Interacción con talleres

Enfoque:
Uso de tablas de especificación de casos de prueba, donde se definen entradas, acciones y resultados esperados.

3.5.1.3 Entorno de Pruebas

Las pruebas se ejecutarán bajo condiciones que simulan el entorno real de operación del sistema:

- **Aplicación Web:** Servidor local con Angular
- **Aplicación Móvil:** Ejecución en emulador/dispositivo físico mediante Flutter
- **Base de Datos:** PostgreSQL (instancia local o en la nube)
- **Backend API**: http://localhost:8000
- **Frontend Web**: http://localhost
- **API Docs:** http://localhost:8000/docs

Datos de Prueba:

Se utilizará un conjunto de datos simulados que incluye:

- Usuarios (clientes y talleres)
- Vehículos registrados
- Incidentes de prueba (batería, llanta, choque, etc.)
- Archivos multimedia (imágenes y audios simulados)

Estos datos serán insertados previamente mediante scripts para garantizar pruebas consistentes y repetibles.

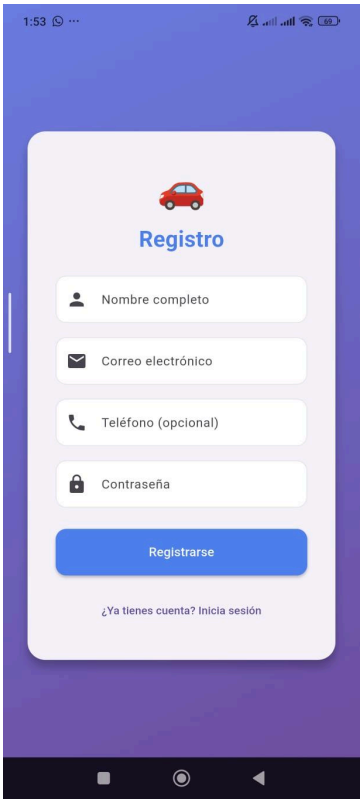
3.5.2 Casos de Pruebas

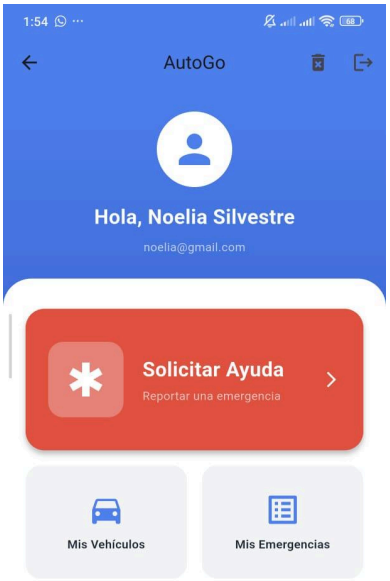
Caso de Prueba: CU1. Registrarse en el sistema (cliente)

Caso de Uso	CU1. Registrarse en el sistema
Actores	Cliente
Entrada	● El usuario ingresa sus datos personales: ○ correo electrónico ○ contraseña ○ nombre completo ○ teléfono ● Presiona el botón “Registrarse”
Condiciones	● El usuario no debe estar previamente registrado en el sistema. ● El correo electrónico ingresado debe ser único. ● Los datos ingresados deben cumplir con el formato requerido.
Procedimiento	1. El usuario accede a la interfaz de registro. 2. Ingresa sus datos personales (correo, contraseña, nombre y teléfono). 3. Verifica que la información ingresada sea correcta. 4. Presiona el botón “Registrarse”. 5. El sistema valida que el correo no esté registrado previamente. 6. El sistema cifra la contraseña del usuario. 7. El sistema almacena la información en la base de datos.
Resultado Final	1. El usuario queda registrado correctamente en el sistema. 2. El sistema almacena sus datos de forma segura. 3. El usuario puede iniciar sesión con sus credenciales.

Interfaz:

Entrada:





Salida:

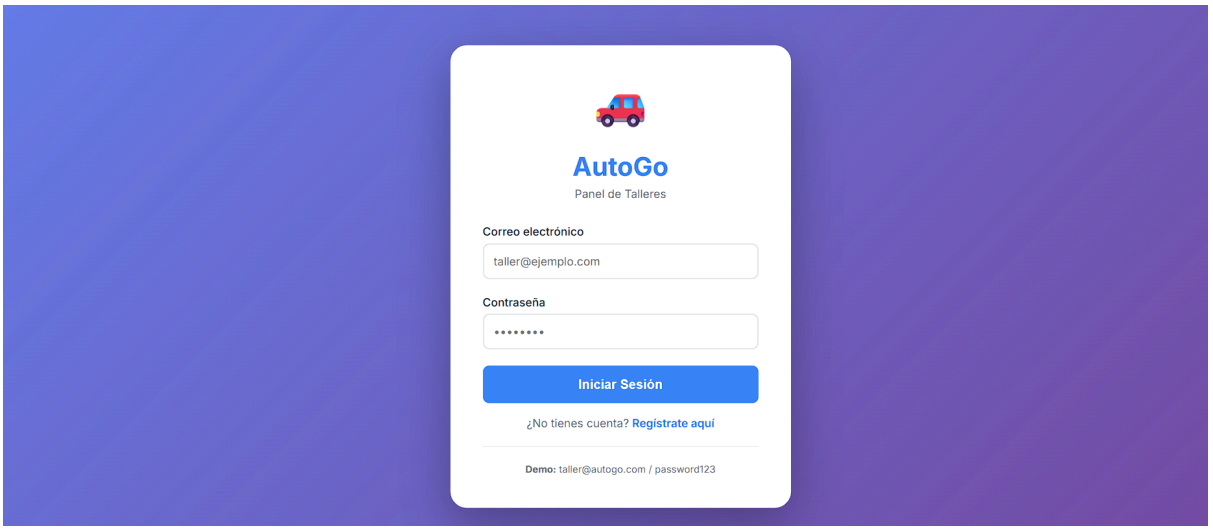


Caso de Prueba: CU2. Iniciar sesión

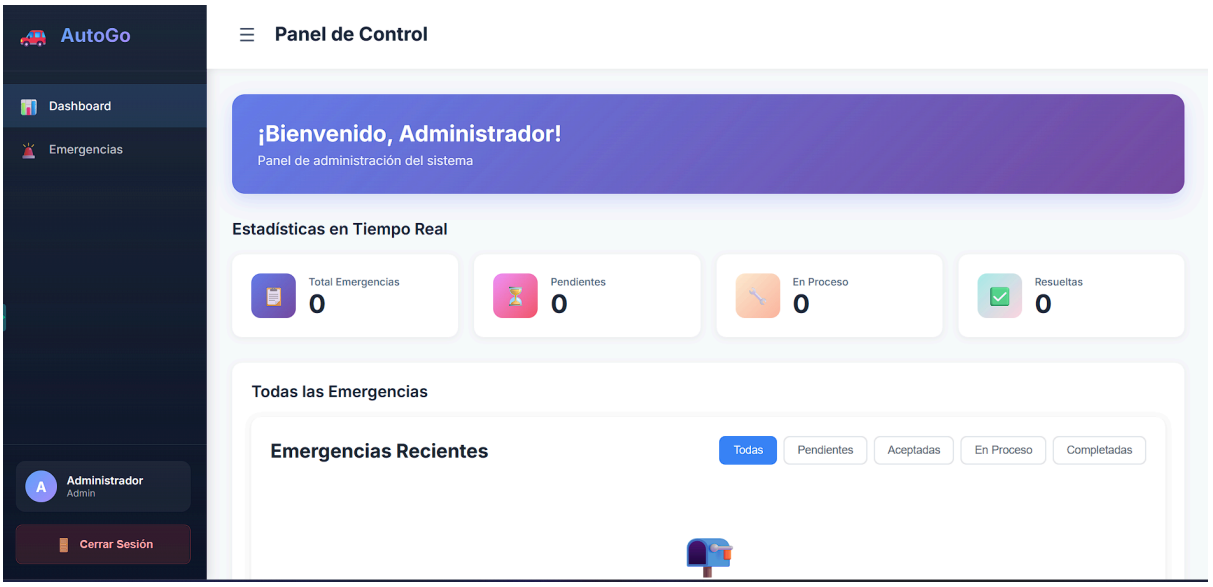
Caso de Uso	CU2. Iniciar Sesión
Actores	Administrador, Talleres, Cliente
Entrada	1. Se ponen los credenciales 2. Se presiona Iniciar Sesión
Condiciones	1. El actor debe poner los datos de una cuenta ya creada
Procedimiento	1. El Usuario rellena espacio por espacio 2. El Usuario verifica que puso bien su información 3. Presiona el botón de Iniciar Sesión
Resultado Final	1. La interfaz de iniciar sesión desaparece 2. La reemplaza el dashboard del usuario

Interfaz

Entrada:



Salida:

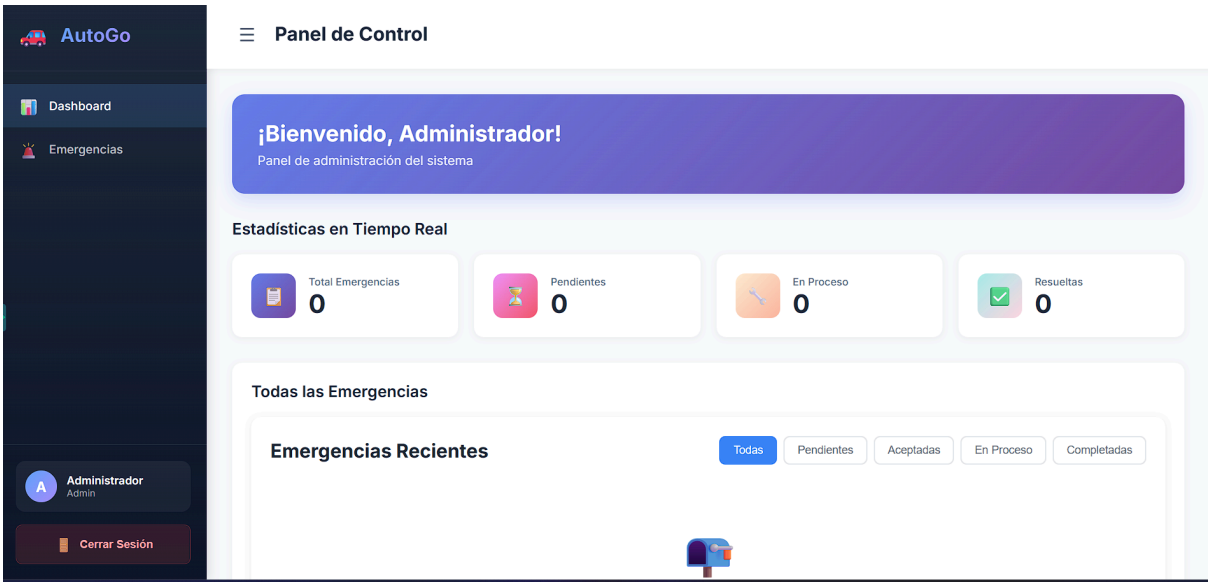


Caso de Prueba: CU3. Cerrar sesión

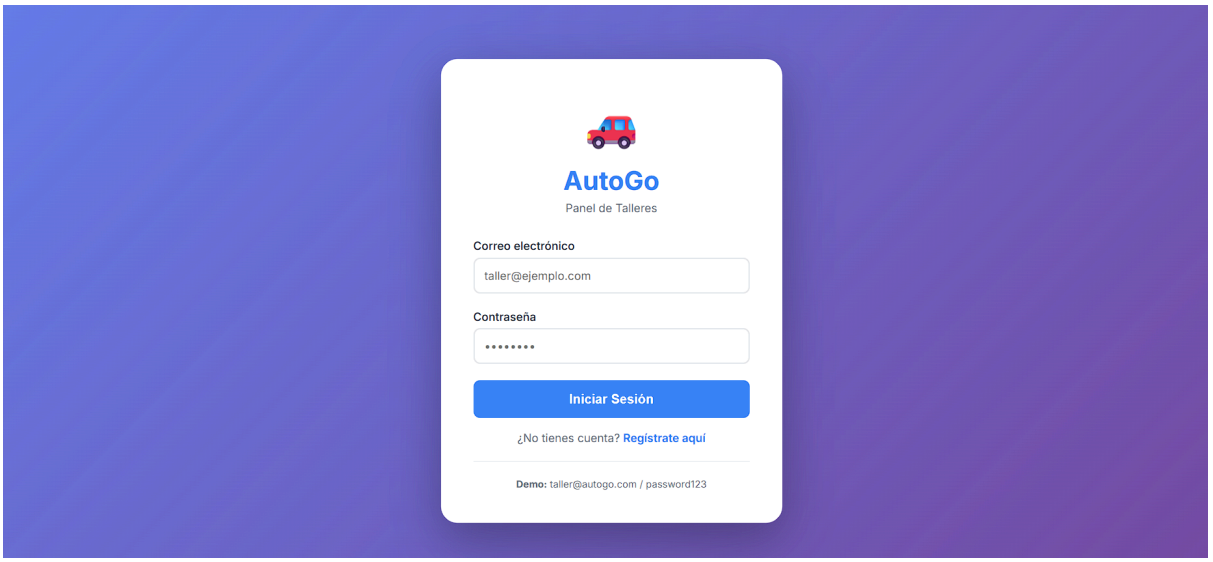
Caso de Uso	CU3. Cerrar Sesión
Actores	Administrador, Talleres, Cliente
Entrada	1. Desde el Dashboard de Usuario buscamos el botón de Cerrar Sesión 2. Presionamos el botón de Cerrar Sesión
Condiciones	2. El actor se encuentra dentro de su cuenta. Es necesario el CU1.
Procedimiento	1. El usuario presiona el botón de cerrar sesión 2. Los envía a la interfaz de Iniciar Sesión
Resultado Final	1. La interfaz del Dashboard cambia 2. Se reemplaza por el Login

Interfaz

Entrada:



Salida:

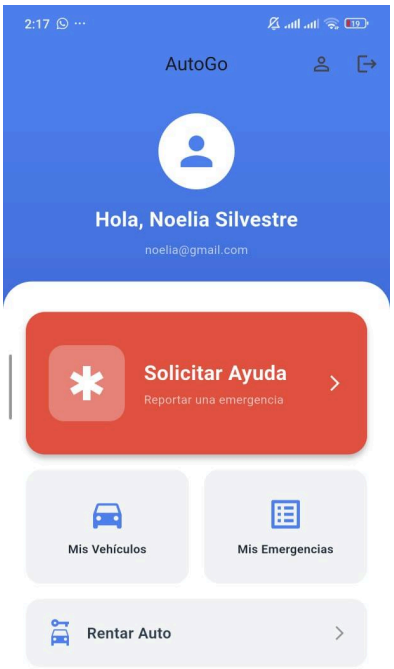


Caso de Prueba: CU4. Gestionar perfil de usuario

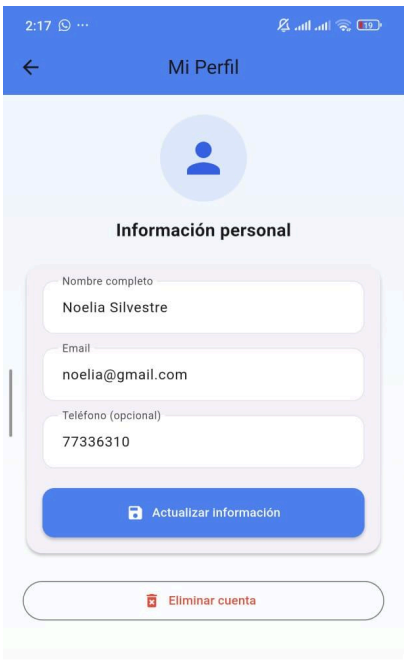
Caso de Uso	CU4. Gestionar perfil de usuario
Actores	Cliente
Entrada	• El usuario accede a su perfil desde la aplicación. • El sistema utiliza el token de autenticación (JWT) para identificar al usuario.

Condiciones	• El usuario debe haber iniciado sesión previamente. • El usuario debe estar autenticado con un token válido.
Procedimiento	1. El usuario accede a la opción de **perfil** en la aplicación. 2. El sistema recibe la solicitud junto con el token de autenticación. 3. El sistema valida el token del usuario. 4. El sistema obtiene la información del usuario desde la base de datos. 5. El sistema devuelve los datos del perfil al usuario. 6. El usuario visualiza su información personal.
Resultado Final	1. El usuario visualiza correctamente su información personal. 2. El sistema muestra los datos actualizados del usuario. 3. Se mantiene la sesión activa del usuario.

Interfaz



Entrada: 



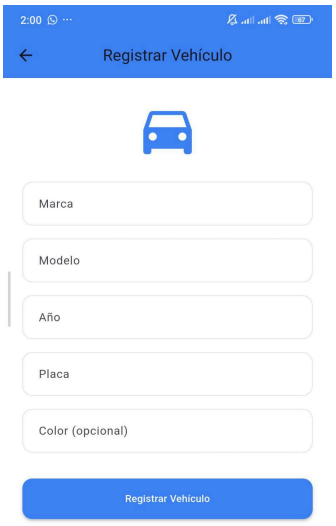
Salida: 

Caso de Prueba: CU5. Registrar vehículo

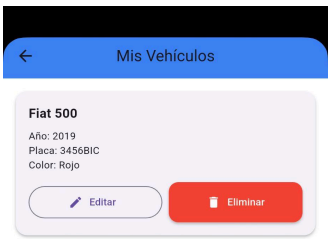
Caso de Uso	CU5. Registrar vehículo
Actores	Cliente
Entrada	El usuario ingresa los datos del vehículo: • marca • modelo • año • placa • color Presiona el botón **“Registrar Vehículo”**
Condiciones	1. El usuario debe haber iniciado sesión. 2. El vehículo no debe estar previamente registrado (placa única). 3. Los datos ingresados deben ser válidos.

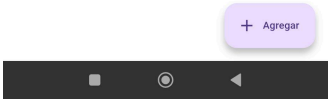
Procedimiento	1. El usuario accede a la opción de **registro de vehículo**. 2. Ingresa los datos del vehículo (marca, modelo, año, placa y color). 3. Verifica que la información sea correcta. 4. Presiona el botón **“Registrar Vehículo”**. 5. El sistema valida que la placa no esté registrada previamente. 6. El sistema asocia el vehículo al usuario autenticado. 7. El sistema guarda la información en la base de datos.
Resultado Final	● El vehículo es registrado correctamente en el sistema. ● El vehículo queda asociado al usuario que lo registró. ● El usuario puede visualizar su vehículo en el sistema.

Interfaz



Entrada:



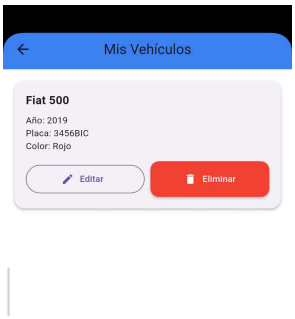
Salida: 

Caso de Prueba: CU6. Gestionar información del vehículo

Caso de Uso	CU6. Gestionar información del vehículo
Actores	Cliente
Entrada	El usuario accede al módulo de **vehículos registrados**. Selecciona una opción: ● visualizar vehículos ● ver detalle de un vehículo ● eliminar vehículo
Condiciones	● El usuario debe haber iniciado sesión. ● El vehículo debe pertenecer al usuario autenticado. ● El vehículo debe existir en la base de datos.
Procedimiento	1. El usuario accede a la sección de **gestión de vehículos**. 2. El sistema obtiene la lista de vehículos asociados al usuario. 3. El usuario puede seleccionar un vehículo para ver su información detallada.

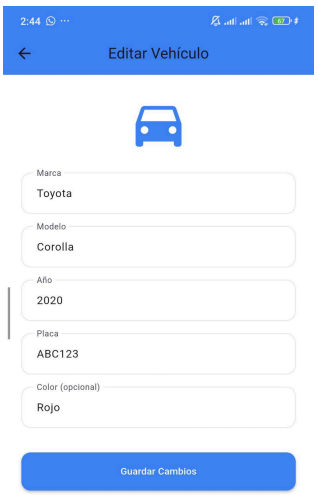
	4. El sistema muestra los datos del vehículo seleccionado. 5. Opcionalmente, el usuario puede eliminar un vehículo. 6. El sistema valida que el vehículo pertenezca al usuario. 7. El sistema elimina el vehículo de la base de datos.
Resultado Final	● El usuario visualiza correctamente sus vehículos registrados. ● El usuario puede consultar el detalle de un vehículo específico. ● El vehículo seleccionado puede ser eliminado correctamente del sistema.

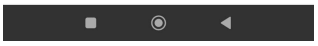
Interfaz



Entrada:





Salida: 

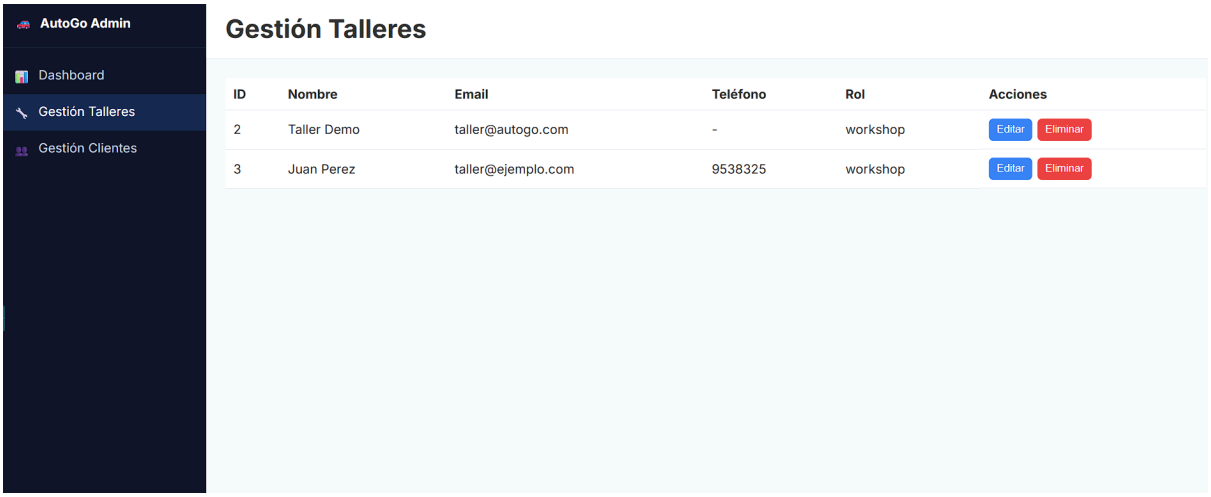
Caso de Prueba: CU18. Gestionar talleres

Caso de Uso	CU18. Gestionar taller
Actores	Taller
Entrada	El usuario accede al módulo de gestión de taller. Ingresa o modifica los datos del taller: ● nombre ● dirección ● ubicación (latitud y longitud) ● porcentaje de comisión ● estado (activo/inactivo)
Condiciones	● El usuario debe haber iniciado sesión. ● El usuario debe tener rol **WORKSHOP**. ● El usuario solo puede gestionar su propio taller. ● Para registrar, el usuario no debe tener previamente un taller creado.
Procedimiento	1. El usuario accede a la sección de **gestión de taller**.

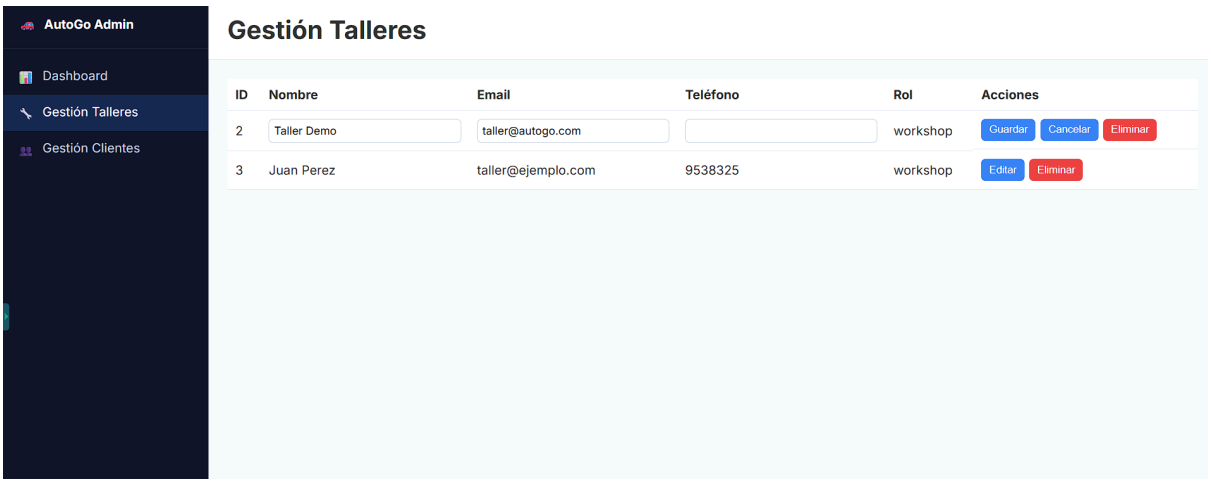
	2. El sistema valida que el usuario tenga rol de taller. 3. Si no tiene un taller registrado, el usuario ingresa los datos para crear uno. 4. El sistema valida la información ingresada. 5. El sistema registra el nuevo taller en la base de datos. 6. Si el taller ya existe, el usuario puede visualizar su información. 7. El usuario puede modificar los datos del taller. 8. El sistema actualiza la información en la base de datos.
Resultado Final	● El taller está registrado correctamente en el sistema. ● El usuario puede visualizar la información de su taller. ● El usuario puede actualizar los datos de su taller. ● La información del taller se mantiene actualizada y asociada al usuario.

Interfaz

Entrada:



Salida:



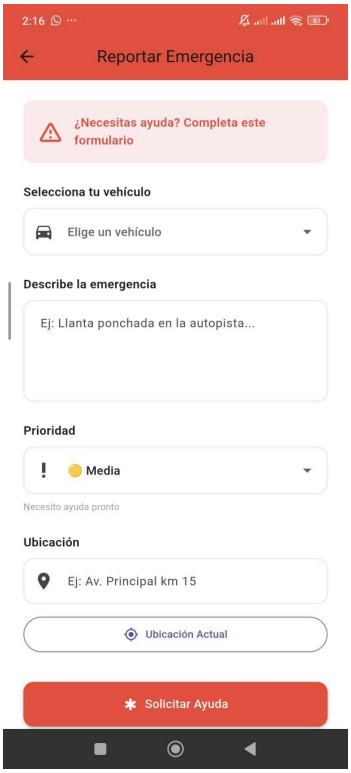
Caso de Prueba: CU7. Gestionar emergencias

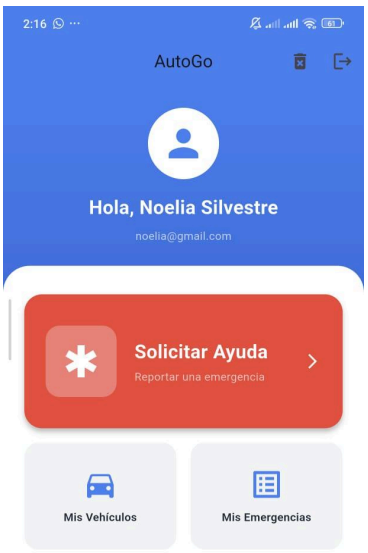
Caso de Uso	CU7. Gestionar emergencias
Actores	Cliente
Entrada	El usuario registra una emergencia ingresando o adjuntando: ● vehículo seleccionado ● descripción del problema ● ubicación en tiempo real ● imagen del incidente ● audio descriptivo ● texto adicional opcional Luego presiona el botón **“Enviar emergencia”**.
Condiciones	El usuario debe haber iniciado sesión. El usuario debe tener al menos un vehículo registrado. Debe existir conexión con el backend para enviar la información. La información mínima requerida debe estar completa.
Procedimiento	El usuario accede al módulo de emergencias. Selecciona el vehículo involucrado. Ingresa la descripción del problema. Adjunta imagen, audio o texto adicional, si corresponde. Comparte su ubicación actual. Presiona el botón **“Enviar emergencia”**. El sistema valida los datos ingresados. El sistema procesa la información recibida. El sistema ejecuta el análisis preliminar mediante IA. El sistema registra la emergencia en la base de datos.

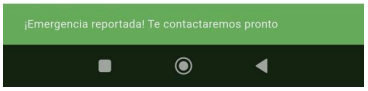
Resultado Final	La emergencia queda registrada correctamente en el sistema. El incidente queda asociado al usuario y al vehículo. El sistema genera una clasificación preliminar y prioridad inicial. El usuario puede consultar posteriormente el estado de su solicitud.

Interfaz

Entrada:





Salida: 

Caso de Prueba: CU8. Consultar estado de la solicitud

Caso de Uso	CU8. Consultar estado de la solicitud
Actores	Cliente
Entrada	El usuario accede a la sección **"Mis solicitudes"** o selecciona una solicitud registrada.
Condiciones	El usuario debe haber iniciado sesión. Debe existir al menos una solicitud registrada a nombre del usuario. La solicitud debe existir en la base de datos.
Procedimiento	El usuario accede al módulo de seguimiento de solicitudes. El sistema consulta las solicitudes asociadas al usuario autenticado. El usuario selecciona una solicitud específica.

	El sistema recupera el estado actual de la solicitud. El sistema muestra el taller asignado, el técnico asignado si existe, y el tiempo estimado de llegada. El usuario visualiza el detalle actualizado del servicio.
Resultado Final	El usuario visualiza correctamente el estado de su solicitud. El sistema muestra información actualizada del proceso de atención. El usuario puede dar seguimiento al servicio solicitado.

Interfaz



Entrada:

Salida: 

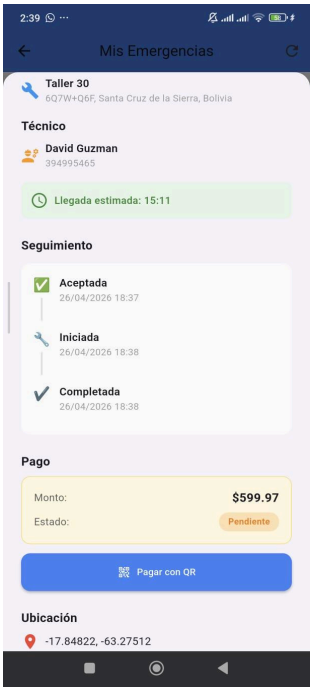
Caso de Prueba: CU9.Realizar pago del servicio

Caso de Uso	CU9. Realizar pago del servicio
Actores	Cliente
Entrada	El usuario selecciona: ● método de pago ● datos de referencia del pago, si aplica Luego presiona el botón **“Pagar por QR”**.
Condiciones	El usuario debe haber iniciado sesión. El servicio debe estar finalizado o habilitado para pago. Debe existir un monto generado para el incidente. El incidente no debe tener ya un pago confirmado.
Procedimiento	El usuario accede al detalle del servicio finalizado. El sistema muestra el monto a pagar.

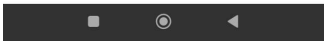
	El usuario selecciona el método de pago. Ingresa los datos requeridos según el método elegido. Presiona el botón **“Pagar servicio”**. El sistema valida la información del pago. El sistema procesa la transacción. El sistema registra el pago en la base de datos. El sistema confirma el pago exitoso.
Resultado Final	El pago queda registrado correctamente. El sistema almacena el método y referencia de pago. El incidente queda asociado a un pago confirmado. El usuario recibe confirmación del pago realizado.

Interfaz

Entrada:





Salida: 

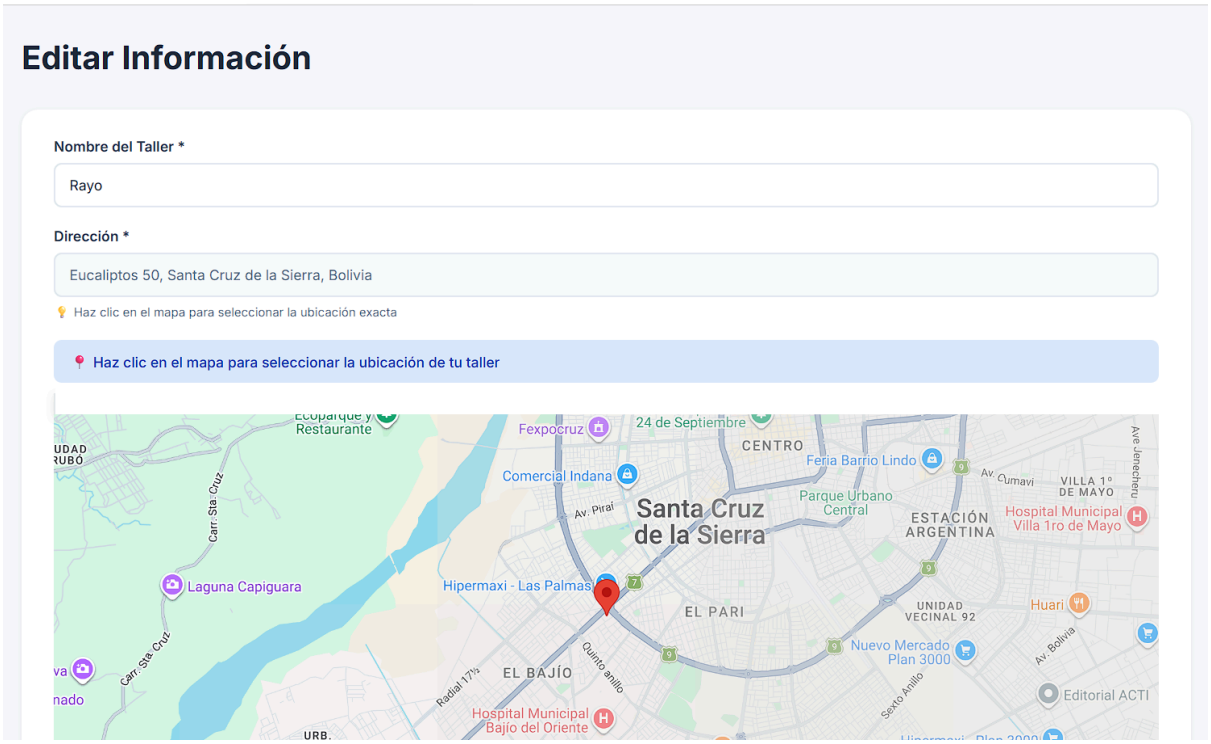
Caso de Prueba: CU10.Registrar taller

Caso de Uso	CU10. Registrar Taller
Actores	Administrador, Talleres
Entrada	El taller ingresa: ● nombre del taller ● dirección ● ubicación geográfica ● datos del propietario o responsable Luego presiona **“Registrar taller”**.
Condiciones	El taller no debe estar previamente registrado. Los datos ingresados deben ser válidos y completos. Si el flujo lo requiere, el administrador debe aprobar el registro.
Procedimiento	El usuario accede al formulario de registro de taller.

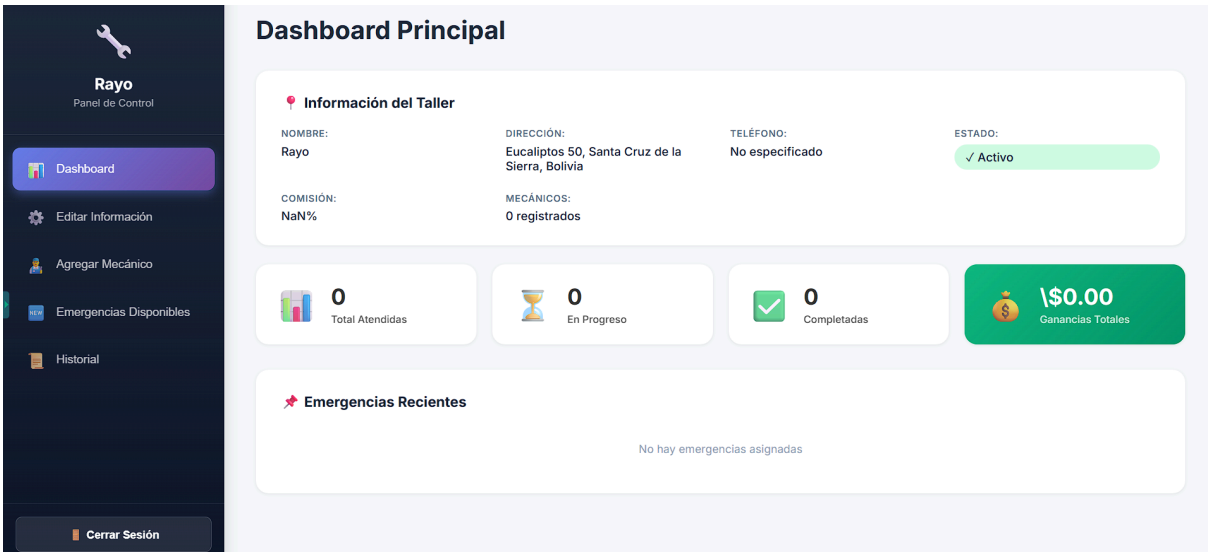
	Ingresa la información solicitada. El sistema valida que no exista otro taller con los mismos datos principales. El sistema valida la integridad de la información. Si corresponde, el administrador revisa la solicitud de registro. El sistema aprueba y almacena el taller en la base de datos. El sistema confirma el registro del taller.
Resultado Final	El taller queda registrado correctamente en el sistema. El taller puede ser habilitado como proveedor de servicios. La información queda disponible para futuras asignaciones.

Interfaz

Entrada:



Salida:



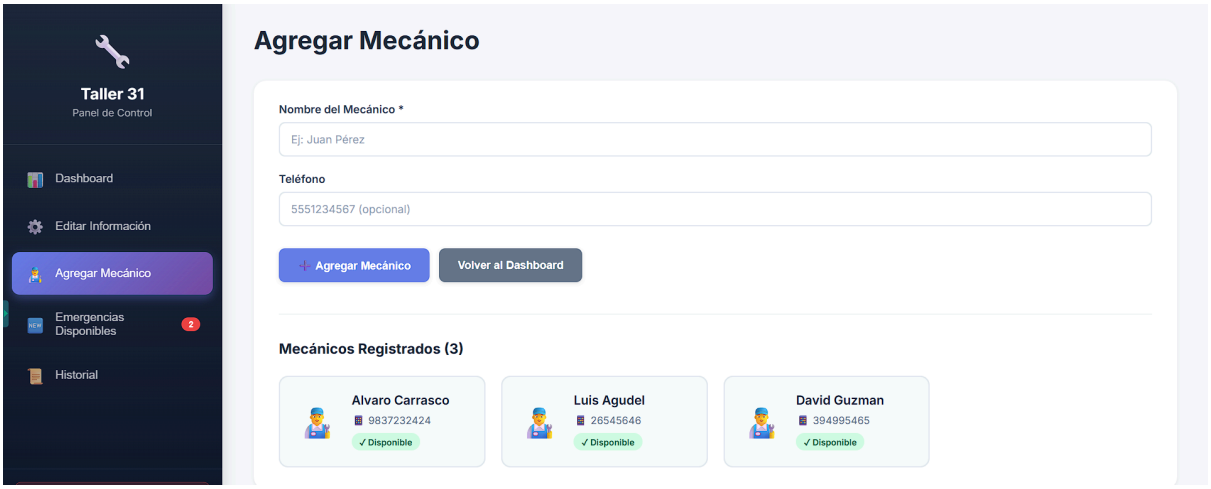
Caso de Prueba: CU11.Gestionar técnicos del taller

Caso de Uso	CU11. Gestionar técnicos del Taller
Actores	Talleres
Entrada	El taller registra, edita o elimina información de un técnico: • nombre • teléfono • estado del técnico Presiona la opción correspondiente: **guardar**, **editar** o **eliminar**.
Condiciones	El taller debe haber iniciado sesión. El taller debe estar registrado en el sistema. Los datos del técnico deben cumplir con el formato requerido.
Procedimiento	El taller accede al módulo de gestión de técnicos. Selecciona la opción de registrar, editar o eliminar técnico. Ingresa o modifica los datos del técnico. El sistema valida la información.

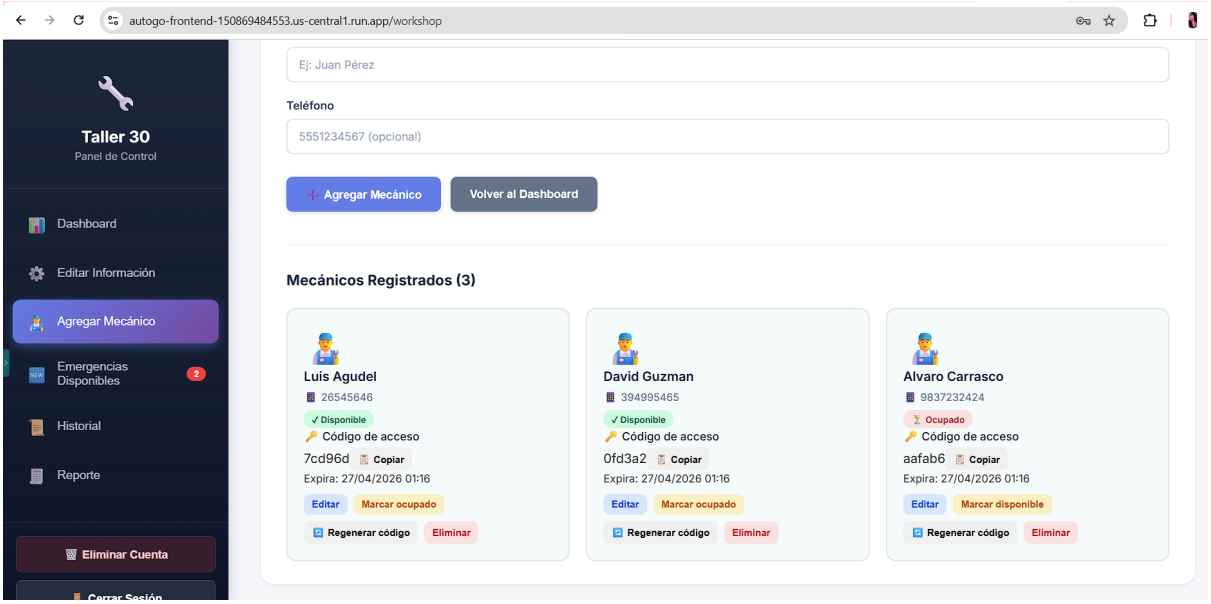
	El sistema guarda los cambios en la base de datos. El sistema actualiza la lista de técnicos del taller.
Resultado Final	El técnico queda registrado, actualizado o eliminado correctamente. La información del equipo técnico del taller se mantiene actualizada. El taller puede usar estos datos para futuras asignaciones.

Interfaz

Entrada:



Salida:

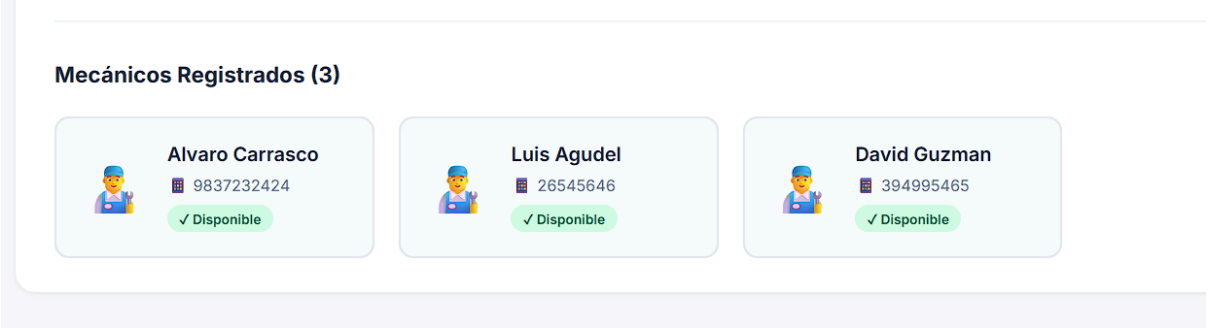


Caso de Prueba: CU12.Gestionar disponibilidad de técnicos

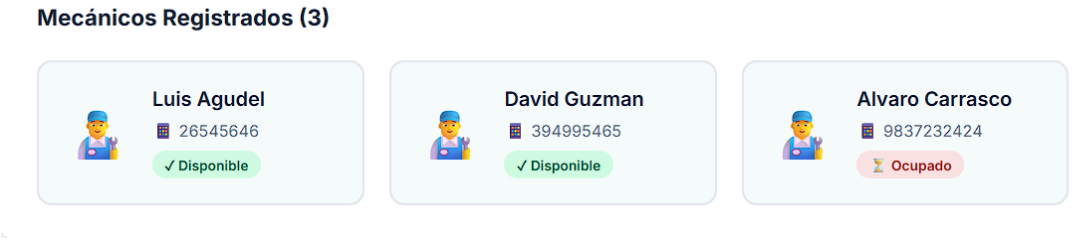
Caso de Uso	CU12. Gestionar Disponibilidad de Técnicos
Actores	Talleres
Entrada	El taller selecciona un técnico y cambia su disponibilidad a: ● disponible ● no disponible
Condiciones	El taller debe haber iniciado sesión. El técnico debe existir y pertenecer al taller autenticado. El técnico debe estar registrado en el sistema.
Procedimiento	El taller accede al módulo de disponibilidad. El sistema muestra la lista de técnicos registrados. El taller selecciona un técnico. Cambia su estado de disponibilidad. El sistema valida la operación. El sistema actualiza la disponibilidad del técnico en la base de datos. El sistema muestra el nuevo estado actualizado.
Resultado Final	La disponibilidad del técnico queda actualizada correctamente. El sistema podrá considerar ese estado al momento de asignar incidentes. El taller visualiza el cambio reflejado en la interfaz.

Interfaz

Entrada:



Salida:



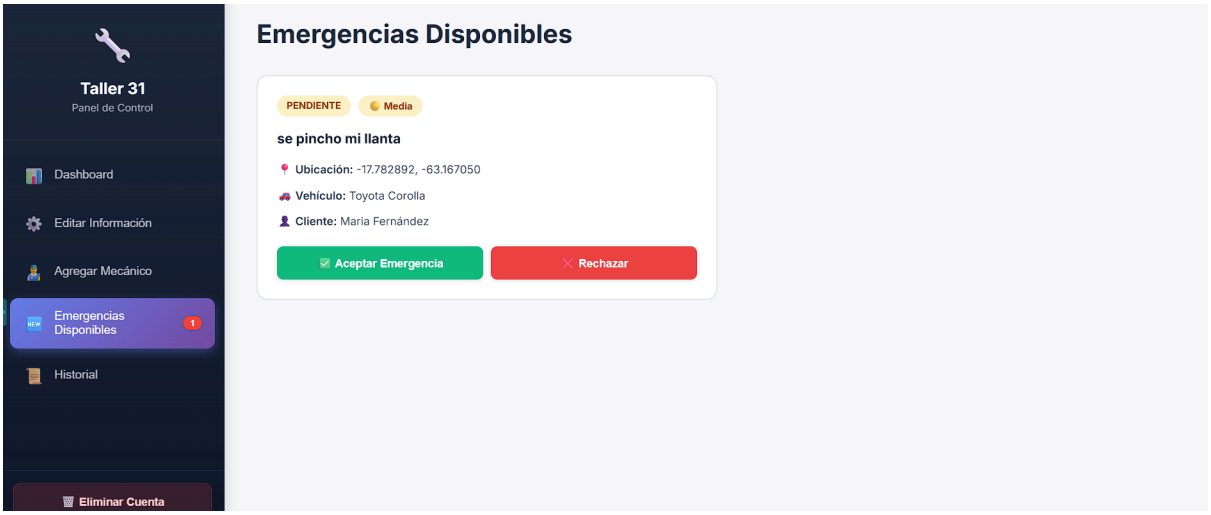
Caso de Prueba: CU13.Gestionar solicitudes (Taller)

Caso de Uso	CU13. Gestionar Solicitudes
Actores	Talleres
Entrada	El taller accede a las solicitudes disponibles y selecciona una opción: • aceptar solicitud • rechazar solicitud
Condiciones	El taller debe haber iniciado sesión. Deben existir solicitudes registradas en el sistema. La solicitud no debe haber sido asignada previamente a otro taller.
Procedimiento	El taller accede al módulo de solicitudes. El sistema muestra las solicitudes de emergencia disponibles. El taller revisará la información del incidente.

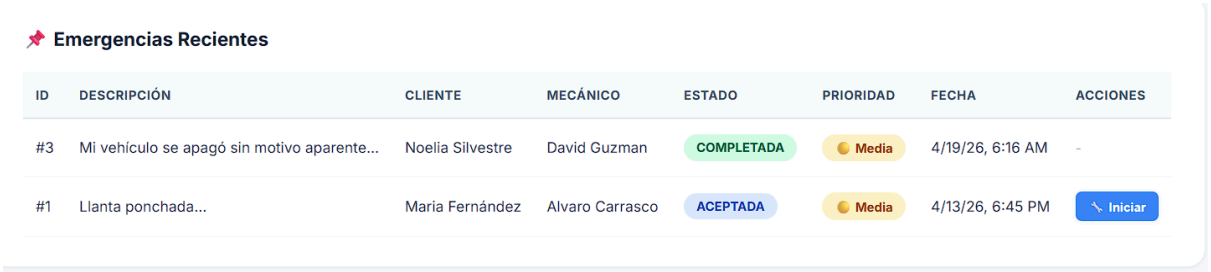
	El taller decide aceptar o rechazar la solicitud. El sistema valida que la solicitud siga disponible. El sistema registra la decisión tomada. Si la solicitud es aceptada, el sistema actualiza su estado. El sistema notifica el resultado correspondiente.
Resultado Final	La solicitud queda aceptada o rechazada correctamente. El sistema registra la decisión del taller. Si fue aceptada, la solicitud continúa con el proceso de atención.

Interfaz

Entrada:



Salida:

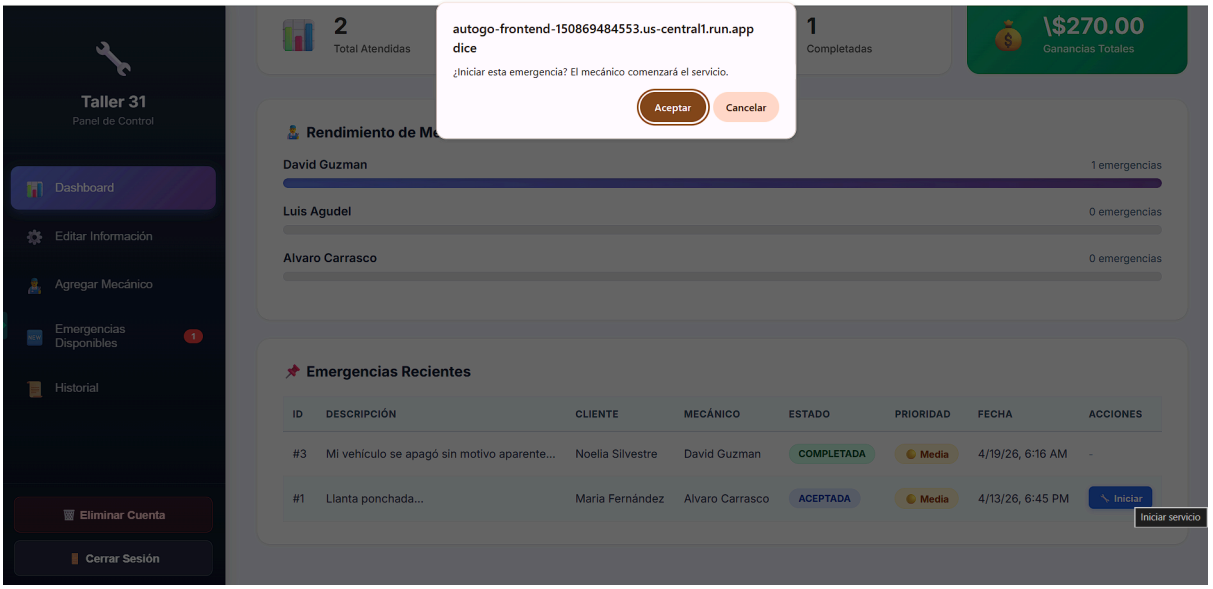


Caso de Prueba: CU14.Actualizar estado del servicio

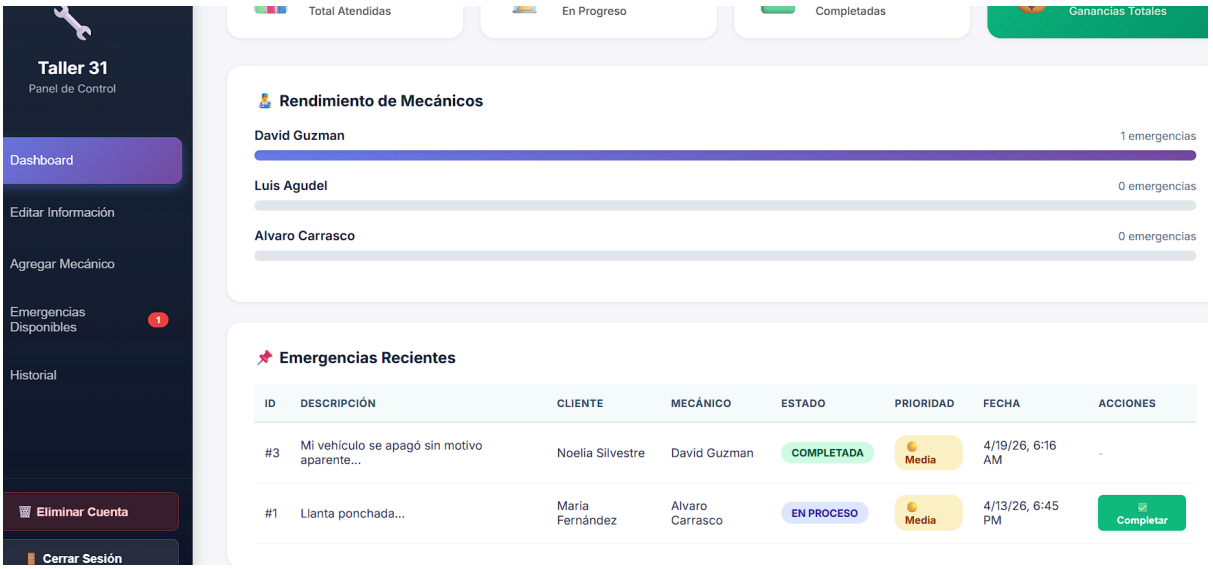
Caso de Uso	CU14. Actualizar estado del servicio
Actores	Administrador, Talleres
Entrada	El actor selecciona un servicio y cambia su estado a: ● en proceso ● finalizado
Condiciones	Debe existir una solicitud previamente aceptada. El actor debe estar autorizado para actualizar el servicio. El servicio debe existir en la base de datos.
Procedimiento	El usuario accede al detalle del servicio asignado. Selecciona la opción de actualizar estado. Elige el nuevo estado del servicio. El sistema valida que el cambio sea permitido. El sistema actualiza el estado en la base de datos. El sistema registra el cambio en el historial del incidente. El sistema notifica al cliente sobre la actualización.
Resultado Final	El estado del servicio queda actualizado correctamente. El historial del incidente registra el cambio realizado. El cliente visualiza la actualización del estado en su solicitud.

Interfaz

Entrada:



Salida:



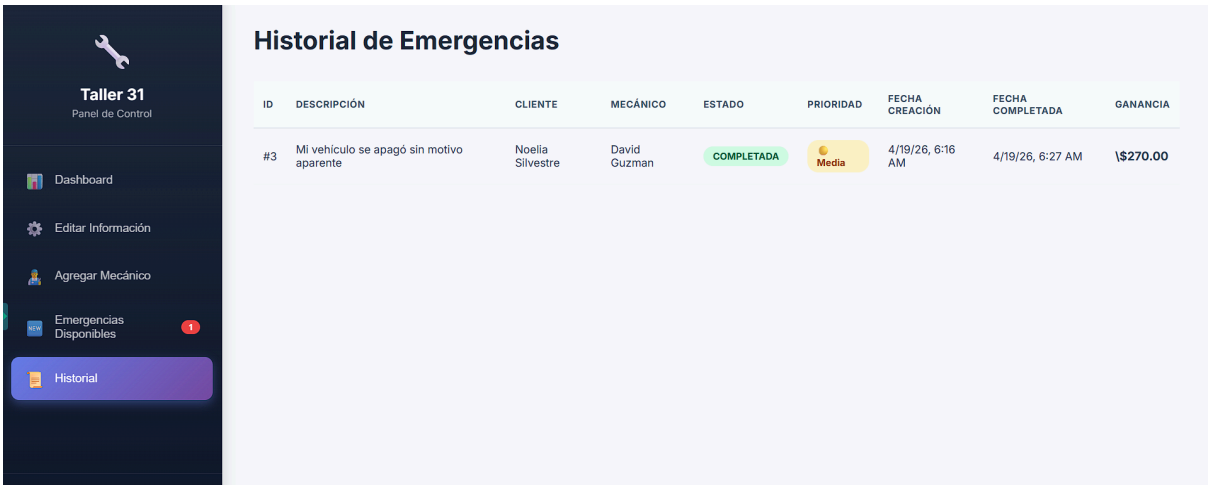
Caso de Prueba: CU15. Consultar historial de atenciones

Caso de Uso	CU15. Consultar historial de atenciones
Actores	Talleres
Entrada	El taller accede al módulo **“Historial de atenciones”**.
Condiciones	El taller debe haber iniciado sesión.

	Deben existir servicios previos registrados para ese taller.
Procedimiento	El taller accede al módulo de historial. El sistema consulta las atenciones registradas asociadas al taller. El sistema recupera los datos históricos de incidentes atendidos. El sistema muestra la lista de servicios realizados. El taller selecciona una atención específica si desea ver más detalle. El sistema presenta la información detallada del servicio seleccionado.
Resultado Final	El taller visualiza correctamente el historial de atenciones. El sistema muestra los servicios realizados con su información relevante. El taller puede consultar detalles de servicios anteriores.

Interfaz

Entrada:



Salida:



Caso de Prueba: CU16.Generar lista de talleres candidatos

Caso de Uso	CU16. Generar lista de talleres candidatos
Actores	Cliente
Entrada	El sistema recibe desde la emergencia registrada: • ubicación del incidente • tipo de problema • prioridad • disponibilidad de talleres y técnicos
Condiciones	Debe existir una emergencia registrada correctamente. Deben existir talleres activos registrados en el sistema. El sistema debe contar con información de ubicación y disponibilidad.
Procedimiento	El cliente registra una emergencia. El sistema recibe los datos del incidente. El sistema analiza la ubicación del cliente. El sistema analiza el tipo de problema reportado. El sistema consulta los talleres activos y la disponibilidad de técnicos.

	El sistema calcula distancias y prioridad de atención. El sistema genera una lista ordenada de talleres candidatos. El sistema selecciona los talleres más adecuados para atender el incidente.
Resultado Final	El sistema genera correctamente la lista de talleres candidatos. Los talleres seleccionados cumplen con criterios de cercanía, disponibilidad y capacidad. La emergencia puede continuar con el proceso de asignación.

Interfaz

Entrada:





Salida: 

Caso de Prueba: CU17.Gestionar Usuarios

Caso de Uso	CU17.Gestionar Usuarios
Actores	Administrador
Entrada	El administrador accede al módulo de usuarios y selecciona una acción: ● visualizar clientes ● editar clientes ● eliminar cliente
Condiciones	El administrador debe haber iniciado sesión. El administrador debe tener permisos para gestionar usuarios. El usuario a gestionar debe existir en el sistema.
Procedimiento	El administrador accede al módulo de gestión de usuarios. El sistema muestra la lista de usuarios registrados. El administrador selecciona un usuario. El administrador decide si desea visualizar, editar o eliminar la cuenta.

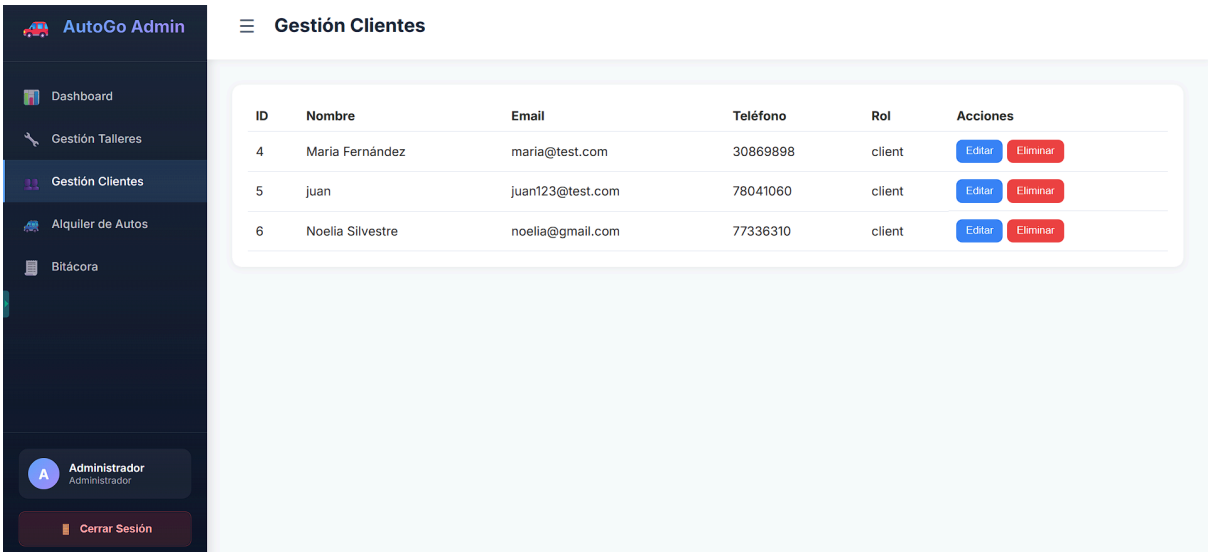
	Si edita, modifica los datos correspondientes del usuario. El sistema valida la información ingresada. El sistema guarda los cambios o elimina el registro según la acción elegida. El sistema actualiza la lista de usuarios.
Resultado Final	El administrador gestiona correctamente las cuentas de usuario. Los cambios realizados quedan registrados en el sistema. La información de usuarios se mantiene actualizada.

Interfaz

Entrada:



Salida:



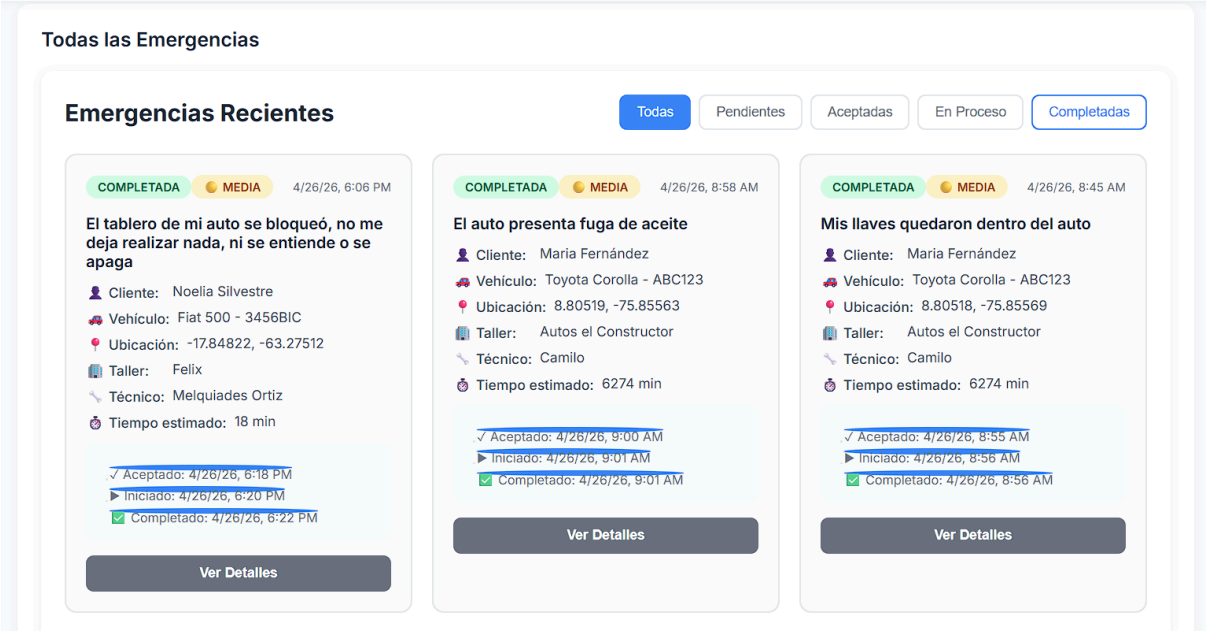
Caso de Prueba: CU19. Gestionar Incidentes

Caso de Uso	CU19. Gestionar Incidentes
Actores	Administrador
Entrada	El administrador accede al módulo de incidentes y selecciona un incidente para: • visualizar detalle • realizar seguimiento • intervenir en el proceso
Condiciones	El administrador debe haber iniciado sesión. Deben existir incidentes registrados en el sistema. El incidente seleccionado debe existir en la base de datos.
Procedimiento	El administrador accede al módulo de incidentes. El sistema muestra la lista de incidentes registrados. El administrador selecciona un incidente específico. El sistema recupera la información detallada del incidente. El administrador revisa estado, prioridad, taller asignado y técnico

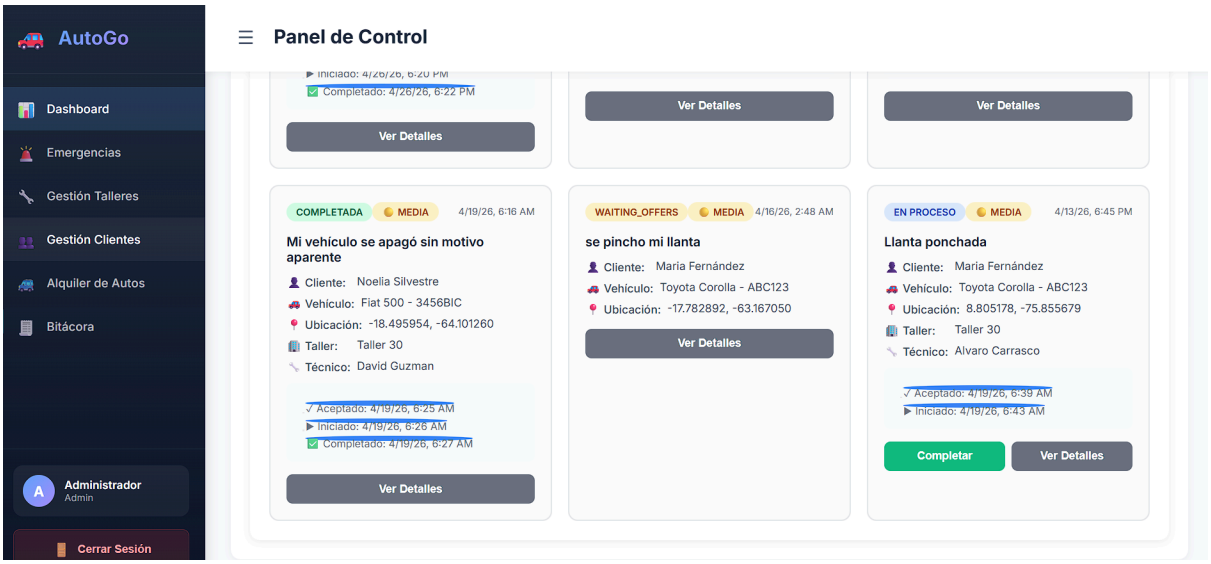
	asignado, si corresponde. El administrador realiza seguimiento del caso o interviene si es necesario. El sistema registra la acción o actualización correspondiente. El sistema muestra la información actualizada del incidente.
Resultado Final	El administrador supervisa correctamente los incidentes registrados. El sistema muestra la información detallada y actualizada del incidente. El incidente queda monitoreado y, si corresponde, intervenido por el administrador.

Interfaz

Entrada:



Salida:



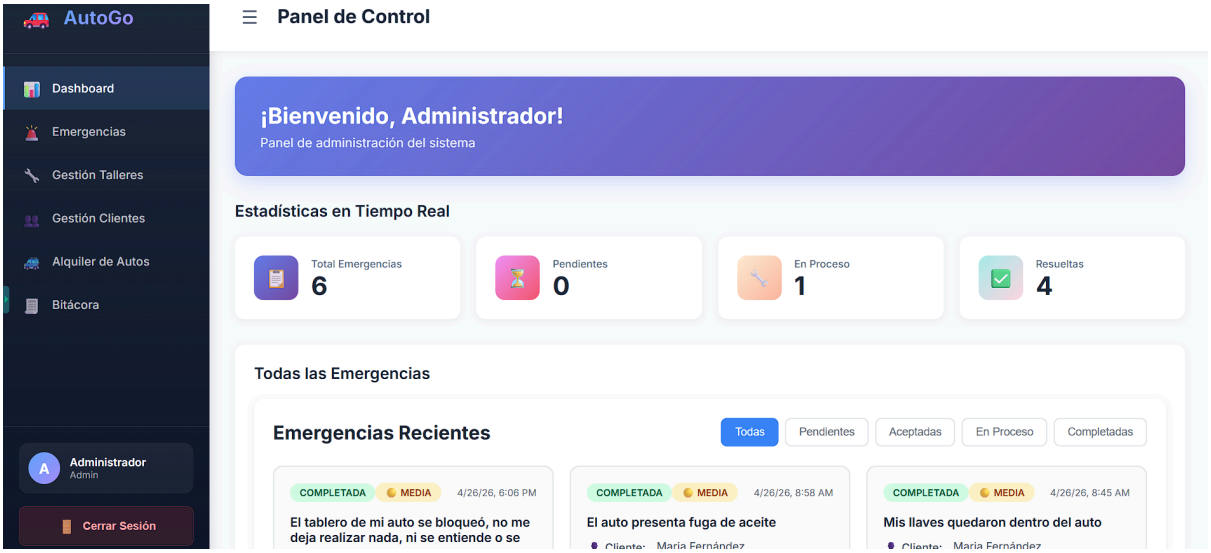
Caso de Prueba: CU20. Bitácora del Sistema

Caso de Uso	CU20. Bitácora del Sistema
Actores	Administrador
Entrada	El administrador accede al módulo de bitácora del sistema para consultar: • accesos al sistema • cambios realizados • eventos importantes registrados
Condiciones	El administrador debe haber iniciado sesión. El sistema debe encontrarse en funcionamiento. Deben existir eventos previamente registrados en la bitácora.
Procedimiento	El sistema registra automáticamente los eventos relevantes durante la operación. El administrador accede al módulo de bitácora. El sistema consulta los registros almacenados. El sistema muestra la lista de eventos del sistema.

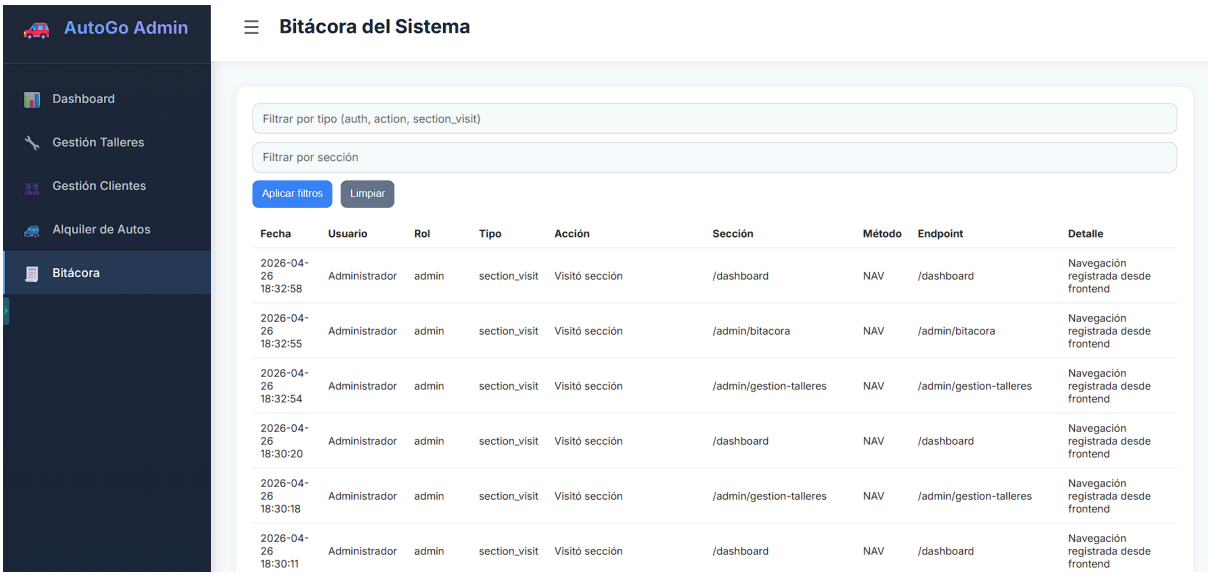
	El administrador revisa accesos, cambios y acciones registradas. Si lo desea, selecciona un evento específico para ver más detalle. El sistema presenta la información detallada del evento seleccionado.
Resultado Final	El administrador consulta correctamente la bitácora del sistema. El sistema muestra los eventos registrados de manera ordenada. La información de auditoría queda disponible para control y seguimiento.

Interfaz

Entrada:



Salida:


AutoGo Admin
Dashboard
Gestión Talleres
Gestión Clientes
Alquiler de Autos
Bitácora
Bitácora del Sistema
Filtrar por tipo (auth, action, section_visit)
Filtrar por sección
Aplicar filtros
Limpiar
Fecha Usuario Rol Tipo Acción Sección Método Endpoint Detalle
2026-04-26 18:32:58 Administrador admin section_visit Visitó sección /dashboard NAV /dashboard Navegación registrada desde frontend
2026-04-26 18:32:55 Administrador admin section_visit Visitó sección /admin/bitacora NAV /admin/bitacora Navegación registrada desde frontend
2026-04-26 18:32:54 Administrador admin section_visit Visitó sección /admin/gestion-talleres NAV /admin/gestion-talleres Navegación registrada desde frontend
2026-04-26 18:30:20 Administrador admin section_visit Visitó sección /dashboard NAV /dashboard Navegación registrada desde frontend
2026-04-26 18:30:18 Administrador admin section_visit Visitó sección /admin/gestion-talleres NAV /admin/gestion-talleres Navegación registrada desde frontend
2026-04-26 18:30:11 Administrador admin section_visit Visitó sección /dashboard NAV /dashboard Navegación registrada desde frontend

CONCLUSIÓN

El desarrollo del sistema de información para la gestión de servicios de auxilio vehicular permitió integrar de manera eficiente múltiples componentes tecnológicos, incluyendo aplicaciones web y móviles, un backend robusto y una base de datos estructurada. A través del uso de inteligencia artificial, geolocalización y procesamiento de datos multimodales, se logró optimizar el proceso de atención de emergencias, desde el registro del incidente hasta la asignación del taller más adecuado.

También, la implementación de casos de uso bien definidos, junto con el diseño de arquitectura y modelos de datos, garantizó una solución escalable, organizada y orientada a cubrir las necesidades tanto de los usuarios como de los talleres. El sistema no solo mejora los tiempos de respuesta ante emergencias, sino que también proporciona trazabilidad, control y transparencia en cada etapa del servicio.

La propuesta desarrollada constituye una solución integral e innovadora que contribuye significativamente a mejorar la eficiencia, calidad y confiabilidad en la gestión de servicios de auxilio vehicular.

RECOMENDACIÓN

Se recomienda continuar con la evolución del sistema incorporando mejoras en aspectos como la seguridad, el rendimiento y la experiencia del usuario. Es importante fortalecer los mecanismos de autenticación y protección de datos, considerando el manejo de información sensible dentro de la plataforma.

Además, se sugiere optimizar y entrenar continuamente los modelos de inteligencia artificial para mejorar la precisión en la clasificación de incidentes y la asignación de talleres. También sería beneficioso implementar pruebas más exhaustivas en entornos reales, con el fin de validar el comportamiento del sistema bajo condiciones de uso reales.

BIBLIOGRAFÍA

- FastAPI. (2024). *FastAPI Documentation*. Recuperado de: https://fastapi.tiangolo.com/
- Angular. (2024). *Angular Official Documentation*. Recuperado de: https://angular.io/
- Flutter. (2024). *Flutter Documentation*. Recuperado de: https://docs.flutter.dev/
- PostgreSQL. (2024). *PostgreSQL Documentation*. Recuperado de: https://www.postgresql.org/docs/
- Python Software Foundation. (2024). *Python Documentation*. Recuperado de: https://docs.python.org/3/
- TypeScript. (2024). *TypeScript Documentation*. Recuperado de: https://www.typescriptlang.org/docs/
- Dart. (2024). *Dart Language Documentation*. Recuperado de: https://dart.dev/guides
- Software Engineering. Sommerville, I. (2016). *Software Engineering* (10th ed.). Pearson.
- Ingeniería de Software. Pressman, R. (2014). *Ingeniería del Software: Un enfoque práctico*. McGraw-Hill.
- Artificial Intelligence: A Modern Approach. Russell, S., & Norvig, P. (2021). *Artificial Intelligence: A Modern Approach*. Pearson.
- Google Developers. (2024). *Machine Learning Crash Course*. Recuperado de: https://developers.google.com/machine-learning
- Mozilla Developer Network (MDN). (2024). *Web Docs*. Recuperado de: https://developer.mozilla.org/
- OpenAI. (2024). *Documentación de modelos de lenguaje e inteligencia artificial*.

URL Y QR

https://github.com/Stefy127/AUTOGO.git



Aplicación WEB
https://autogo-frontend-150869484553.us-central1.run.app/login
Aplicación MÓVIL
https://drive.google.com/drive/folders/1wWwIIYDduKNWDlSj_bMkrqSDHARc_IHa?usp=drive_link

Pestaña 2

Autenticación y gestión de usuarios

CU1: Registrarse en el sistema (cliente)

CU2: Iniciar sesión

CU3: Cerrar sesión

CU4: Gestionar perfil de usuario

Gestión de vehículos

CU5: Registrar vehículo

CU6: Gestionar información del vehículo

Gestión de emergencias

CU7: Gestionar Emergencias

- Registrar emergencia vehicular
- Enviar ubicación en tiempo real
- Adjuntar evidencias (imagen, audio, texto)

CU8: Consultar estado de la solicitud

- Visualizar taller asignado
- Ver tiempo estimado de llegada

📌 (Esto viene directamente del módulo móvil del cliente)

Pagos

CU9: Realizar pago del servicio

Gestión de talleres (Aplicación Web)

CU10: Registrar taller

CU11: Gestionar técnicos del taller

CU12: Gestionar disponibilidad de técnicos

Gestión de solicitudes (Taller)

CU13: Gestionar solicitudes (Taller)

Visualizar solicitudes disponibles

Ver información del incidente

Aceptar solicitud

Rechazar solicitud

CU14: Actualizar estado del servicio

CU15: Consultar historial de atenciones

📌 (Basado en funcionalidades del taller)

Asignación inteligente

CU16: Generar lista de talleres candidatos

Administración

CU17: Gestionar usuarios

CU18: Gestionar talleres

CU19: Gestionar incidentes

CU20: Bitacora del Sistema

CU1. Registrarse en el sistema (cliente)

CU2. Iniciar sesión

CU3. Cerrar sesión

CU4. Gestionar perfil de usuario

CU5. Registrar vehículo

CU6. Gestionar información del vehículo

CU7. Gestionar emergencias

CU8. Consultar estado de la solicitud

CU9. Realizar pago del servicio

CU10. Registrar taller

CU11. Gestionar técnicos del taller

CU12. Gestionar disponibilidad de técnicos

CU13. Gestionar solicitudes (Taller)

CU14. Actualizar estado del servicio

CU15. Consultar historial de atenciones

CU16. Generar lista de talleres candidatos

CU17. Gestionar usuarios

CU18. Gestionar talleres
CU19. Gestionar incidentes

CU20. Bitácora del sistema

paquete Autenticación y Seguridad
paquete Gestion de Usuarios
paquete Gestion de Vehiculos
paquete Gestión de Emergencias
paquete Gestión de Solicitudes
paquete Gestión de Talleres
paquete Gestión de Pagos
paquete Asignación Inteligente
paquete Administración del Sistema